СОДЕРЖАНИЕ

СТРУКТУРНЫЙ АНАЛИЗ и ПРОЕКТИРОВАНИЕ 2

1. Понятие структурного анализа 2

1.1. Структурный подход: основы, особенности 2
1.2. Принципы структурного анализа 4
1.3. Классификация структурных методологий 4
1.4. Средства структурного анализа 6

2. Анализ требований 8

2.1. Функциональные модели 8
2.1.1. DFD – диаграммы потоков данных 8
2.1.1.2. Словарь данных 20
2.1.1.3. Методы задания спецификации процессов 23
2.1.2. SADT – диаграммы моделирования бизнес-процессов 33
Общие свойства модели 35
Модель отвечает на вопросы 35
Модель имеет единственный субъект 35
У модели может быть только одна точка зрения 36
Иерархия диаграмм 36
Отличия DFD и DEF0 37
2.2. Модели данных 44
2.2.1. Описание методологий IDEF1X и IE 44
Типы сущностей 45
Мощность связи 46
Типы связей 46
Неспецифические связи 49
Связи категоризации в IDEF1X 49
Обязательные и необязательные неидентифицирующие связи 51
Ключи 52
2.2.2. Нотация IDEF1X 52
2.2.2. Расширенные функции ERWin 53
Синхронизация с базой данных 54
Программирование триггеров и процедур 54
Правила и начальные значения 55
Домены 55
Связывание данных 55
2.3. Модели управления 57
2.3.1. Диаграмма потоков управления (CFD) 59
2.3.2. Спецификации управления 59

Общий порядок построения модели требований 65

3. Проектирование 68

3.1. Задачи этапа проектирования. Модель реализации 68
3.2. Функциональные модели проектирования 69
3.2.1. Модель системных процессов (С.89 Вендров) 69
3.2.2. Модель пользовательского интерфейса 69
3.2.3. Структурные схемы программ (структурные карты) 70

Коллективная работа по созданию программного обеспечения 72

Инструментарий технологии разработки программного обеспечения. CASE-технология создания программного обеспечения. 74

Принципы построения, структура и технология использования систем автоматизированного проектирования и разработки программного обеспечения. 77

СТРУКТУРНЫЙ АНАЛИЗ и ПРОЕКТИРОВАНИЕ

Умение чесать левой ногой за правым ухом является показателем низкой квалификации человека как аналитика-программиста. Аналитик должен уметь найти способ и инструмент, чтобы решение оказалось естественным.

Из книги Н.Н.Непейвода, И.Н.Скопин «Основания программирования»

1. Понятие структурного анализа

Структурным анализом принято называть метод исследования системы, который начинается с ее общего обзора и затем детализируется, приобретая иерархическую структуру со все большим числом уровней.

1.1. Структурный подход: основы, особенности

Структурный подход к разработке систем получил широкое распространение (и был признан стандартом де-факто) в 1980-х годах. Основой подхода чаще всего являются методики описания бизнес процессов с помощью диаграмм потоков данных (DFD), диаграмм структурного анализа и проектирования (SADT), диаграмм, изображающих потоки работ (IDEF3), а также диаграмм сущность-связь (entity relationship diagrams – ERD) для моделирования данных. В силу определенных особенностей, о которых будет сказано позднее, в России наиболее распространена методология DFD для описания бизнес-процессов организации и ERD для моделирования данных.

Структурный подход является функционально-ориентированным и рассматривает DFD-диаграммы в качестве движущей силы разработки проекта системы и в дальнейшем ее программной реализации. Ориентация на функции означает, что в процессе разработки проекта системы выделяются функции, являющиеся условно независимыми частями системы. Это позволяет с одной стороны рассматривать их как независимые друг от друга части системы, а с другой стороны создавать из них более крупные фрагменты системы за счет взаимодействия функций.

Позднее, в качестве одного из непосредственных результатов широкого распространения моделей реляционных баз данных, значение DFD-диаграмм в структурной разработке снизилось, и подход стал более ориентированным на данные, и, соответственно, акцент в разработке сместился на ERD-диаграммы.

Сочетание DFD- и ERD-диаграмм дает относительно полные модели анализа, которые фиксируют все функции и данные системы на требуемом уровне абстракции независимо от особенностей аппаратного и программного обеспечения. Затем модель анализа преобразу-

ется в проектную модель, которая обычно выражается в понятиях реляционных баз данных. После этого следует этап реализации.

Структурный подход к анализу и проектированию отличается рядом особенностей, некоторые из них не очень хорошо увязываются с современными методами конструирования ПО.

- Этот подход скорее является **последовательным и трансформационным**, чем итеративным подходом с наращиванием возможностей (т.е. этот подход не способствует непрерывному процессу разработки, осуществляемому посредством итеративной детализации и пошаговой поставки ПО с наращенными возможностями).
- Этот подход направлен на поставку негибких решений, которые способны удовлетворить набор определенных бизнес-функций, но которые может быть трудно, масштабировать и расширять в будущем,·
- Этот подход предполагает разработку «с чистого листа» и не поддерживает повторное использование уже существующих компонент.

Трансформационный характер структурного подхода является источником повышенного риска неправильно истолковать исходные требования пользователей по ходу разработки. Такой риск усиливается за счет необходимости постепенной замены относительно декларативной семантики моделей анализа на процедурные решения для проектных моделей и программного кода реализации (это является следствием того, что модели анализа семантически богаче, чем базовые проектные и реализационные модели).

Методы структурного анализа и проектирования стремятся преодолеть сложность больших систем путем расчленения их на части («черные ящики») и иерархической организации этих черных ящиков. Выгода в использовании черных ящиков заключается в том, что их пользователю не требуется знать, как они работают, необходимо знать лишь его входы и выходы, а также его назначение (т.е. функцию, которую он выполняет).

Второй важной идеей, лежащей в основе структурных методов, является идея иерархии. Для понимания сложной системы недостаточно разбиения ее на части, необходимо эти части организовать определенным образом, а именно в виде иерархических структур. Все сложные системы Вселенной организованы в иерархии. Да и сама она включает галактики, звездные системы, планеты, …, молекулы, атомы, элементарные частицы. Человек при создании сложных систем также подражает природе. Любая организация имеет директора, заместителей по направлениям, иерархию руководителей подразделений, рядовых служащих.

Наконец, третий момент: структурные методы широко используют графические нотации, также служащие для облегчения понимания сложных систем.

1.2. Принципы структурного анализа

Проблема сложности, о которой мы уже говорили во введении, является главной проблемой, которую приходится решать при создании больших и сложных систем любой природы, в том числе и ИС. Основной принцип управления любой сложной системой был известен давно: "divide et impera" - "разделяй и властвуй". Методы структурного анализа и проектирования стремятся преодолеть сложность больших систем путем расчленения их на части. Сложная программная система на верхнем уровне должна состоять из небольшого числа относительно независимых компонентов с четко определенными интерфейсами. Затем декомпозиции подвергаются выделенные на первом этапе компоненты, и так далее до заданного уровня детализации. Таким образом, система представляется иерархией с несколькими уровнями абстракции.

При этом автоматизируемая система сохраняет целостное представление, в котором все составляющие компоненты взаимоувязаны. При разработке системы "снизу-вверх" от отдельных задач ко всей системе целостность теряется, возникают проблемы при информационной стыковке отдельных компонентов.

Таким образом, базовыми принципами структурного подхода являются:

- принцип "разделяй и властвуй"
- принцип иерархического упорядочения, когда составные части системы организуются в иерархические древовидные структуры.

Примечание. Выделение двух базовых принципов не означает, что остальные принципы являются второстепенными, поскольку игнорирование любого из них может привести к непредсказуемым последствиям (в том числе и к провалу всего проекта). Основными из этих принципов являются следующие:

- принцип *абстрагирования* - заключается в выделении существенных аспектов системы и отвлечения от несущественных;
- принцип *формализации* - заключается в необходимости строгого методического подхода к решению проблемы;
- принцип *непротиворечивости* - заключается в обоснованности и согласованности элементов;
- принцип *структурирования данных* - заключается в том, что данные должны быть структурированы и иерархически организованы.

1.3. Классификация структурных методологий

Метод – совокупность приемов или операций для получения искомого результата.
Методология – совокупность методов, применяемых в какой-либо науке.

Роль методологии заключается в регламентации процесса разработки сложных систем. Она описывает последовательность шагов, модели и подходы, тщательное следование которым приведет к хорошо работающим системам.

Структурные методологии жестко регламентируют фазы анализа требований и проектирования спецификаций и отражают подход к разработке ПО с позиций рецептов "кулинарной книги".

Обратите внимание! Методология охватывает не только этап анализа, но и другие этапы. И в первую очередь этап проектирования.

В настоящее время наибольшее распространение получили методологии:

- SADT (Structured Analysis and Design Technique),
- структурного системного анализа Гейна-Сарсона (Gane-Sarson),
- структурного анализа и проектирования Йодана Де Марко (Yourdon/DeMarko),
- развития систем Джексона (Jackson),
- развития структурных систем Варнье-Орра (Wamier-Orr),
- анализа и проектирования систем реального ремени Уорда-Меллора (Ward-Mellor) и Хатли (Hatley),

- информационного моделирования Мартина (Martin).

Несмотря на достаточно широкий спектр используемых методов и диаграммных техник, большинство методологий базируется на следующей "классической" совокупности:

- диаграммы потоков данных в нотации Йодана Де Марко или Гейна-Сарсона, обеспечивающие анализ требований и функциональное проектирование информационных систем;
- расширения Хатли и Уорда-Меллора для проектирования систем реального времени, основанные на диаграммах переходов состояний, таблицах и деревьях решений, картах и схемах потоков управления;
- диаграммы "сущность-связь" (в нотации Чена или Баркера) или скобочные диаграммы Варнье-Орра для проектирования структур данных,схем БД, форматов файлов как части всего проекта;
- структурные карты Джексона и/или Константайна для проектирования межмодульных взаимодействий и внутренней структуры модулей, позволяющие развить модель анализа, построенную на базе перечисленных средств, до модели реализации.

Современные структурные методологии по целевому использованию следует классифицировать:

1. на *системы реального времени* (СРВ)
2. на *информационные системы* (ИС)

Основная особенность систем реального времени заключается в том, что они контролируют и контролируются внешними событиями; реагирование на эти события во времени - основная и первоочередная функция таких систем. По отношению к СРВ говорят, что полученный поздно ответ приравнивается к неправильному ответу (по-английски «The right answer late is wrong»). Система реального времени не должна обязательно быть быстрой, это распространенное заблуждение. Система реального времени должна выдавать реакцию в ответ на информацию, поступающую на ее вход в гарантированные промежутки времени.

Главные отличия информационных систем от систем реального времени приведены в таблице 1, средствами поддержки этих особенностей и различаются соответствующие структурные методологии.

Таблица 1

Информационные системы	Системы реального времени
Управляемы данными	Управляемы событиями
Сложные структуры данных	Простые структуры данных
Большой объем входных данных	Малое кол-во входных данных
Интенсивный ввод/вывод	Интенсивные вычисления
Машинная независимость	Машинная зависимость

Примечание. Необходимо отметить, что для проектирования систем реального времени используются специальные типы структурных диаграмм: *диаграммы потоков управления, диаграммы переходов состояний, контекстные графы, матрицы состояний/событий, таблицы решений* и др. Однако многие из них являются вариациями структурных диаграмм для проектирования информационных систем. Более того, известные методологии проектирования систем реального времени (в частности, методологии Хатли и Уорда-

Меллора) базируются на перечисленных методологиях проектирования информационных систем, расширяя их соответствующими диаграммными техниками.

Информационно-управляющая система

Информационно-управляющая система (ИУС) – цифровая система контроля или управления некоторым реальным объектом, называемым обычно объект управления. На вход ИУС поступает информация с датчиков, на выходе ИУС вырабатывается управляющее воздействие посредством исполнительного устройства.

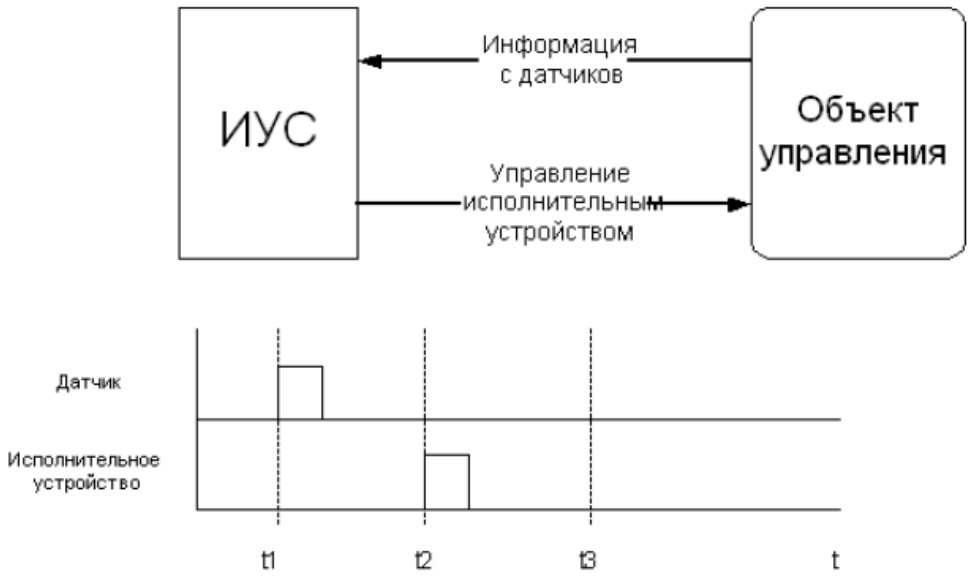


Рисунок 1. Взаимодействие ИУС и ОУ происходит в реальном масштабе времени

Информационно-управляющая система (или встроенная система) и система реального времени не являются синонимами. Отличие СРВ от ИУС состоит в том, что управляющая составляющая в СРВ не является обязательной. В класс СРВ может быть включена как информационно-управляющая система, так и система, относящаяся к информационным. Для примера к СРВ можно отнести компьютерные игры и системы резервирования авиабилетов.

1.4. Средства структурного анализа

Для целей моделирования систем вообще, и структурного анализа в частности, используются три группы средств, иллюстрирующих:

- функции, которые система должна выполнять;
- отношения между данными;
- зависящее от времени поведение системы (аспекты реального времени).

Соответственно во всех методологиях структурного анализа используются три группы средств моделирования:

1. диаграммы, иллюстрирующие функции, которые система должна выполнять, и связи между этими функциями. Для этой связи чаще всего используются DFD (Data Plow Diagrams - диаграммы потоков данных совместно со словарями данных и спецификациями процессов или миниспецификациями) или SADT;

2. диаграммы, моделирующие данные и их взаимосвязи (ERD, Entity-Relationship Diagrams - диаграммы "сущность-связь");
3. диаграммы, моделирующие поведение системы в зависимости от времени (диаграммы потоков управления CFD, диаграммы переходов состояний и т.п.).

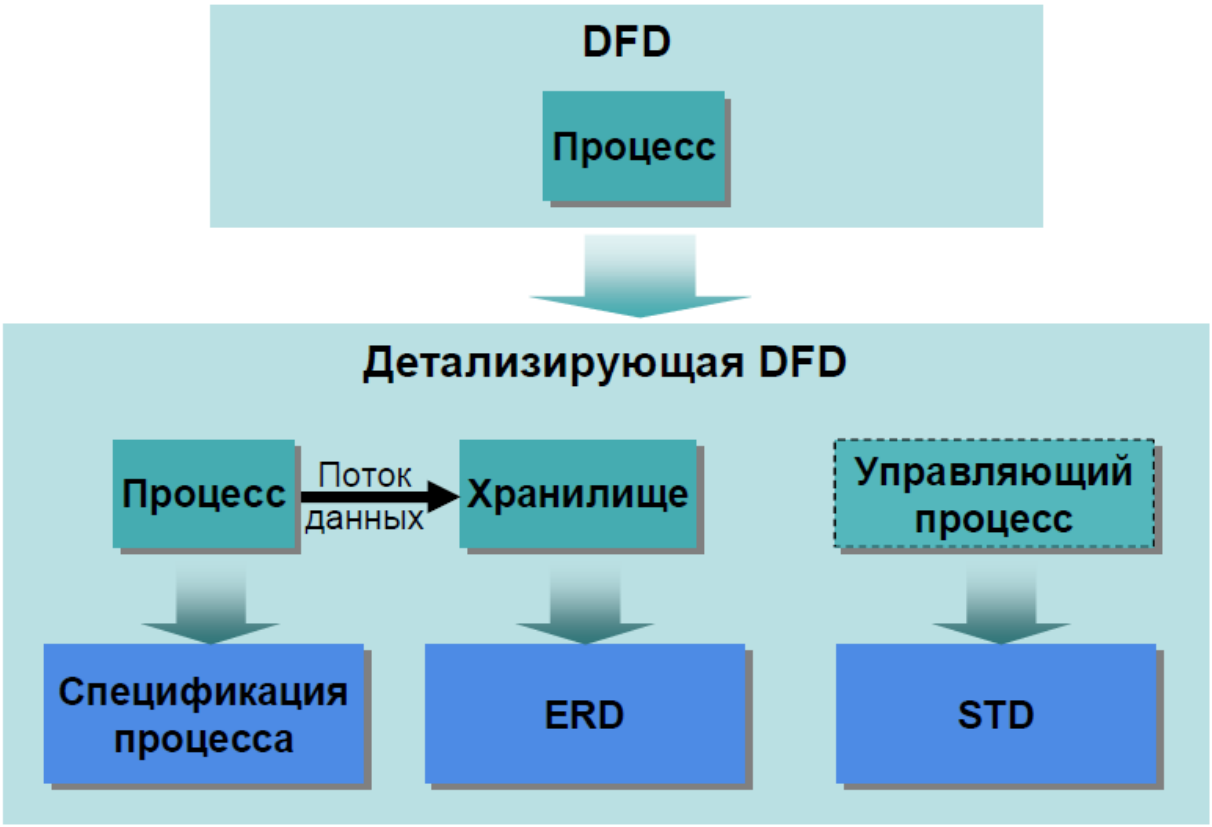


Рис. 2. ***Компоненты логической модели***

Логическая DFD показывает внешние по отношению к системе источники и стоки (адресаты) данных, идентифицирует логические функции (процессы) и группы элементов данных, связывающие одну функцию с другой(потоки), а также идентифицирует хранилища (накопители) данных, к которым осуществляется доступ. Структуры потоков данных и определения их компонент хранятся и анализируются в словаре данных. Каждая логическая функция (процесс) может быть детализирована с помощью DFD нижнего уровня; когда дальнейшая детализация перестает быть полезной, переходят к выражению логики функции при помощи спецификации процесса (миниспецификации). Содержимое каждого хранилища также сохраняют в словаре данных, модель данных хранилища раскрывается с помощью ERD. В случае наличия реального времени DFD дополняется средствами описания зависящего от времени поведения системы, раскрывающимися с помощью STD.

Эти связи показаны на рис.2.

Перечисленные средства дают полное описание системы независимо от того, является ли она существующей или разрабатываемой с нуля. Таким образом, строится логическая функциональная спецификация – подробное описание того, что должна делать система, освобожденное насколько это возможно от рассмотрения путей реализации. Это дает проектировщику четкое представление о конечных результатах, которые следует достигать.

2. Анализ требований

Анализ требований разрабатываемой системы является важнейшим среди всех этапов ЖЦ. Он оказывает существенное влияние на все последующие этапы, являясь в то же время наименее изученным и понятным процессом. На этом этапе, во-первых, необходимо понять, что предполагается сделать, а во-вторых, задокументировать это, т.к. если требования не зафиксированы и не сделаны доступными для участников проекта, то они вроде бы и не существуют. При этом язык, на котором формулируются требования, должен быть достаточно прост и понятен заказчику.

На фазе анализа строится *модель среды* (*Environmental Model*). Построение модели среды включает:

- **анализ поведения системы**: определение назначения ИС, построение начальной контекстной диаграммы потоков данных (DFD) и формирование матрицы списка событий (ELM), построение функциональных диаграмм;
- **анализ данных**: определение состава потоков данных и построение диаграмм структур данных или схем данных (DSD), конструирование логической модели данных в виде ER-диаграммы.

Рассмотрим вопросы построения функциональных моделей и логической модели данных.

2.1. Функциональные модели

Функциональные модели позволяют описать систему с точки зрения выполняемых ею функций. Подобные модели могут служить источником функциональных требований к проектируемой системе.

2.1.1. DFD – диаграммы потоков данных

Метод DFD основан на использовании трех взаимосвязанных спецификаций:

1. **диаграмм потоков данных**
2. **словаря данных**
3. **спецификации процессов**

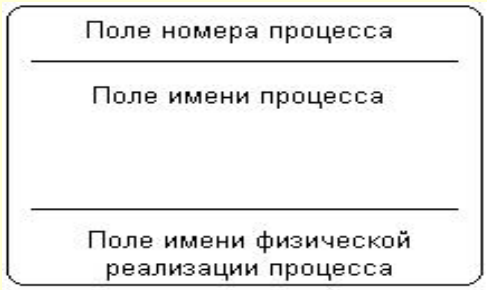
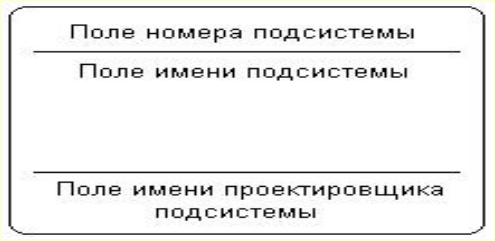
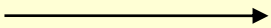
2.1.1.1. Диаграммы потоков данных

Основной элемент структурного анализа – *диаграмма потоков данных* Диаграмма потоков данных – графическое средство для изображения информационных потоков и преобразований, которым подвергаются данные от входа к выходу системы.

Основными компонентами DFD которой являются различные потоки данных, которые переносят информацию от одной подсистемы к другой. Каждая из подсистем выполняет определенные преобразования входного потока данных и передает результаты обработки информации в виде потоков данных для других подсистем. Поэтому DFD-модели удобно использовать для описания документооборота и обработки информации.

Для изображения DFD традиционно используются две различные нотации: Йодана и Гейна-Сарсона.

Нотация Йодана	Нотация Гейна-Сарсона	Описание

		Изображение внешней сущности на диаграмме потоков данных
		Изображение накопителя на диаграмме потоков данных
		Изображение процесса на диаграмме потоков данных
		Изображение подсистемы на диаграмме потоков данных
		Изображение потока данных

Элементы DFD-модели

Процесс (process) – процесс представляет собой преобразование входных потоков данных в выходные в соответствии с определенным алгоритмом. Физически процесс может быть реализован различными способами: это может быть подразделение организации (отдел), выполняющее обработку входных документов и выпуск отчетов, программа, аппаратно реализованное логическое устройство и т.д.

Номер процесса служит для его идентификации. В поле имени вводится наименование процесса в виде предложения с активным недвусмысленным глаголом в неопределенной форме (вычислить, рассчитать, проверить, определить, создать, получить), за которым следуют существительные в винительном падеже, например:

- "Ввести сведения о клиентах";
- "Выдать информацию о текущих расходах";
- "Проверить кредитоспособность клиента".

Использование таких глаголов, как "обработать", "модернизировать" или "отредактировать" означает, как правило, недостаточно глубокое понимание данного процесса и требует дальнейшего анализа.

Информация в поле физической реализации показывает, какое подразделение организации, программа или аппаратное устройство выполняет данный процесс.

Системы и подсистемы. При построении модели сложной ИС она может быть представлена в самом общем виде на так называемой контекстной диаграмме в виде одной системы как единого целого, либо может быть декомпозирована на ряд подсистем.
Номер подсистемы служит для ее идентификации. В поле имени вводится наименование подсистемы в виде предложения с подлежащим и соответствующими определениями и дополнениями

Внешний объект (external entity) – внешняя сущность представляет собой материальный предмет или физическое лицо, представляющее собой источник или приемник информации, например, заказчики, персонал, поставщики, клиенты, склад. Определение некоторого объекта или системы в качестве внешней сущности указывает на то, что она находится за пределами границ анализируемой ИС. В процессе анализа некоторые внешние сущности могут быть перенесены внутрь диаграммы анализируемой ИС, если это необходимо, или, наоборот, часть процессов ИС может быть вынесена за пределы диаграммы и представлена как внешняя сущность.

Внешняя сущность обозначается квадратом , расположенным как бы "над" диаграммой и бросающим на нее тень, для того, чтобы можно было выделить этот символ среди других обозначений.

Накопитель (store) – накопитель данных представляет собой абстрактное устройство для хранения информации, которую можно в любой момент поместить в накопитель и через некоторое время извлечь, причем способы помещения и извлечения могут быть любыми. Накопитель данных идентифицируется буквой "D" и произвольным числом. Имя накопителя выбирается из соображения наибольшей информативности для проектировщика. Накопитель данных может быть реализован физически в виде микрофиши, ящика в картотеке, таблицы в оперативной памяти, файла на магнитном носителе и т.д.

Накопитель данных в общем случае является прообразом будущей базы данных и описание хранящихся в нем данных должно быть увязано с информационной моделью.

Поток (flow) – поток данных определяет информацию, передаваемую через некоторое соединение от источника к приемнику. Реальный поток данных может быть информацией, передаваемой по кабелю между двумя устройствами, пересылаемыми по почте письмами, магнитными лентами или дискетами, переносимыми с одного компьютера на другой и т.д.

Поток данных на диаграмме изображается линией, оканчивающейся стрелкой, которая показывает направление потока. Каждый поток данных имеет имя, отражающее его содержание.

Стрелки описывают движение объектов из одной части системы в другую. Поскольку в DFD каждая сторона работы не имеет четкого назначения, как в IDEF0, стрелки могут подходить и выходить из любой грани прямоугольника работы. В DFD также применяются двунаправленные стрелки для описания диалогов типа "команда-ответ" между работами, между работой и внешней сущностью и между внешними сущностями

Все диаграммы иерархически ваимосвязаны. Первым шагом при пострении иерархии DFD является построение контекстных диаграмм.

Контекстная диаграмма

Хотя контекстная диаграмма данных является лишь особым вариантом DFD, она играет важную роль в процессе проектирования систем и поэтому должна быть упомянута особо. На контекстной диаграмме, в отличие от DFD, изображаются внешние сущности (или терминаторы). Под внешними сущностями понимают окружение системы, то есть множество объектов, взаимодействующих с системой, но не входящих в ее состав. В качестве примеров внешних сущностей можно указать следующие объекты :

- другие системы;
- пользователи;
- каналы информационного обмена.

Проектировании относительно простых ИС. На контекстной диаграмме специфицируемая система представляется в виде одного единственного процесса, связанного с внешними сущностями потоками данных. Контекстная диаграмма представляет требования к системе на самом верхнем уровне - уровне взаимодействия с окружением. Разработка DFD-модели всей системы начинается именно с диаграмм данного типа.

Перед построением контекстной DFD необходимо проанализировать внешние события (внешние объекты), оказывающие влияние на функционирование системы.

Список событий строится в виде матрицы (ELM) и описывает различные действия внешних сущностей и реакцию ИС на них.

№	Описание события	Реакция
1	Клиент вставил в банкомат кредитную карточку	Обработка данных с кредитной карточки
2	Клиент ввел личный номер	Идентификация клиента

ПРИМЕР. В качестве примера рассмотрим упрощенную модель процесса получения некоторой суммы наличными по кредитной карточке клиентом банка. Внешними сущностями данного примера являются клиент банка и, возможно, служащий банка, который контролирует процесс обслуживания клиентов. Накопителем данных может быть база данных о состоянии счетов отдельных клиентов банка. Отдельные потоки данных отражают характер передаваемой информации, необходимой для обслуживания клиента банка. Соответствующая модель для данного примера может быть представлена в виде диаграммы потоков данных (рис. 2.19).



Рис. 2.19. Пример диаграммы DFD для процесса получения некоторой суммы наличными по кредитной карточке

Проектировании сложных ИС. Если для сложной системы ограничиться единственной контекстной диаграммой, то она будет содержать слишком большое количество источников и приемников информации, которые трудно расположить на листе бумаги нормального формата, и кроме того, единственный главный процесс не раскрывает структуры распределенной системы. Признаками сложности (в смысле контекста) могут быть:

- наличие большого количества внешних сущностей (десять и более);
- распределенная природа системы;
- многофункциональность системы с уже сложившейся или выявленной группировкой функций в отдельные подсистемы.

Для сложных ИС строится иерархия контекстных диаграмм. При этом контекстная диаграмма верхнего уровня содержит не единственный главный процесс, а набор подсистем, соединенных потоками данных. Контекстные диаграммы следующего уровня детализируют контекст и структуру подсистем. Иерархия контекстных диаграмм определяет взаимодействие основных функциональных подсистем проектируемой ИС как между собой, так и с внешними входными и выходными потоками данных и внешними объектами (источниками и приемниками информации), с которыми взаимодействует ИС.

Построение модели

Главная цель построения иерархического множества DFD заключается в том, чтобы сделать требования ясными и понятными на каждом уровне детализации, а также разбить эти требования на части с точно определенными отношениями между ними. Для достижения этого целесообразно пользоваться следующими рекомендациями:

- Размещать на каждой диаграмме от 3 до 6-7 процессов. Верхняя граница соответствует человеческим возможностям одновременного восприятия и понимания структуры сложной системы с множеством внутренних связей, нижняя граница выбрана по соображениям здравого

смысла: нет необходимости детализировать процесс диаграммой, содержащей всего один или два процесса.

- Не загромождать диаграммы несущественными на данном уровне деталями.
- Декомпозицию потоков данных осуществлять параллельно с декомпозицией процессов; эти две работы должны выполняться одновременно, а не одна после завершения другой.
- Выбирать ясные, отражающие суть дела, имена процессов и потоков для улучшения понимаемости диаграмм, при этом стараться не использовать аббревиатуры.
- Однократно определять функционально идентичные процессы на самом верхнем уровне, где такой процесс необходим, и ссылаться на него на нижних уровнях.
- Пользоваться простейшими диаграммными техниками: если что-либо возможно описать с помощью DFD, то это и необходимо делать, а не использовать для описания более сложные объекты.
- Отделять управляющие структуры от обрабатывающих структур (т.е. процессов), локализовать управляющие структуры.

В соответствии с этими рекомендациями процесс построения модели разбивается на следующие этапы:

1. Расчленение множества требований и организация их в основные функциональные группы.
2. Идентификация внешних объектов, с которыми система должна быть связана.
3. Идентификация основных видов информации, циркулирующей между системой и внешними объектами.
4. Предварительная разработка контекстной диаграммы, на которой основные функциональные группы представляются процессами, внешние объекты - внешними сущностями, основные виды информации - потоками данных между процессами и внешними сущностями.
5. Изучение предварительной контекстной диаграммы и внесение в нее изменений по результатам ответов на возникающие вопросы по всем ее частям.
6. Построение контекстной диаграммы путем объединения всех процессов предварительной диаграммы в один процесс, а также группирования потоков.
7. Формирование DFD первого уровня на базе процессов предварительной контекстной диаграммы.
8. Проверка основных требований по DFD первого уровня.
9. Декомпозиция каждого процесса текущей DFD с помощью детализирующей диаграммы или спецификации процесса.
10. Проверка основных требований по DFD соответствующего уровня,
11. Добавление определений новых потоков в словарь данных при каждом их появлении на диаграммах.
12. Параллельное (с процессом декомпозиции) изучение требований (в том числе и вновь поступающих), разбиение их на элементарные и идентификация процессов или спецификаций процессов, соответствующих этим требованиям.
13. После построения двух-трех уровней проведение ревизии с целью проверки корректности и улучшения понимаемости модели.
14. Построение спецификации процесса (а не простейшей диаграммы) в случае, если некоторую функцию сложно или невозможно выразить комбинацией процессов.

Для каждой подсистемы, присутствующей на контекстных диаграммах, выполняется ее детализация при помощи ДПД. Каждый процесс на ДПД, в свою очередь, может быть детализирован при помощи ДПД или миниспецификации.

Пример системы взаимосвяанных диаграмм показан на рисунке.

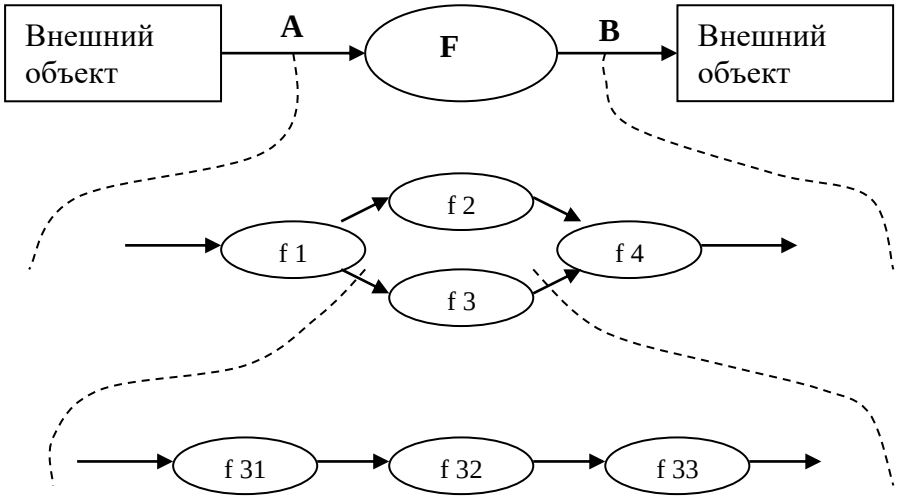


Рис. Система взаимосвяанных диаграмм потоков данных

Примечание. Важно сохранить непрерывность информационного потока и его согласаванность. Это значит, что входы и выходы у каждого пробразователя на любом уровне должны оставаться прежними. В диаграмме отсутствуют точные указания на последовательност обработки. Точные указания откладываются до этапа проектирования.

Миниспецификация (описание логики процесса) должна формулировать его основные функции таким образом, чтобы в дальнейшем специалист, выполняющий реализацию проекта, смог выполнить их или разработать соответствующую программу.

Миниспецификация является конечной вершиной иерархии ДПД. Решение о завершении детализации процесса и использовании миниспецификации принимается аналитиком исходя из следующих критериев:

- наличия у процесса относительно небольшого количества входных и выходных потоков данных (2-3 потока);
- возможности описания преобразования данных процессом в виде последовательного алгоритма;
- выполнения процессом единственной логической функции преобразования входной информации в выходную;
- возможности описания логики процесса при помощи миниспецификации небольшого объема (не более 20-30 строк).

При построении иерархии ДПД переходить к детализации процессов следует только после определения содержания всех потоков и накопителей данных, которое описывается при помощи структур данных. Структуры данных конструируются из элементов данных и могут содержать альтернативы, условные вхождения и итерации. Условное вхождение означает, что данный компонент может отсутствовать в структуре. Альтернатива означает, что в структуру может входить один из перечисленных элементов. Итерация означает вхождение любого числа элементов в указанном диапазоне. Для каждого элемента данных может указываться его тип (непрерывные или дискретные данные). Для непрерывных данных может указываться единица измерения (кг, см и т.п.), диапазон значений, точность представления и форма физического кодирования. Для дискретных данных может указываться таблица допустимых значений.

После построения законченной модели системы ее необходимо верифицировать (проверить на полноту и согласованность). В полной модели все ее объекты (подсистемы, процессы, потоки данных) должны быть подробно описаны и детализированы. Выявленные недетализированные объекты следует детализировать, вернувшись на предыдущие шаги разработки. В согласованной модели для всех потоков данных и накопителей данных должно выполняться правило сохранения информации: все поступающие куда-либо данные должны быть считаны, а все считываемые данные должны быть записаны.



В отличие от стрелок IDEF0, которые представляют собой жесткие взаимосвязи, стрелки DFD показывают, как объекты (включая данные) двигаются от одной работы к другой. Это представление потоков совместно с хранилищами данных и внешними сущностями делает модели DFD более похожими на физические характеристики системы - движение объектов (data flow), хранение объектов (data stores), поставка и распространение объектов (external entities) (рис. 1.49)

Пример банковской задачи

В качестве примера создания модели рассмотрим фрагмент проекта системы, организующей работу банкомата по обслуживанию клиента по его кредитной карте. Этот пример будет строиться поэтапно, на нем будут продемонстрированы базовые техники структурного анализа и проектирования по мере их определения.

На рис. 2.3 приведена контекстная диаграмма системы с единственным процессом ОБСЛУЖИТЬ, идентифицирующая внешние сущности КЛИЕНТ и КОМПЬЮТЕР БАНКА, хранящий информацию о счетах всех клиентов. Опишем потоки данных, которыми обменивается проектируемая система с внешними объектами.

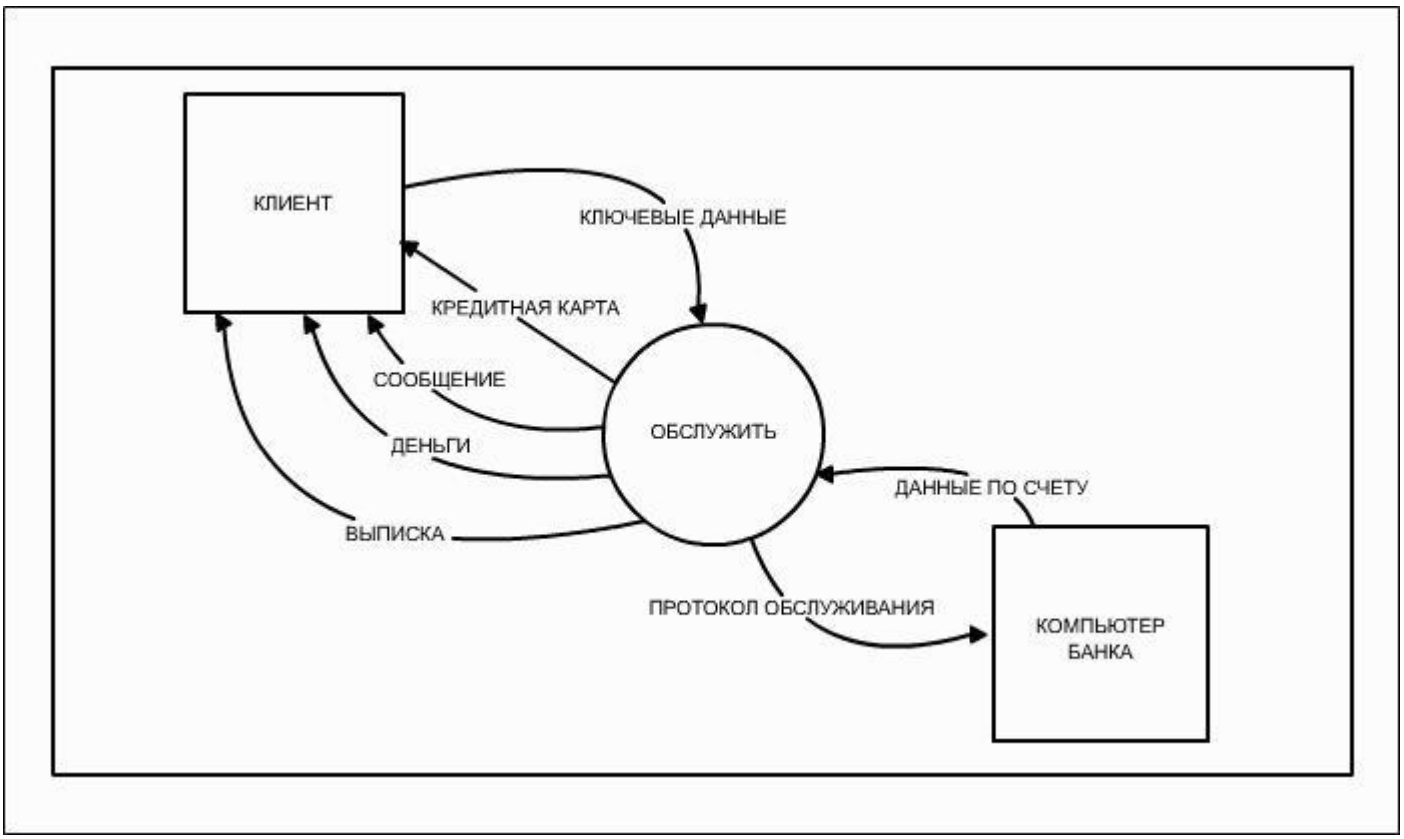


Рис. 2.3: Контекстная диаграмма банковской задачи

Для банковского обслуживания клиенту необходимо предоставить системе свою КРЕДИТНУЮ КАРТУ для автоматического считывания с нее информации (ПАРОЛЬ, ЛИМИТ ДЕНЕГ, ДЕТАЛИ КЛИЕНТА), а также сообщить свои КЛЮЧЕВЫЕ ДАННЫЕ, а именно ПАРОЛЬ и ЗАПРОС НА ОБСЛУЖИВАНИЕ, т.е. требуемую ему услугу (например, снятие со счета наличных денег). Банковское обслуживание с позиций клиента, в свою очередь, должно обеспечить следующее:

- выдать СООБЩЕНИЕ, приглашающее клиента ввести КЛЮЧЕВЫЕ ДАННЫЕ;
- выдать клиенту ДЕНЬГИ;
- выдать клиенту ВЫПИСКУ по проведенному обслуживанию, включающую ВЫПИСКУ О ДЕНЬГАХ, ВЫПИСКУ ПО БАЛАНСУ и ВЫПИСКУ ПО ОПЕРАЦИИ, проведенной банком.

Контекстный процесс и КОМПЬЮТЕР БАНКА должны обмениваться следующей информацией:

- ДАННЫЕ ПО СЧЕТУ клиента в банке;
- ПРОТОКОЛ ОБСЛУЖИВАНИЯ, включающий информацию об ОБРАБОТАННОЙ ДОКУМЕНТАЦИИ, изымаемой ДЕНЕЖНОЙ СУММЕ и ДАННЫЕ ПО ИСТОРИИ ЗАПРОСА.

Контекстный процесс может быть детализирован DFD первого уровня как показано на рис. 2.4. Эта диаграмма содержит 4 процесса и хранилище ДАННЫЕ КРЕДИТНОЙ КАРТЫ, которое изображено дважды на диаграмме с целью избежания пересечений линий потоков данных.

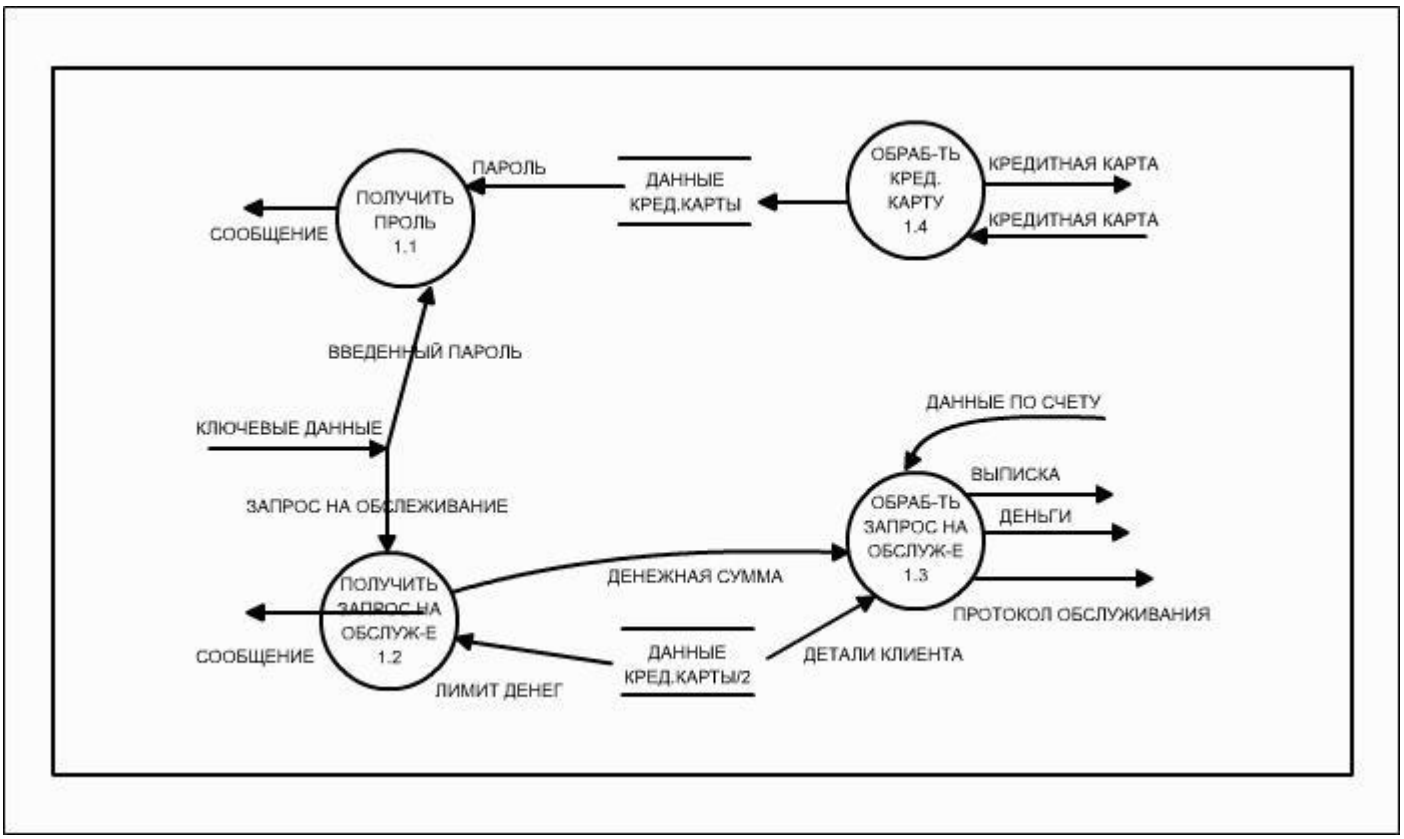


Рис. 2.4: Детализация процесса ОБСЛУЖИТЬ

Процесс 1.1 (ПОЛУЧИТЬ ПАРОЛЬ) осуществляет прием и проверку пароля клиента и имеет на входе/выходе следующие потоки:

- внешний выходной поток СООБЩЕНИЕ для информирования клиента о готовности принять пароль;
- входной поток ВВЕДЕННЫЙ ПАРОЛЬ как элемент внешнего потока КЛЮЧЕВЫЕ ДАННЫЕ;
- входной поток ПАРОЛЬ из хранилища ДАННЫЕ КРЕДИТНОЙ КАРТЫ для проверки вводимого клиентом пароля.

Процесс 1.2 (ПОЛУЧИТЬ ЗАПРОС НА ОБСЛУЖИВАНИЕ) осуществляет прием и проверку запроса клиента на проведение необходимой ему банковской операции и имеет на входе/выходе следующие потоки:

- внешний выходной поток СООБЩЕНИЕ для информирования клиента о своей готовности принять запрос на обслуживание;
- входной поток ЗАПРОС НА ОБСЛУЖИВАНИЕ как элемент внешнего потока КЛЮЧЕВЫЕ ДАННЫЕ;
- входной поток ЛИМИТ ДЕНЕГ из хранилища ДАННЫЕ КРЕДИТНОЙ КАРТЫ для контроля наличия денег на счете клиента.

Процесс 1.3 (ОБРАБОТАТЬ ЗАПРОС НА ОБСЛУЖИВАНИЕ) имеет внешний входной поток ДАННЫЕ ПО СЧЕТУ (из внешней сущности КОМПЬЮТЕР БАНКА), входной поток ДЕТАЛИ КЛИЕНТА (из хранилища), а также внешние выходные потоки ВЫПИСКА, ДЕНЬГИ и ПРОТОКОЛ ОБСЛУЖИВАНИЯ.

Процесс 1.4 (ОБРАБОТАТЬ КРЕДИТНУЮ КАРТУ) осуществляет считывание информации с кредитной карты и имеет на входе внешний поток КРЕДИТНАЯ КАРТА, а на выходе поток ДАННЫЕ КРЕДИТНОЙ КАРТЫ. Отметим, что нет необходимости в идентификации последнего потока, т.к. идентифицировано соответствующее хранилище.

Процессы 1.1, 1.2 и 1.4 являются элементарными, поэтому нет необходимости в их детализации с помощью DFD уровня 2 (они будут раскрыты с помощью спецификаций про-

цессов в главе 4). Процесс 1.3 может быть детализирован с помощью DFD второго уровня как показано на рис. 2.5. Эта диаграмма содержит 4 элементарных процесса, спецификации которых также будут приведены в главе 4.

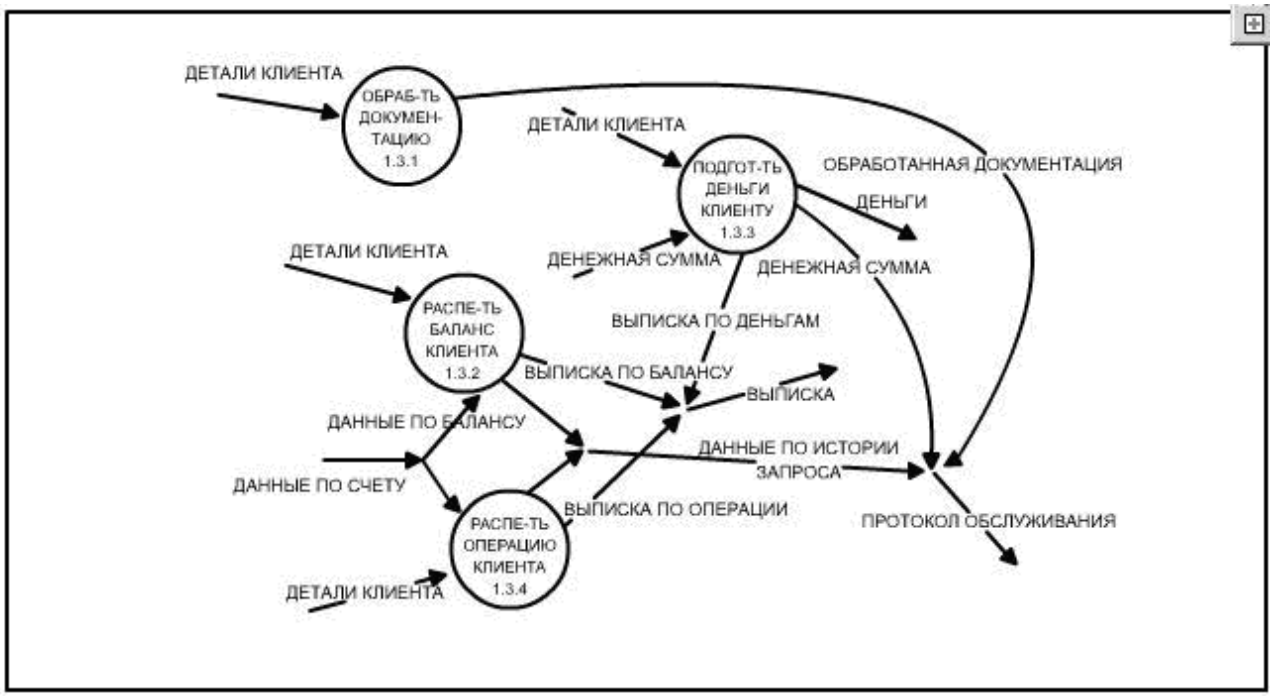


Рис. 2.5: Детализация процесса ОБРАБОТАТЬ ЗАПРОС НА ОБСЛУЖИВАНИЕ

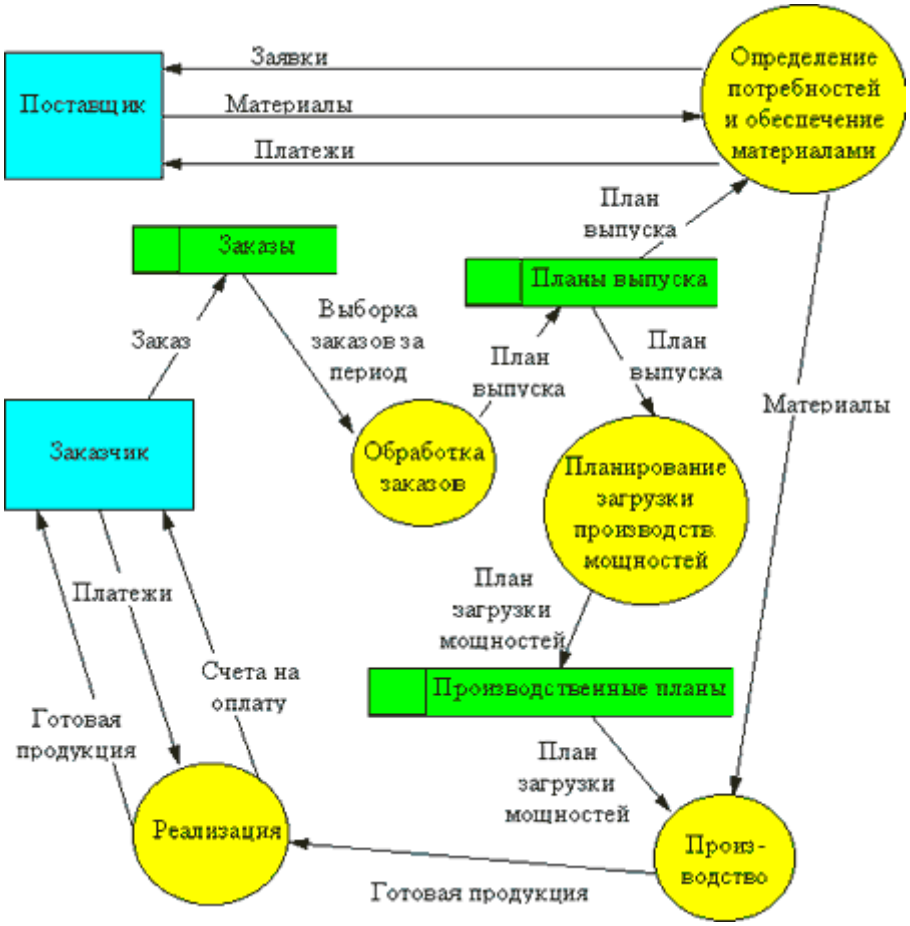
Процесс 1.3.1 (ОБРАБОТАТЬ ДОКУМЕНТАЦИЮ БАНКА) осуществляет обработку внутренней банковской документации по клиенту и имеет входной поток ДЕТАЛИ КЛИЕНТА и выходной поток ОБРАБОТАННАЯ ДОКУМЕНТАЦИЯ (часть внешнего потока ПРОТОКОЛ СДЕЛКИ).

Процесс 1.3.2 (РАСПЕЧАТАТЬ БАЛАНС КЛИЕНТА) выдает справку по истории счета клиента и по балансу клиента. Входные потоки - ДЕТАЛИ КЛИЕНТА и ДАННЫЕ ПО БАЛАНСУ (часть внешнего потока ДАННЫЕ ПО СЧЕТУ), выходные потоки ВЫПИСКА ПО БАЛАНСУ (часть внешнего потока ВЫПИСКА) и ДАННЫЕ ПО ИСТОРИИ ЗАПРОСА (часть внешнего потока ПРОТОКОЛ ОБСЛУЖИВАНИЯ).

Процесс 1.3.3 (ПРИГОТОВИТЬ ДЕНЬГИ КЛИЕНТУ) обеспечивает выдачу наличных денег и информирование компьютера банка об изъятых из банка деньгах. Он имеет входные потоки ДЕНЕЖНАЯ СУММА и ДЕТАЛИ КЛИЕНТА, и выходные потоки ДЕНЬГИ и ДЕНЕЖНАЯ СУММА (часть потока ПРОТОКОЛ ОБСЛУЖИВАНИЯ).

Процесс 1.3.4 (РАСПЕЧАТАТЬ ОПЕРАЦИЮ КЛИЕНТА) выдает справку по истории счета и уведомление по проведенной операции. Входные потоки ДАННЫЕ ПО СЧЕТУ и ДЕТАЛИ КЛИЕНТА, выходные потоки - ВЫПИСКА ПО ОПЕРАЦИИ (часть потока ВЫПИСКА) и ДАННЫЕ ПО ИСТОРИИ ЗАПРОСА (часть потока ПРОТОКОЛ ОБСЛУЖИВАНИЯ).

Пример изготовления на заказ. Построим DFD-диаграмму для предприятия, строящего свою деятельность по принципу "изготовление на заказ". На основании полученных заказов формируется план выпуска продукции на определенный период. В соответствии с этим планом определяются потребность в комплектующих изделиях и материалах, а также график загрузки производственного оборудования. После изготовления продукции и проведения платежей, готовая продукция отправляется заказчику.



Эта диаграмма представляет самый верхний уровень функциональной модели. Естественно, это весьма грубое описание предметной области. Уточнение модели производится путем детализации необходимых функций на DFD-диаграмме следующего уровня. Так мы можем разбить функцию "Определение потребностей и обеспечение материалами" на подфункции "Определение потребностей", "Поиск поставщиков", "Заключение и анализ договоров на поставку", "Контроль платежей", "Контроль поставок", связанные собственными потоками данных, которые будут представлены на отдельной диаграмме. Детализация модели должна производится до тех пор, пока она не будет содержать всю информацию, необходимую для построения информационной системы

2.1.1.2. Словарь данных

В словаре данных определяются все потоки данных и управления, а также хранилища данных и управления. Внешние сущности не описываются в словаре данных ввиду того, что они принадлежат окружению системы, а не самой системе. В словаре присутствуют описания двух типов потоков - примитивов и составных потоков. Примитивы в отличие от составных потоков не определяются через другие потоки и примитивы.

Диаграммы потоков данных обеспечивают удобное описание функционирования компонент системы, но не снабжают аналитика средствами описания деталей этих компонент, а именно, какая информация преобразуется процессами и как она преобразуется. Для решения первой из перечисленных задач предназначены текстовые средства моделирования, служащие для описания структуры преобразуемой информации и получившие название словарей данных.

Словарь данных представляет собой определенным образом организованный список всех элементов данных системы с их точными определениями, что дает возможность различным категориям пользователей (от системного аналитика до программиста) иметь общее понимание всех входных и выходных потоков и компонент хранилищ. Определения элементов данных в словаре осуществляются следующими видами описаний:

- описанием значений потоков и хранилищ, изображенных на DFD;
- описанием композиции агрегатов данных, движущихся вдоль потоков, т.е. комплексных данных, которые могут расчленяться на элементарные символы (например, *АДРЕС ПОКУПАТЕЛЯ* содержит *ПОЧТОВЫЙ ИНДЕКС, ГОРОД, УЛИЦУ* и т.д.);
- описанием композиции групповых данных в хранилище;
- специфицированием значений и областей действия элементарных фрагментов информации в потоках данных и хранилищах;
- описанием деталей отношений между хранилищами.

Содержимое словаря данных

Для каждого потока данных в словаре необходимо хранить имя потока, его тип и атрибуты. Информация по каждому потоку состоит из ряда словарных статей, каждая из которых начинается с ключевого слова - заголовка соответствующей статьи, которому предшествует символ “**@**”.

По типу потока в словаре содержится информация, идентифицирующая:

- простые (элементарные) или групповые (комплексные) потоки;
- внутренние (существующие только внутри системы) или внешние (связывающие систему с другими системами) потоки;
- потоки данных или потоки управления;
- непрерывные (принимающие любые значения в пределах определенного диапазона) или дискретные (принимающие определенные значения) потоки.

Атрибуты потока данных включают:

- имена-синонимы потока данных в соответствии с узлами изменения имени;
- БНФ-определение в случае группового потока (см. 3.2);
- единицы измерения потока;
- диапазон значений для непрерывного потока, типичное его значение и информацию по обработке экстремальных значений;
- список значений и их смысл для дискретного потока;
- список номеров диаграмм различных типов, в которых поток встречается;

- список потоков, в которые данный поток входит (как элемент БНФ-определения);
- комментарий, включающий дополнительную информацию (например о цели введения данного потока).

БНФ - нотация

БНФ-нотация позволяет формально описать расщепление/объединение потоков. Поток может расщепляться на собственные отдельные ветви, на компоненты потока-предка или на то и другое одновременно. При расщеплении/объединении потока существенно, чтобы каждый компонент потока-предка являлся именованным. Если поток расщепляется на подпотоки, необходимо, чтобы все подпотоки являлись компонентами потока-предка. И наоборот, при объединении потоков каждый компонент потока-предка должен по крайней мере однажды встречаться среди подпотоков. Отметим, что при объединении подпотоков нет необходимости осуществлять исключение общих компонент, а при расщеплении подпотоки могут иметь такие общие (одинаковые) компоненты.

Важно понимать, что точные определения потоков содержатся в словаре данных, а не на диаграммах. Например, на диаграмме может иметься групповой узел с входным потоком *X* и выходными подпотоками *Y* и *Z*. Однако это вовсе не означает, что соответствующее определение в словаре данных обязательно должно быть *X=Y+Z*. Это определение может быть следующим:

$$X=A+B+C;\ Y=A+B;\ Z=B+C$$

Такие определения хранятся в словаре данных в так называемой **БНФ-статье**. БНФ-статья используется для описания компонент данных в потоках данных и в хранилищах. Ее синтаксис имеет вид:

@БНФ *= <простой оператор>* **|** *<БНФ-выражение> ,*

где *<простой оператор>* есть текстовое описание, заключенное в "/", а *<БНФ-выражение>* есть выражение в форме Бэкуса-Наура, допускающее следующие операции отношений:

- **=** - означает "композиция из",
- **+** - означает "*И*",
- **[|]** - означает "*ИЛИ*",
- **( )** - означает, что компонент в скобках не обязателен,
- **{ }** - означает итерацию компонента в скобках,
- **" "** - означает литерал.

Итерационные скобки могут иметь нижний и верхний предел, например:

- *3{болт}7 - от 3 до 7 итераций*
- *1{болт} - 1 и более итераций*
- *{шайба}3 - не более 3 итераций*

БНФ-выражение может содержать произвольные комбинации операций:

- ***@БНФ*** *= [винт ! болт + 2{гайка}2 + (прокладка) |клей]*

Ниже приведен пример описания потока данных с помощью БНФ:

- ***@ИМЯ*** *= ВОСЬМЕРИЧНАЯ ЦИФРА*
- ***@ТИП*** *= дискретный поток*
- ***@БНФ*** = ["0"|"1"|"2"|"3"|"4"|"5"|"6"|"7"]

Посмотрим, как некоторые потоки, присутствующие на вышеприведенных диаграммах потоков данных, представляются в словаре данных.

- ***@ИМЯ*** *= ВВЕДЕННАЯ КРЕДИТНАЯ КАРТА*
- ***@ТИП*** *= управляющий поток*
- ***@БНФ*** *= /указывает, что кредитная карта введена/*
- ***@ИМЯ*** *= ДАННЫЕ КРЕДИТНОЙ КАРТЫ*
- ***@ТИП*** *= дискретный поток*
- ***@БНФ*** *= ПАРОЛЬ + ДЕТАЛИ КЛИЕНТА + ЛИМИТ ДЕНЕГ*
- ***@ИМЯ*** *= ДАННЫЕ ПО БАЛАНСУ*
- ***@ТИП*** *= дискретный поток*
- ***@БНФ*** *= /текущий баланс счета клиента/*
- ***@ЕДИНИЦА ИЗМЕРЕНИЯ*** *= доллар*
- ***@ДИАПАЗОН*** *= +/- 100000*
- ***@ТОЧНОСТЬ*** *= .01*
- ***@ИМЯ*** *= ДЕНЬГИ*
- ***@ТИП*** *= дискретный поток*
- ***@БНФ*** *= /деньги, выдаваемые клиенту/*
- ***@ЕДИНИЦА ИЗМЕРЕНИЯ*** *= доллар*
- ***@НОРМА*** *= 5..1000*
- ***@КОММЕНТАРИЙ*** *Сумма выдаваемых денег должна делиться на 5*
- ***@ИМЯ*** *= ПРОТОКОЛ ОБСЛУЖИВАНИЯ*
- ***@ТИП*** *= дискретный поток*
- ***@БНФ*** *= (ОБРАБОТАННАЯ ДОКУМЕНТАЦИЯ)*
- *+ (ДЕНЕЖНАЯ СУММА)*
- *+ (ДАННЫЕ ПО ИСТОРИИ ЗАПРОСА)*

Таблица специальных символов словаря

Символ	Значение	Описание
=	Состоит из	Данный символ означает, что поток, указанный слева от него, состоит из потоков, перечисленных справа
+	Объединить	Объединяет потоки в группу, не определяя порядок, в котором они должны быть в ней представлены. Объединение в группу соответствует операции "Логическое И".
{ }	Итерация	Выражение, заключенное в скобки, может встречаться произвольное число раз в определяемом потоке. Выражение может быть проиндексировано, в этом случае до и после скобки указывается число. Всего допустимо 3 комбинации: • { }N - 0,1,..., N-1 или N итераций • N{ } - N или более итераций • N{ }N - в точности N итераций
[|]	Выбор одного из	Внутри скобок может располагаться произвольное число выражений, разделенных знаком |. Определяемый поток состоит в точности из одного из указанных выражений.

( )	По выбору	Выражение, заключенное в скобки может как встречаться так и не встречаться в определяемом потоке.
/* */	Комментарий	Текст, заключенный между знаками /* и */, представляет собой комментарий
\ \	Примитив	Выражение, заключенное в данные скобки представляет собой примитив, который должен быть описан в соответствующей таблице.

2.1.1.3. Методы задания спецификации процессов

Спецификация процесса (СП) используется для описания функционирования процесса в случае отсутствия необходимости детализировать его с помощью DFD (т.е. если он достаточно невелик, и его описание может занимать до одной страницы текста). Фактически СП представляют собой алгоритмы описания задач, выполняемых процессами: множество всех СП является полной спецификацией системы. СП содержат номер и/или имя процесса, списки входных и выходных данных и тело (описание) процесса, являющееся спецификацией алгоритма или операции, трансформирующей входные потоки данных в выходные. Известно большое число разнообразных методов, позволяющих задать тело процесса, соответствующий язык может варьироваться от **структурированного естественного языка** или псевдокода до **визуальных языков проектирования** (типа **FLOW-форм** и **диаграмм Насси-Шнейдермана**) и формальных компьютерных языков.

Независимо от используемой нотации спецификация процесса должна начинаться с ключевого слова (например, ***@СПЕЦПРОЦ***). Требуемые входные и выходные данные должны быть специфицированы следующим образом:

@ВХОД = <имя символа данных>
@ВЫХОД = <имя символа данных>
@ВХОДВЫХОД = <имя символа данных>,
где <имя символа данных> - соответствующее имя из словаря данных.

Эти ключевые слова должны использоваться перед определением СП, например,

@ВХОД = СЛОВА ПАМЯТИ
@ВЫХОД = ХРАНИМЫЕ ЗНАЧЕНИЯ
@СПЕЦПРОЦ
Для всех СЛОВ ПАМЯТИ выполнить:
Распечатать ХРАНИМЫЕ ЗНАЧЕНИЯ
@

Ситуация, когда символ данных является одновременно входным и выходным, может быть описана двумя способами: либо символ описывается два раза с помощью ***@ВХОД*** и ***@ВЫХОД***, либо один раз с помощью ***@ВХОДВЫХОД***.

Иногда в СП задаются **пред-** и **пост-условия** выполнения данного процесса. В предусловии записываются объекты, значения которых должны быть истинны перед началом выполнения процесса, что обеспечивает определенные гарантии безопасности для пользователя. Аналогично, в случае наличия пост-условия гарантируется, что значения всех входящих в него объектов будут истинны при завершении процесса.

Спецификации должны удовлетворять следующим требованиям:

- для каждого процесса нижнего уровня должна существовать одна и только одна спецификация;
- спецификация должна определять способ преобразования входных потоков в выходные;
- нет необходимости (на данном этапе) определять метод реализации этого преобразования;
- спецификация должна стремиться к ограничению избыточности - не следует переопределять то, что уже было определено на диаграмме или в словаре даннных;
- набор конструкций для построения спецификации должен быть простым и стандартным.

Ниже рассматриваются некоторые наиболее часто используемые методы задания спецификаций процессов.

Структурированный естественный язык

Структурированный естественный язык применяется для читабельного, строгого описания спецификаций процессов. Он является разумной комбинацией строгости языка программирования и читабельности естественного языка и состоит из подмножества слов, организованных в определенные логические структуры, арифметических выражений и диаграмм.

В состав языка входят следующие основные символы:

- глаголы, ориентированные на действие и применяемые к объектам;
- термины, определенные на любой стадии проекта ПО (например, задачи, процедуры, символы данных и т.п.);
- предлоги и союзы, используемые в логических отношениях;
- общеупотребительные математические, физические и технические термины;
- арифметические уравнения;
- таблицы, диаграммы, графы и т.п.;
- комментарии.

Управляющие структуры языка имеют один вход и один выход. К ним относятся:

1) последовательная конструкция:
ВЫПОЛНИТЬ *функция1*
ВЫПОЛНИТЬ *функция2*
ВЫПОЛНИТЬ *функция3*

2) конструкция выбора:
ЕСЛИ *<условие>* ***ТО***
ВЫПОЛНИТЬ *функция1*
ИНАЧЕ
ВЫПОЛНИТЬ *функция2*
КОНЕЦЕСЛИ

3) итерация:
ДЛЯ *<условие>*
ВЫПОЛНИТЬ *функция*
КОНЕЦДЛЯ

или
ПОКА *<условие>*
ВЫПОЛНИТЬ *функция*
КОНЕЦПОКА

При использовании структурированного естественного языка приняты следующие соглашения:

1. Логика процесса выражается в виде комбинации последовательных конструкций, конструкций выбора и итераций.
2. Ключевые слова ***ЕСЛИ, ВЫПОЛНИТЬ, ИНАЧЕ*** и т.д. должны быть написаны заглавными буквами.
3. Слова или фразы, определенные в словаре данных, должны быть написаны заглавными буквами.
4. Глаголы должны быть активными, недвусмысленными и ориентированными на целевое действие (заполнить, вычислить, извлечь, а не модернизировать, обработать).
5. Логика процесса должна быть выражена четко и недвусмысленно.

Ниже приведен пример спецификации процесса 1 (ПОЛУЧИТЬ ПАРОЛЬ) для диаграммы, изображенной на рис. 2.8.

@ВХОД *= ВВЕДЕННЫЙ ПАРОЛЬ*
@ВХОД *= ПАРОЛЬ*
@ВЫХОД *= СООБЩЕНИЕ*
@ВЫХОД *= КОРРЕКТНЫЙ ПАРОЛЬ*
@СПЕЦПРОЦ *1.1 ПОЛУЧИТЬ ПАРОЛЬ*
ВЫПОЛНИТЬ *выдать СООБЩЕНИЕ клиенту,*
запрашивающее ввод пароля
принять ВВЕДЕННЫЙ ПАРОЛЬ
ДОТЕХПОРПОКА *ВВЕДЕННЫЙ ПАРОЛЬ = ПАРОЛЬ*
или были сделаны три попытки ввода
КОНЕЦВЫПОЛНИТЬ
ВЫПОЛНИТЬ *установить флаг КОРРЕКТНЫЙ*
ПАРОЛЬ в случае равенства
@ *КОНЕЦ СПЕЦИФИКАЦИИ ПРОЦЕССА 1.1*

Таблицы и деревья решений

Структурированный естественный язык неприемлем для некоторых типов преобразований. Например, если действие зависит от нескольких переменных, которые в совокупности могут продуцировать большое число комбинаций, то его описание будет слишком запутанным и с большим числом уровней вложенности. Для описания подобных действий традиционно используются таблицы и деревья решений.

Проектирование спецификаций процессов с помощью **таблиц решений** (ТР) заключается в задании матрицы, отображающей множество входных **условий** в множество **действий**.

ТР состоит из двух частей. Верхняя часть таблицы используется для определения условий. Обычно условие является ***ЕСЛИ***-частью оператора ***ЕСЛИ-ТО*** и требует ответа "да-нет". Однако иногда в условии может присутствовать и ограниченное множество значений, например, *ЯВЛЯЕТСЯ ЛИ ДЛИНА СТРОКИ БОЛЬШЕЙ, МЕНЬШЕЙ ИЛИ РАВНОЙ ГРАНИЧНОМУ ЗНАЧЕНИЮ?*

Нижняя часть ТР используется для определения действий, т.е. ***ТО***-части оператора ***ЕСЛИ-ТО***. Так, в конструкции

ЕСЛИ *ИДЕТ ДОЖДЬ* ***ТО*** *РАСКРЫТЬ ЗОНТ*
ИДЕТ ДОЖДЬ является условием, а *РАСКРЫТЬ ЗОНТ* - действием.

Левая часть ТР содержит собственно описание условий и действий, а в правой части перечисляются все возможные комбинации условий и, соответственно, указывается, какие конкретно действия и в какой последовательности выполняются, когда определенная комбинация условий имеет место.

Поясним вышесказанное на примере спецификации процесса выбора символов из входного потока. При выборе символов необходимо руководствоваться следующими правилами:

1) если очередной символ является управляющим, то подать звуковой сигнал и вернуть код ошибки;
2) если буфер формируемой строки заполнен, то подать звуковой сигнал и вернуть код ошибки;
3) если очередной символ не находится в заданном диапазоне, то подать звуковой сигнал и вернуть код ошибки;
4) иначе поместить символ в буфер, увеличить значение счетчика выбранных символов и вернуть новое значение счетчика.
Таблица решений для данного примера выглядит следующим образом (таблица 4.1):

Таблица 4.1

	УСЛОВИЯ	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
C1	isctrl(c)	Д	Д	Д	Д	Н	Н	Н	Н
C2	I > max_lenght	Д	Д	Н	Н	Д	Д	Н	Н
C3	out_of_range(c)	Д	Н	Д	Н	Д	Н	Д	Н
	ДЕЙСТВИЯ								
D1	beep()	1	1	1	1	1	1	1	
D2	return(ERROR)	2	2	2	2	2	2	2	
D3	return(++i)								2
D4	putchar(c)								1

Заметим, что если выполняется условие *C1*, то нет необходимости в проверке условий *C2* и *C3*. Поэтому комбинации условий 1, 2, 3, 4 могут быть заменены обобщающей комбинацией (*Д,*-,-), где "-" означает любую из возможных альтернатив (в данном случае, *Д* или *Н*). Аналогично, комбинации условий 5 и 6 могут быть заменены обобщающей комбинацией (*Н,Д,*-). Редуцированная таким образом таблица решений будет иметь следующий вид (таблица 4.2):

Таблица 4.2

	УСЛОВИЯ	**1**	**2**	**3**	**4**
C1	isctrl(c)	Д	Н	Н	Н
C2	I > max_lenght	-	Д	Н	Н
C3	out_of_range(c)	-	-	Д	Н

	ДЕЙСТВИЯ				
D1	beep()	1	1	1	
D2	return(ERROR)	2	2	2	
D3	return(++i)				2
D4	putchar(c)				1

Отметим, что на основе таблицы решений легко осуществляется автоматическая кодогенерация. Для вышеприведенного примера соответствующий код может выглядеть следующим образом:

```
IF (isctrl(c)) { beep(); return(ERROR) }
   ELSE {
   IF (i>max_length) { beep(); return(ERROR) }
   ELSE {
   IF (out_of_range(c)) { beep(); return(ERROR) }
   ELSE { putchar(c); return(++i) }
   }
   }
```

Построение ТР рекомендуется осуществлять по следующим шагам:

1. Идентифицировать все условия (или переменные) в спецификации. Идентифицировать все значения, которые каждая переменная может иметь.
2. Вычислить число комбинаций условий. Если все условия являются бинарными, то существует 2**N комбинаций N переменных.
3. Идентифицировать каждое из возможных действий, которые могут вызываться в спецификации.
4. Построить пустую таблицу, включающую все возможные условия и действия, а также номера комбинаций условий.
5. Выписать и занести в таблицу все возможные комбинации условий.
6. Редуцировать комбинации условий.
7. Проверить каждую комбинацию условий и идентифицировать соответствующие выполняемые действия.
8. Выделить комбинации условий, для которых спецификация не указывает список выполняемых действий.
9. Обсудить построенную таблицу.

Вариантом таблицы решений является дерево решений (ДР), позволяющее взглянуть на процесс условного выбора с позиции схемы. Дерево решений для вышерассмотренного примера приведено на рис. 4.1

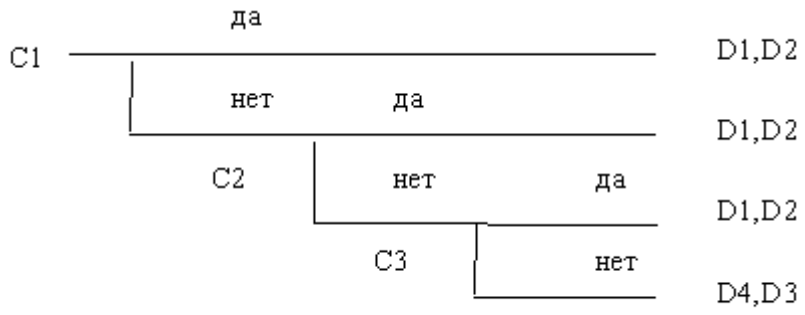


Рис. 4.1. **Дерево решений**

Обычно ДР используется при малом числе действий и когда не все комбинации условий возможны, а ТР - при большом числе действий и когда возможно большинство комбинаций условий.

Визуальные языки проектирования спецификаций

Визуальные языки проектирования являются относительно новой, оригинальной методикой разработки спецификаций процесса. Они базируются на основных идеях структурного программирования и позволяют определять потоки управления с помощью специальных иерархически организованных схем.

Одним из наиболее известных подходов к визуальному проектированию спецификаций является подход с использованием **FLOW-форм**. Каждый символ FLOW-формы имеет вид прямоугольника и может быть вписан в любой внутренний прямоугольник любого другого символа. Символы помечаются с помощью предложений на естественном языке или с использованием математической нотации.

Символы FLOW-форм приведены на рис. 4.2. Каждый символ является блоком обработки. Каждый прямоугольник внутри любого символа также представляет собой блок обработки.

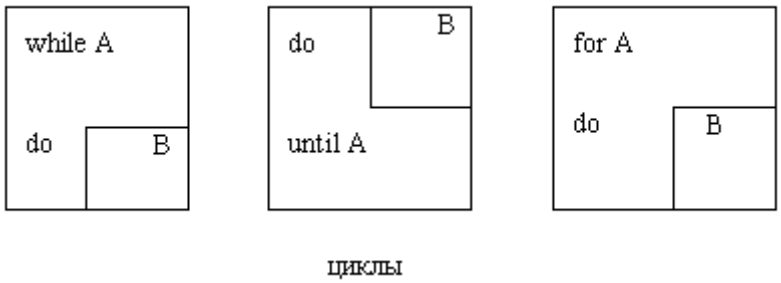


Рис. 4.2. Символы FLOW-форм.

На рис 4.3. приведен пример использования данного подхода при проектировании спецификации процесса, обеспечивающего упорядочивание определенным образом элементов массива и являющегося фрагментом алгоритма сортировки методом "поплавка".

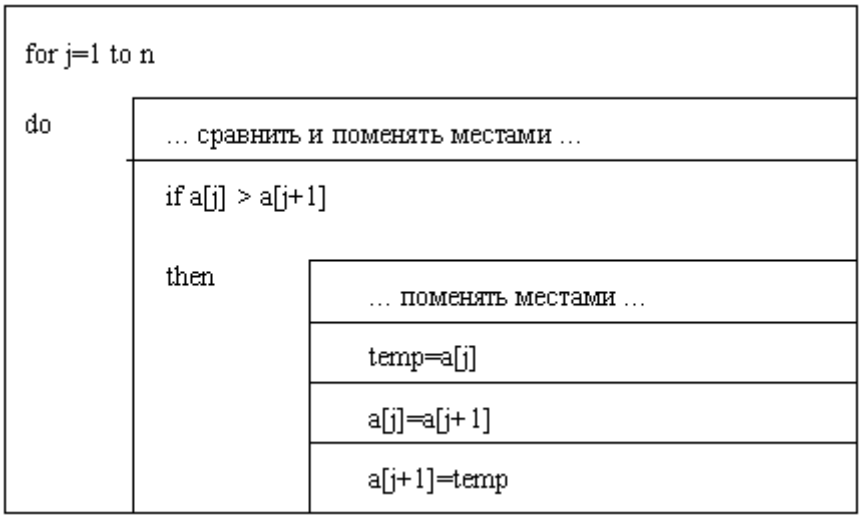


Рис. 4.3. Пример FLOW-формы

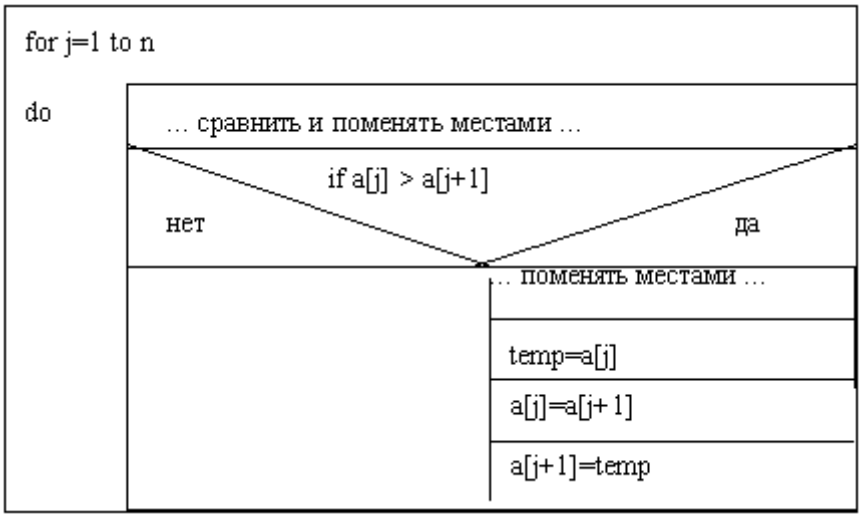


Рис. 4.4. Диаграмма Насси-Шнейдермана.

Дальнейшее развитие FLOW-формы получили в диаграммах Насси-Шнейдермана. На этих диаграммах символы последовательной обработки и цикла изображаются также, как и соответствующие символы FLOW-форм. В символах условного выбора и case-выбора собственно условие располагается в верхнем треугольнике, выбираемые варианты - на нижних сторонах треугольника, а блоки обработки - под выбираемыми вариантами. Диаграмма Насси-Шнейдермана для вышеприведенного примера изображена на рис. 4.4.

Сравнение методов

Спектр методов задания спецификаций процессов в соответствии с увеличением трудности их проектирования приведен на рис 4.5. Наиболее трудным методом задания СП являются языки программирования (C, COBOL, FORTRAN и др.). Сложность заключается в том, что языки программирования концентрируют внимание на деталях реализации, а потоки данных в DFD представляются абстрактно (их фактическая композиция определяется в словаре данных). Поэтому сложность - не в написании СП, а в их синхронизации и согласовании с DFD, поскольку при редактировании DFD, вообще говоря, должны корректироваться и спецификации процессов.

Текстовое описание	*Структурированный естественный язык*	*таблица решений*	*дерево решений*	*Визуальный язык*	*язык программирования*

Рис. 4.5. Спектр методов задания спецификаций процессов

Перечислим некоторые положительные и отрицательные стороны рассмотренных методов задания СП.

Структурированный естественный язык применяется в случаях, когда детали СП известны не полностью. Он обеспечивает быстрое проектирование СП, прост в использовании, легко понимаем проектировщиками и программистами, а также конечным пользователем. К его недостаткам относятся отсутствие процедурных возможностей и неспособность к автоматической кодогенерации из-за наличия неоднозначностей.

Таблицы и деревья решений позволяют управлять сложными комбинациями условий и действий, обеспечивают визуальное (табличное и графическое, соответственно) представление СП и легко понимаемы конечным пользователем. Кроме этого, таблицы решений позволяют легко идентифицировать несущественности и бреши в СП. Главным недостатком методов является отсутствие процедурных возможностей.

Визуальные языки проектирования поддерживаются автоматической кодогенерацией, позволяют осуществлять декомпозицию СП. Их недостаток - трудность модификации СП при изменении деталей.

Спецификации процессов для примера банковской задачи

Приведем спецификации процессов для рассмотренного в 2.5 примера банковской задачи с использованием структурированного естественного языка.

1) Спецификация процесса 1.1 приведена в 4.1.
2) Спецификация процесса 1.2.

@ВХОД *= ЛИМИТ ДЕНЕГ*
@ВХОД *= ЗАПРОС НА ОБСЛУЖИВАНИЕ*
@ВЫХОД *= ДЕНЕЖНАЯ СУММА*
@ВЫХОД *= СООБЩЕНИЕ*
@ВЫХОД *= ТРЕБУЕМОЕ ОБСЛУЖИВАНИЕ*
@СПЕЦПРОЦ *1.2 ПОЛУЧИТЬ ЗАПРОС НА ОБСЛУЖИВАНИЕ*
ВЫПОЛНИТЬ *выдать СООБЩЕНИЕ клиенту по вводу запроса на обслуживание*
принять ЗАПРОС НА ОБСЛУЖИВАНИЕ
обновить данные ТРЕБУЕМОЕ ОБСЛУЖИВАНИЕ (а именно,
ЗАПРОС ДОКУМЕНТАЦИИ, ЗАПРОС ДЕНЕГ,
ЗАПРОС БАЛАНСА, ЗАПРОС НА ОПЕРАЦИЮ)
ЕСЛИ *был сделан ЗАПРОС ДЕНЕГ*
ТО *ВЫПОЛНИТЬ запросить ДЕНЕЖНУЮ СУММУ*
выдать требуемую ДЕНЕЖНУЮ СУММУ с учетом того,
что она не должно превышать ЛИМИТ ДЕНЕГ
КОНЕЦЕСЛИ
ДОТЕХПОРПОКА *запрашивается продолжение обслуживания*
или не все обслуживание было выполнено
КОНЕЦВЫПОЛНИТЬ
@ КОНЕЦ СПЕЦИФИКАЦИИ ПРОЦЕССА *1.2*

3) Спецификация процесса 1.3.
3.1) Спецификация процесса 1.3.1.

@ВХОД *= ЗАПРОС ДОКУМЕНТАЦИИ*
@ВХОД *= ДЕТАЛИ КЛИЕНТА*
@ВЫХОД *= ОБРАБОТАННАЯ ДОКУМЕНТАЦИЯ*
@СПЕЦПРОЦ *1.3.1 ОБРАБОТАТЬ ДОКУМЕНТАЦИЮ БАНКА*
По получении ЗАПРОСА ДОКУМЕНТАЦИИ выдать ОБРАБОТАННУЮ ДОКУМЕНТАЦИЮ,
содержащую ДЕТАЛИ КЛИЕНТА, КОМПЬЮТЕРУ БАНКА
@КОНЕЦ СПЕЦИФИКАЦИИ *ПРОЦЕССА 1.3.1*

3.2) Спецификация процесса 1.3.2.

@ВХОД *= ДАННЫЕ ПО БАЛАНСУ*
@ВХОД *= ЗАПРОС БАЛАНСА*
@ВХОД *= ДЕТАЛИ КЛИЕНТА*
@ВЫХОД *= ДАННЫЕ ПО ИСТОРИИ ЗАПРОСА*
@ВЫХОД *= ВЫПИСКА ПО БАЛАНСУ*
@СПЕЦПРОЦ *1.3.2 РАСПЕЧАТАТЬ БАЛАНС КЛИЕНТА*
По получении ЗАПРОСА БАЛАНСА выдать ДАННЫЕ ПО ИСТОРИИ ЗАПРОСА
Затем выдать ВЫПИСКУ ПО БАЛАНСУ, содержащую ДАННЫЕ ПО БАЛАНСУ@
КОНЕЦ СПЕЦИФИКАЦИИ ПРОЦЕССА 1.3.2

3.3) Спецификация процесса 1.3.3.

@ВХОД *= ДЕНЕЖНАЯ СУММА*
@ВХОД *= ЗАПРОС ДЕНЕГ*
@ВХОД *= ДЕТАЛИ КЛИЕНТА*
@ВЫХОД *= ДЕНЬГИ*
@ВЫХОД *= ВЫПИСКА О ДЕНЬГАХ*
@ВЫХОД *= ДЕНЕЖНАЯ СУММА*
@СПЕЦПРОЦ *1.3.3 ПРИГОТОВИТЬ ДЕНЬГИ ДЛЯ КЛИЕНТА*
По получении ЗАПРОСА ДЕНЕГ выдать ДЕНЬГИ по значению ДЕНЕЖНОЙ СУММЫ
Выдать ВЫПИСКУ О ДЕНЬГАХ, содержащую ДЕНЕЖНУЮ СУММУ
Передать КОМПЬЮТЕРУ БАНКА информацию о ДЕНЕЖНОЙ СУММЕ
@ КОНЕЦ СПЕЦИФИКАЦИИ *ПРОЦЕССА 1.3.3*

3.4) Спецификация процесса 1.3.4.

@ВХОД *= ДАННЫЕ ПО СЧЕТУ*
@ВХОД *= ЗАПРОС НА ОПЕРАЦИЮ*
@ВХОД *= ДЕТАЛИ КЛИЕНТА*
@ВЫХОД *= ДАННННЫЕ ПО ИСТОРИИ ЗАПРОСА*
@ВЫХОД *= ВЫПИСКА ПО ОПЕРАЦИИ*
@СПЕЦПРОЦ *1.3.4 РАСПЕЧАТАТЬ ОПЕРАЦИЮ КЛИЕНТА*
получении ЗАПРОСА НА ОПЕРАЦИЮ выдать ДАННЫЕ ПО ИСТОРИИ ЗАПРОСА для специфицирования ДЕТАЛЕЙ КЛИЕНТА, чтобы получить текущие ДАННЫЕ ПО СЧЕТУ
Выдать ВЫПИСКУ ПО ОПЕРАЦИИ, содержащую ДАННЫЕ ПО СЧЕТУ
@ КОНЕЦ СПЕЦИФИКАЦИИ *ПРОЦЕССА 1.3.4*

4) Спецификация процесса 1.4.

@ВХОД *= УДАЛЕННАЯ КРЕДИТНАЯ КАРТА*
@ВХОДВЫХОД *= КРЕДИТНАЯ КАРТА*
@ВЫХОД *= ДАННЫЕ КРЕДИТНОЙ КАРТЫ*
@ВЫХОД *= ВВЕДЕННАЯ КРЕДИТНАЯ КАРТА*
@СПЕЦПРОЦ *1.4 ОБРАБОТАТЬ КРЕДИТНУЮ КАРТУ*
ВЫПОЛНИТЬ считать КРЕДИТНУЮ КАРТУ
записать в хранилище ДАННЫЕ КРЕДИТНОЙ КАРТЫ
выдать управляющий поток ВВЕДЕННАЯ КРЕДИТНАЯ КАРТА
по получении управляющего потока УДАЛЕННАЯ КРЕДИТНАЯ КАРТА удалить КРЕДИТНУЮ КАРТУ
@ КОНЕЦ СПЕЦИФИКАЦИИ *ПРОЦЕССА 1.4*

2.1.2. SADT – диаграммы моделирования бизнес-процессов

История. Начало разработки диаграмм функционального моделирования относится к середине 1960-х годов, когда Дуглас Т. Росс предложил специальную технику моделирования, получившую название SADT (Structured Analysis & Design Technique). Военно-воздушные силы США использовали методику SADT в качестве части своей программы интеграции компьютерных и промышленных технологий (Integrated Computer Aided Manufacturing, ICAM) и назвали ее **IDEFO** (Icam DEFinition). Целью программы ICAM было увеличение эффективности компьютерных технологий в сфере проектирования новых средств вооружений и ведения боевых действий. Одним из результатов этих исследований являлся вывод о том, что описательные языки не эффективны для документирования и моделирования процессов функционирования сложных систем. Подобные описания на естественном языке не обеспечивают требуемого уровня непротиворечивости и полноты, имеющих доминирующее значение при решении задач моделирования.

В рамках программы ICAM было разработано несколько графических языков моделирования, которые получили следующие названия:

- Нотация IDEF0 — для документирования процессов производства и отображения информации об использовании ресурсов на каждом из этапов проектирования систем.
- Нотация IDEF1 — для документирования информации о производственном окружении систем.
- Нотация IDEF2 — для документирования поведения системы во времени.
- Нотация IDEF3 — специально для моделирования бизнес-процессов.

Нотация IDEF2 никогда не была полностью реализована. Нотация IDEF1 в 1985 году была расширена и переименована в IDEF1X. Методология IDEF-SADT, нашла применение в правительственных и коммерческих организациях, поскольку на тот период времени вполне удовлетворяла различным требованиям, предъявляемым к моделированию широкого класса систем.

В начале 1990 года специально образованная группа пользователей IDEF (IDEF Users Group) предприняла попытку создания стандарта для IDEFO и IDEF1X. Эта попытка оказалась успешной и завершилась принятием в 1993 году стандарта правительства США, известного как FIPS для данных двух технологий IDEFO и IDEF1X. В течение последующих лет этот стандарт продолжал активно развиваться и послужил основой для реализации в некоторых первых CASE-средствах.

На начальных этапах создания ИС необходимо понять как работает организация , которую необходимо автоматизировать. Для описания работы предприятия необходимо построить модель, которая должна отражать все бизнес-процессы организации. Наиболее удобным языком моделирования бизнес-процессов является IDEFO

Методология IDEF-SADT представляет собой совокупность методов, правил и процедур, предназначенных для построения функциональной модели сиетемы какой-либо предметной области. Функциональная модель SADT отображает структуру процессов функционирования системы и ее отдельных подсистем, т. е. выполняемые ими действия и связи между этими действиями. Для этой цели строятся специальные модели, которые позволяют в наглядной форме представить последовательность определенных действий. Исходными строительными блоками любой модели IDEFO процесса являются *деятельность* (activity) и *стрелки* (arrows).

Основные понятия. Рассмотрим кратко основные понятия методологии IDEF-SADT, которые используются при построении диаграмм функционального моделирования.

Деятельность представляет собой некоторое действие или набор действий, которые имеют фиксированную цель и приводят к некоторому конечному результату. Иногда деятельность называют просто *процессом*. Модели IDEFO отслеживают различные виды деятельности системы, их описание и взаимодействие с другими процессами. На диаграммах деятельность или процесс изображается прямоугольником, который называется блоком.

Стрелка служит для обозначения некоторого носителя или воздействия, которые обеспечивают перенос данных или объектов от одной деятельности к другой. Стрелки также

необходимы для описания того, что именно производит деятельность и какие ресурсы она потребляет. Это так называемые *роли* стрелок — ICOM — сокращение первых букв от названий соответствующих стрелок IDEFO. При этом различают стрелки четырех видов:

- I (Input) — вход, т. е. все, что поступает в процесс или потребляется процессом.
- C (Control) — управление или ограничения на выполнение операций процесса.
- O (Output) — выход или результат процесса.
- M (Mechanism) — механизм, который используется для выполнения процесса.

Методология IDEFO однозначно определяет, каким образом изображаются на диаграммах стрелки каждого вида ICOM. Стрелка Вход (Input) выходит из левой стороны рамки рабочего поля и входит слева в прямоугольник процесса. Стрелка Управление (Control) входит и выходит сверху. Стрелка Выход (Output) выходит из правой стороны процесса и входит в правую сторону рамки. Стрелка Механизм (Mechanism) входит в прямоугольник процесса снизу. Таким образом, базовое представление процесса на диаграммах IDEFO имеет следующий вид (рис. 2.13).



Техника построения диаграмм представляет собой главную особенность методологии IDEF-SADT. Место соединения стрелки с блоком определяет тип интерфейса. При этом все функции моделируемой системы и интерфейсы на диаграммах представляются в виде соответствующих блоков процессов и стрелок ICOM. Управляющая информация входит в блок сверху, в то время как информация, которая подвергается обработке, изображается с левой стороны блока. Результаты процесса представляются как выходы процесса и показываются с правой стороны блока. В качестве механизма может выступать человек или автоматизированная система, которые реализуют данную операцию. Соответствующий механизм на диаграмме представляется стрелкой, которая входит в блок процесса снизу.

Одной из наиболее важных особенностей методологии IDEF-SADT является постепенное введение все более детальных представлений модели системы по мере разработки отдельных диаграмм. Построение модели IDEF-SADT начинается с представления всей системы в виде простейшей диаграммы, состоящей из одного блока процесса и стрелок ICOM, служащих для изображения основных видов взаимодействия с объектами вне системы. Поскольку исходный процесс представляет всю систему как единое целое, данное представление является наиболее общим и подлежит дальнейшей декомпозиции.

ПРИМЕР. Для иллюстрации основных идей методологии IDEF-SADT рассмотрим следующий простой пример. В качестве процесса будем представлять деятельность по оформле-

нию кредита в банке. Входом данного процесса является заявка от клиента на получение кредита, а выходом — соответствующий результат, т. е. непосредственно кредит. При этом управляющими факторами являются правила оформления кредита, которые регламентируют условия получения соответствующих финансовых средств в кредит. Механизмом данного процесса является служащий банка, который уполномочен выполнить все операции по оформлению кредита. Пример исходного представления процесса оформления кредита в банке изображен на рис. 2.14.



Общие свойства модели

Для каждой SADT-модели должна быть определена цель, обозначены границы системы, и зафиксирована точка зрения.

Модель отвечает на вопросы

SADT-модель (модель структурного анализа и проектирования) дает полное, точное и адекватное описание системы, имеющее конкретное назначение. Это назначение, называемое целью модели (purpose), вытекает из формального определения модели в SADT:

M есть модель системы S, если M может быть использована для получения ответов на вопросы относительно S с точностью A.

Таким образом, целью модели является получение ответов на некоторую совокупность вопросов. Эти вопросы неявно присутствуют (подразумеваются) в процессе анализа и, следовательно, они руководят созданием модели и направляют его. Это означает, что сама модель должна будет дать ответы на эти вопросы с заданной степенью точности. Если модель отвечает не на все вопросы или ее ответы недостаточно точны, то мы говорим, что модель не достигла своей цели. Определяя модель таким образом, SADT закладывает основы практического моделирования

Познакомившись с постановкой задачи и кратким описанием процесса, автор составил список вопросов и свел этот список в одно предложение. Это предложение становится целью модели, а список вопросов сохраняется как детализация этого предложения. После завершения работы над моделью информация, содержащаяся в модели, будет отвечать на поставленные вопросы.

Модель имеет единственный субъект

Модель является некоторым толкованием системы. Поэтому субъектом моделирования служит сама система. Однако моделируемая система никогда не существует изолированно: она всегда связана с окружающей средой. Причем зачастую трудно сказать, где кончается система и начинается среда. По этой причине в методологии SADT подчеркивается необхо-

димость точного определения границ системы (scope). SADT-модель всегда ограничивает свой субъект, т.е. модель устанавливает точно, что является и что не является субъектом моделирования, описывая то, что входит в систему, и подразумевая то, что лежит за ее пределами. Ограничивая субъект, SADT-модель помогает сконцентрировать внимание именно на описываемой системе и позволяет избежать включения посторонних субъектов. Вот почему мы утверждаем, что SADT-модель должна иметь единственный субъект.

У модели может быть только одна точка зрения

С определением модели тесно связана позиция, с которой наблюдается система и создается ее модель. Поскольку качество описания системы резко снижается, если оно не сфокусировано ни на чем, SADT требует, чтобы модель рассматривалась все время с одной и той же позиции.

"Точку зрения" (viewpoint) лучше всего представлять себе как место (позицию) человека или объекта, в которое надо встать, чтобы увидеть систему в действии. С этой фиксированной точки зрения можно создать согласованное описание системы так, чтобы модель не дрейфовала вокруг да около, и в ней не смешивались бы несвязанные описания.

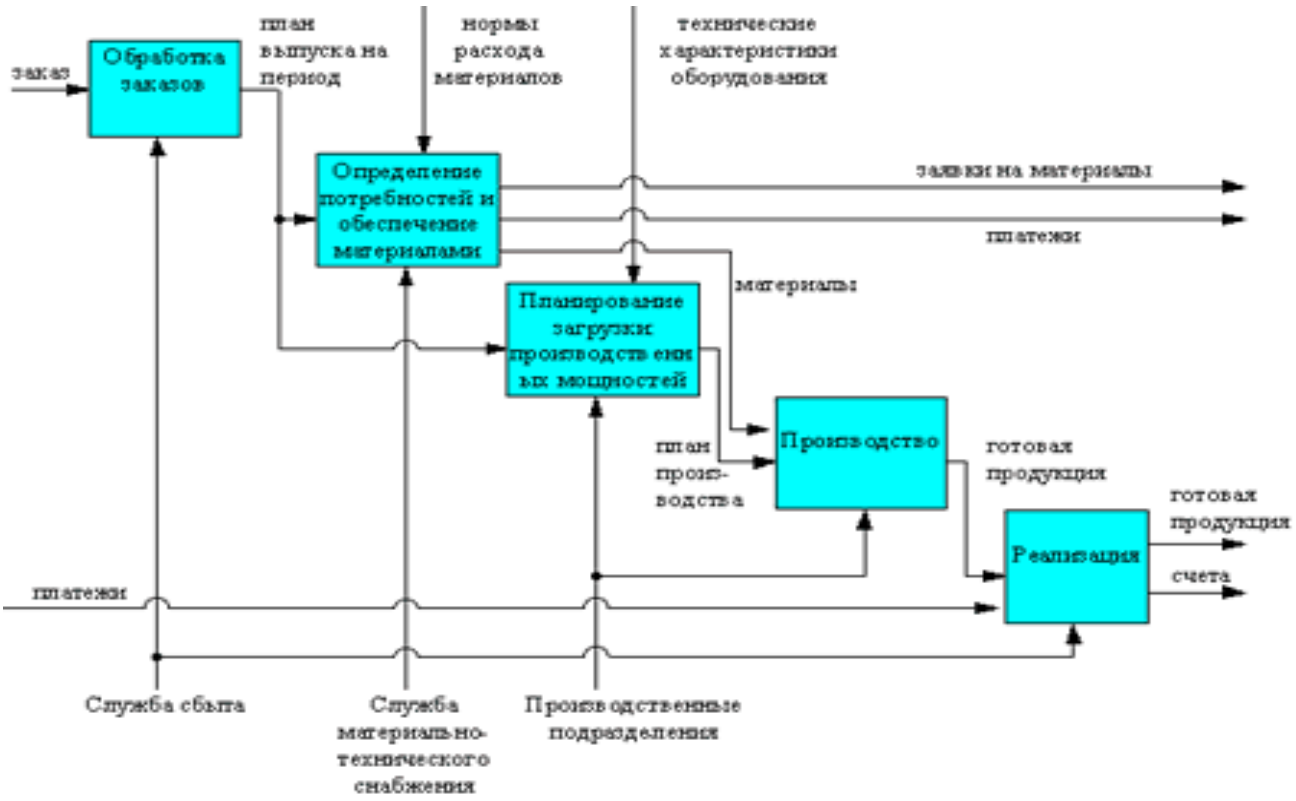
Иерархия диаграмм

В конечном итоге модель IDEF-SADT представляет собой серию иерархически взаимосвязанных диаграмм с сопроводительной документацией, которая разбивает исходное представление сложной системы на отдельные составные части. Детали каждого из основных процессов представляются в виде более детальных процессов на других диаграммах. В этом случае каждая диаграмма нижнего уровня является декомпозицией некоторого процесса из более общей диаграммы. Поэтому на каждом шаге декомпозиции более общая диаграмма конкретизируется на ряд более детальных диаграмм.

Выводы

В настоящее время диаграммы структурного системного анализа IDEF-SADT продолжают использоваться целым рядом организаций для построения и детального анализа функциональной модели существующих на предприятии бизнес-процессов, а также для разработки новых бизнес-процессов. Основной недостаток данной методологии связан с отсутствием явных средств для объектно-ориентированного представления моделей сложных систем. Хотя некоторые аналитики отмечают важность знания и применения нотации IDEF-SADT, ограниченные возможности этой методологии применительно к реализации соответствующих графических моделей в объектно-ориентированном программном коде существенно сужают диапазон решаемых с ее помощью задач.





Отличия DFD и DEF0

Подчеркнём принципиальную разницу между моделированием материальных и моделированием информационных систем. Эта разница состоит в некорректности использования DFD для моделирования материальных систем по следующим причинам:

1. В DFD - стрелки символизируют только входные и выходные информационные потоки;
2. в отличие от DFD в IDEF0 - нет ограничений на объекты, которые могут символизировать стрелки - направленная линия, состоящая из одного или более сегментов, которая представляет открытый канал или трубопровод, транспортирующий данные или объекты от источника (начало стрелки) к пользователю (головка стрелки). Уже только это обязывает - практически эксклюзивно - осуществлять моделирование материальных систем - только в IDEF0; а это - основные вопросы бизнес процесс реинжиниринга;
3. В IDEF0 - кроме стрелок входа и выхода (как и в DFD) - используются стрелки, символизирующие управление (Control Arrow) и механизмы (Mechanism Arrow);
4. Control Arrow (как и Output Arrow) - должны быть указаны в обязательном порядке для каждой Activity ФМ!;
5. В DFD управление может быть представлено только стрелкой входа "ЗАПРОС информации" на которую распространяется вышеуказанное предметное ограничение - представлять только информационный поток,
6. IDEF0, в отличие от DFD, не имеет ограничений по предмету субъекта управления (стрелка Control Arrow), в отличие от IDEF0, в DFD нет ограничений на топологическую структуру стрелок-связей; что угрожает некорректностями в формировании топологических структур связей ФМ; так как эти топологические ограничения представляют специфику материальных систем.

В таблице приведены результаты опроса, проводимого и опубликованного на сайте IDEFinfo.Ru "Какой технологии (функционального моделирования) Вы отдаёте своё предпочтение?" Опрос проводится с 13 апреля 2005 года. На 1 октября 2008 года опрошено 1292 человека.

Текущие результаты опроса, проводимого на сайте IDEFinfo.Ru

№№	Методики моделирования	Результаты опроса
1	IDEF6	554 / 42,9 %
2	UML	211 / 16,3 %
3	ARIS	160 / 12,4 %
4	Пока не определился	150 / 11,6 %
5	Не занимаюсь моделированием	73 / 5,79 %
6	Разрабатываю технологию сам	66 / 5,1 %
7	DFD	50 / 3,9 %
8	Другое	29 / 2,2%

2.1.3. IDEF3 – диаграммы потоков работ

Предназначение IDEF3

IDEF3 является стандартом документирования технологических процессов, происходящих на предприятии, и предоставляет инструментарий для наглядного исследования и моделирования их сценариев**. Сценарием (Scenario)** мы называем описание последовательности изменений свойств объекта, в рамках рассматриваемого процесса (например, описание последовательности этапов обработки детали в цеху и изменение её свойств после прохождения каждого этапа). Исполнение каждого сценария сопровождается соответствующим документооборотом, который состоит из двух основных потоков:

1. документов, определяющих структуру и последовательность процесса (технологических указаний, описаний стандартов и т.д.),
2. документов, отображающих ход его выполнения (результатов тестов и экспертиз, отчетов о браке, и т.д.).

Для эффективного управления любым процессом, необходимо иметь детальное представление об его сценарии и структуре сопутствующего документооборота. Средства документирования и моделирования IDEF3 позволяют выполнять следующие задачи:

- Документировать имеющиеся данные о технологии процесса, выявленные, скажем, в процессе опроса компетентных сотрудников, ответственных за организацию рассматриваемого процесса.
- Определять и анализировать точки влияния потоков сопутствующего документооборота на сценарий технологических процессов.
- Определять ситуации, в которых требуется принятие решения, влияющего на жизненный цикл процесса, например изменение конструктивных, технологических или эксплуатационных свойств конечного продукта.
- Содействовать принятию оптимальных решений при реорганизации технологических процессов.
- Разрабатывать имитационные модели технологических процессов, по принципу "КАК БУДЕТ, ЕСЛИ..."

Два типа диаграмм в IDEF3

Существуют два типа диаграмм в стандарте IDEF3, представляющие описание одного и того же сценария технологического процесса в разных ракурсах:.

1. диаграммы *Описания Последовательности Этапов Процесса* (Process Flow Description Diagrams, PFDD)
2. диаграммы *Состояния Объекта и его Трансформаций в Процессе* (Object State Transition Network, OSTN).

Предположим, требуется описать процесс окраски детали в производственном цеху на предприятии. С помощью диаграмм PFDD документируется последовательность и описание стадий обработки детали в рамках исследуемого технологического процесса. Диаграммы OSTN используются для иллюстрации трансформаций детали, которые происходят на каждой стадии обработки.

На следующем примере, опишем, как графические средства IDEF3 позволяют документировать вышеуказанный производственный процесс окраски детали. В целом, этот процесс состоит непосредственно из самой окраски, производимой на специальном оборудовании и

этапа контроля ее качества, который определяет, нужно ли деталь окрасить заново (в случае несоответствия стандартам и выявления брака) или отправить ее в дальнейшую обработку.

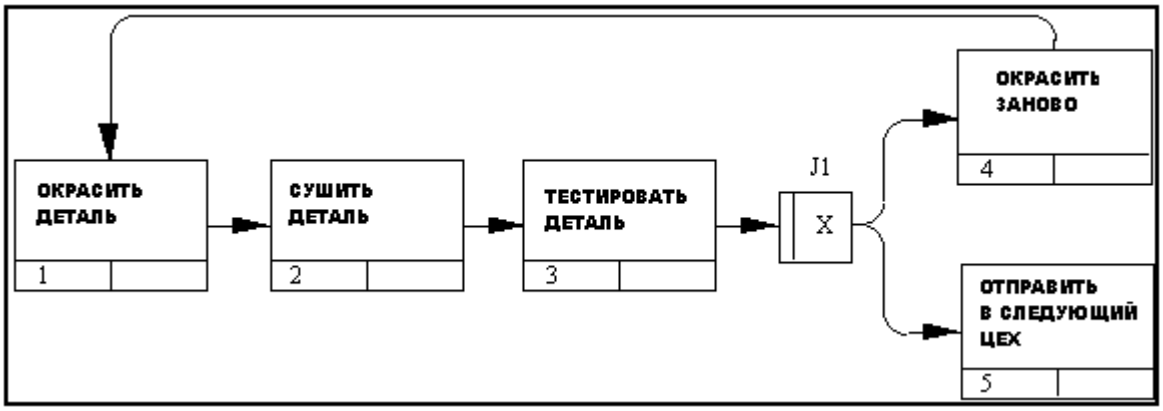


Рисунок 1. Пример PFDD диаграммы.

На рис.1 изображена диаграмма PFDD, являющаяся графическим отображение сценария обработки детали. Прямоугольники на диаграмме PFDD называются **функциональными элементами** или элементами поведения (Unit of Behavior, UOB) и обозначают событие, стадию процесса или принятие решения. Каждый UOB имеет свое имя, отображаемое в глагольном наклонении и уникальный номер. Стрелки или **линии** являются отображением перемещения детали между UOB-блоками в ходе процесса. Линии бывают следующих видов:

Обозначение	Описание
	Старшая (Precedence) - сплошная линия, связывающая UOB. Рисуется слева направо или сверху вниз. Показывает, что работа-источник должна завершиться прежде, чем работа-цель начнется
	Потоки объектов (Object Flow)- стрелка с двумя наконечниками используется для описания того факта, что объект (деталь) используется в двух или более единицах работы, например, когда объект порождается в одной работе и используется в другой.
	Отношения (Relational Link)- пунктирная линия, использующаяся для изображения связей между UOB

Объект, обозначенный J1 - называется **перекрестком** (Junction). Перекрестки используются для отображения логики взаимодействия стрелок (потоков) при слиянии и разветвлении или для отображения множества событий, которые могут или должны быть завершены перед началом следующей работы. Различают перекрестки для слияния (Fan-in Junction) и разветвления (Fan-out Junction) стрелок. Перекресток не может использоваться одновременно для слияния и для разветвления. При внесении перекрестка в диаграмму необходимо указать тип перекрестка. Классификация возможных типов перекрестков приведена в таблице.

Таблица 1.4. *Типы перекрестков*

Обозначение	Наименование	Смысл в случае слияния стрелок (Fan-in Junction)	Смысл в случае разветвления стрелок (Fan-out Junction)

&	Asynchronous AND	Все предшествующие процессы должны быть завершены	Все следующие процессы должны быть запущены
&	Synchronous AND	Все предшествующие процессы завершены одновременно	Все следующие процессы запускаются одновременно
O	Asynchronous OR	Один или несколько предшествующих процессов должныбыть завершены	Один или несколько следующих процессов должны быть запущены
O	Synchronous OR	Один или несколько предшествующих процессов завершены одновременно	Один или несколько следующих процессов запускаются одновременно
X	XOR (Exclusive OR)	Только один предшествующий процесс завершен	Только один следующий процесс запускается

Все перекрестки в PFDD диаграмме нумеруются, каждый номер имеет префикс "J".

Сценарий, отображаемый на диаграмме, можно описать в следующем виде:

Деталь поступает в окрасочный цех, подготовленной к окраске. В процессе окраски наносится один слой эмали при высокой температуре. После этого, производится сушка детали, после которой начинается этап проверки качества нанесенного слоя. Если тест подтверждает недостаточное качество нанесенного слоя (недостаточную толщину, неоднородность и т.д.), то деталь заново пропускается через цех окраски. Если деталь успешно проходит контроль качества, то она отправляется в следующий цех для дальнейшей обработки.

Каждый функциональный блок UOB может иметь последовательность **декомпозиций**, и, следовательно, может быть детализирован с любой необходимой точностью. Под декомпозицией мы понимаем представление каждого UOB с помощью отдельной IDEF3 диаграммы. Например, мы можем декомпозировать UOB "Окрасить Деталь", представив его отдельным процессом и построив для него свою PFDD диаграмму. При этом эта диаграмма будет называться **дочерней**, по отношению к изображенной на рис. 1, а та, соответственно **родительской**. Номера UOB дочерних диаграмм имеют сквозную нумерацию, т.е., если родительский UOB имеет номер "1", то блоки UOB на его декомпозиции будут соответственно иметь номера "1.1", "1.2" и т.д. Применение принципа декомпозиции в IDEF3 позволяет структурировано описывать процессы с любым требуемым уровнем детализации.

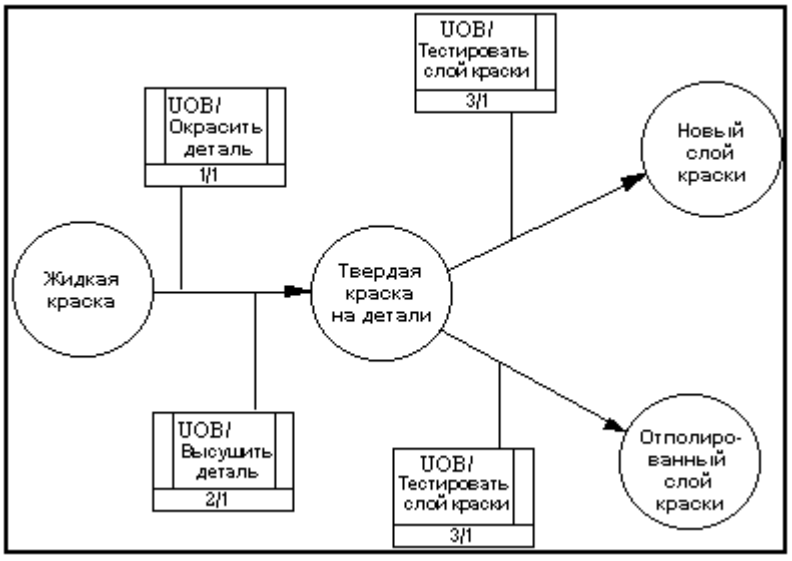


Рисунок 2. Пример OSTN диаграммы

Если диаграммы PFDD представляют технологический процесс "С точки зрения наблюдателя", то другой класс диаграмм IDEF3 OSTN позволяет рассматривать тот же самый процесс "С точки зрения объекта". На рис.2 представлено отображение процесса окраски с точки зрения OSTN диаграммы. **Состояния объекта** (в нашем случае детали) и **Изменение состояния** являются ключевыми понятиями OSTN диаграммы. Состояния объекта отображаются окружностями, а их изменения направленными линиями. Каждая линия имеет ссылку на соответствующий функциональный блок UOB, в результате которого произошло отображаемое ей изменение состояния объекта.

Объект ссылки. Объект ссылки в IDEF3 выражает некую идею, концепцию или данные, которые нельзя связать со стрелкой, перекрестком или работой (рис. 3). Объект ссылки изображается в виде прямоугольника, похожего на прямоугольник работы. В качестве имени можно использовать имя какой-либо стрелки с других диаграмм или имя сущности из модели данных. Объекты ссылки должны быть связаны с единицами работ или перекрестками пунктирными линиями. Официальная спецификация IDEF3 различает три стиля объектов ссылок - безусловные (unconditional), синхронные (synchronous) и асинхронные (asynchronous). BPwin поддерживает только безусловные объекты ссылок. Синхронные и асинхронные объекты ссылок, используемые в диаграммах переходов состояний объектов, не поддерживаются.

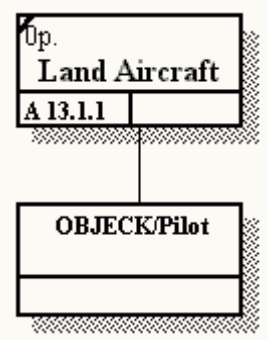


Рисунок 3. *Объект ссылки*

При внесении объектов ссылок помимо имени следует указывать тип объекта ссылки. Типы объектов ссылок приведены в табл. 1.5.

Таблица 1.5. *Типы объектов ссылок*

Тип объекта ссылки	Цель описания
OBJECT	Описывает участие важного объекта в работе
GOTO	Инструмент циклического перехода (в повторяющейся последовательности работ), возможно на текущей диаграмме, но не обязательно. Если все работы цикла присутствуют на текущей диаграмме, цикл может также изображаться стрелкой, возвращающейся на стартовую работу. GOTO может ссылаться на перекресток
UOB (Unit of behavior)	Применятся, когда необходимо подчеркнуть множественное использование какой-либо работы, но без цикла. Например, работа "Контроль качества" может быть использована в процессе "Изготовления изделия" несколько раз, после каждой единичкой операции. Обычно этот тип ссылки не используется для моделирования автоматически запускающихся работ
NOTE	Используется для документирования важной информации, относящейся к каким-либо графическим объектам на диаграмме. NOTE является альтернативой внесению текстового объекта в диаграмму
ELAB (Elaboration)	Используется для усовершенствования графиков или их более детального описания. Обычно употребляется для детального описания разветвления и слияния стрелок на перекрестках

Декомпозиция работ. В IDEF3 декомпозиция используется для детализации работ. Методология IDEF3 позволяет декомпозировать работу многократно, т.е. работа может иметь множество дочерних работ. Это позволяет в одной модели описать альтернативные потоки. Возможность множественной декомпозиции предъявляет дополнительные требования к нумерации работ. Так, номер работы состоит из номера родительской работы, версии декомпозиции и собственного номера работы на текущей диаграмме (рис. 1.54).



Рисунок 1.54. Номер единицы работы (UOW)

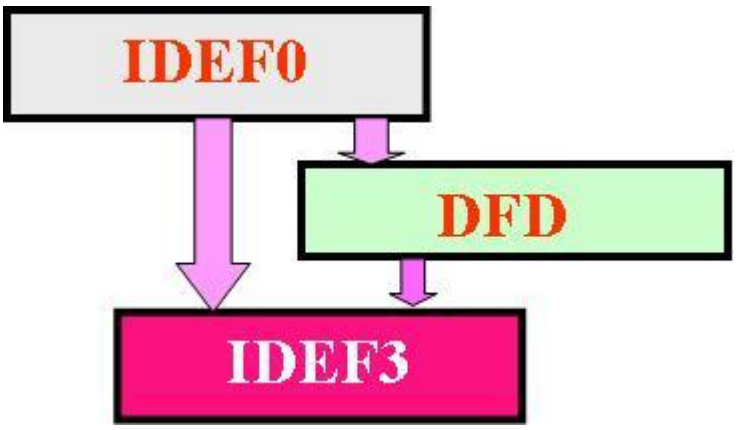
Рассмотрим процесс декомпозиции диаграмм IDEF3, включающий взаимодействие автора (аналитика) и одного или нескольких экспертов предметной области.

ВЫВОДЫ (по функциональному моделированию)

Для структурного анализа бизнес процессов в CASE-системах чаще всего используются методологии IDEF0, IDEF3 и DFD. Когда и какие методологии применять?

IDEF0 лучше всего применять как средство анализа и логического моделирования систем на ранних стадиях работы над проектом. Надо отметить, что использование SADT-моделей ограничивается только стадией анализа, поскольку они не предназначены для проектирования ПО.

Данные анализа, полученные с использованием моделирования IDEF0, обычно используются на стадии разработки моделей IDEF3 и диаграмм потоков данных DFD.



На рис отражена дисциплина взаимного расположения диаграмм в разных методиках. Показано, что IDEF0 диаграмма может быть декомпозирована либо в IDEF0 же, либо в IDEF3, либо в DFD. В свою очередь DFD может быть декомпозирована в DFD или в IDEF3. IDEF3 декомпозируется только в IDEF3 же.

2.2. Модели данных

Цель моделирования данных состоит в обеспечении разработчика ПС концептуальной схемой базы данных, которая относительно легко могут быть отображена в любую систему баз данных.

Наиболее распространенным средством моделирования данных являются диаграммы "сущность-связь" (ERD). С их помощью определяются важные для предметной области объекты (сущности), их свойства (атрибуты) и отношения друг с другом (связи). ERD непосредственно используются для проектирования реляционных баз данных.

Нотация ERD была впервые введена П. Ченом (Chen) и получила дальнейшее развитие в работах Баркера С нотацией Чена вы уже знакомы из курса «<Базы данных». Поэтому мы познакомимся с методологией IDEF1X, получившей распространение в CASE-системах.

2.2.1. Описание методологий IDEF1X и IE

Уровни моделирования данных делятся на общие модели, изображающие сущности в общем не детализированном виде, содержащем сущности важные для рассматриваемой предметной области и модели, детально изображающие взаимосвязи между сущностями предметной области в терминах конкретной СУБД, выбранной для реализации модели данных. Таким образом, на самом нижнем уровне представления модели в значительной степени зависят от выбранной реализации (СУБД) и, следовательно, модель, созданная для использования в СУБД Access будет значительно отличаться от SQL Server и тем более Oracle. Модели верхнего уровня технологически независимы и могут содержать информацию об элементах, физически не сохраняемых в БД.

Таким образом, уровни создания информационной модели можно представить следующим образом (рис.).



Рис. Уровни создания моделей данных

Модели верхнего уровня делятся на две категории.

1. ERD (*Entity Relationship Diagram*) диаграммы Сущность-связь содержат наиболее общие для рассматриваемой предметной области сущности и связи между ними.
2. KB (*Key Based) Models*) – ключевые модели (модели, основанные на ключах) устанавливают границы требований к информации предметной области и содержат все ее сущности. В KB моделях начинают проявляться детали и особенности строения данных.

Модели нижнего уровня также делятся на две категории.

1. FA (*Fully Attributed*) модель с полным набором атрибутов, все отношения которой приведены к третьей нормальной форме, содержащая все необходимые элементы для полной реализации базы данных.
2. TM (*Transformation Model*) модель, представляющая собой трансформационную модель из реляционной в модель, предназначенную для реализации в конкретной СУБД с учетом ее особенностей. TM модель в большинстве случаев не удовлетворяет условиям третьей нормальной формы, т.к. она оптимизирована для использования в рамках конкретной СУБД с учетом ее особенностей и возможностей.

В зависимости от уровня интеграции ИС физическая модель данных может быть как уровня системы в целом (глобальная модель), отображая взаимодействие информационных объектов масштаба ИС, так и уровня подсистем (локальная модель), отображая, соответственно, элементы, присущие только подсистемам ИС.

Заметим, что в идеальной ситуации должно существовать несколько локальных физических моделей, каждая из которых предназначена для своей подсистемы с возможностью взаимного использования элементов моделей.

Метод IDEF1, разработанный Т.Рэмей (T.Ramey), также основан на подходе П.Чена и позволяет построить модель данных, эквивалентную реляционной модели в третьей нормальной форме. В настоящее время на основе совершенствования методологии IDEF1 создана ее новая версия - методология IDEF1X. IDEF1X разработана с учетом таких требований, как простота изучения и возможность автоматизации. IDEF1X-диаграммы используются рядом распространенных CASE-средств (в частности, ERwin, Design/IDEF).

Типы сущностей

Сущность в методологии IDEF1X является независимой от идентификаторов или просто независимой, если каждый экземпляр сущности может быть однозначно идентифицирован без определения его отношений с другими сущностями. Сущность называется зависимой от идентификаторов или просто зависимой, если однозначная идентификация экземпляра сущности зависит от его отношения к другой сущности (рисунок 2.30).

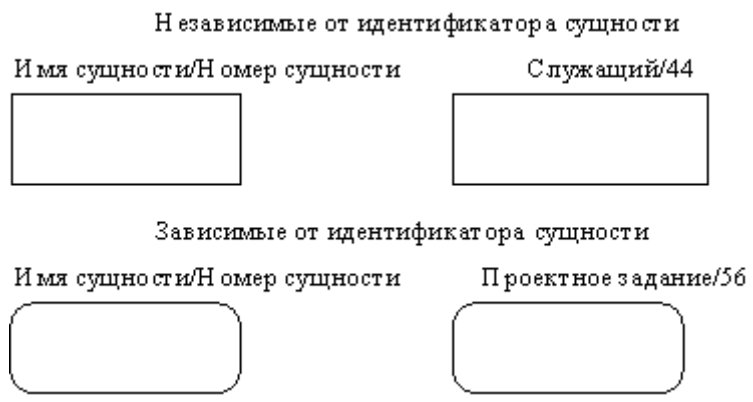


Рис. 2.30. Сущности

Каждой сущности присваивается уникальное имя и номер, разделяемые косой чертой "/" и помещаемые над блоком.

Мощность связи

Связь может дополнительно определяться с помощью указания степени или мощности (количества экземпляров сущности-потомка, которое может существовать для каждого экземпляра сущности-родителя). В IDEF1X могут быть выражены следующие мощности связей:

- каждый экземпляр сущности-родителя может иметь ноль, один или более связанных с ним экземпляров сущности-потомка;
- каждый экземпляр сущности-родителя должен иметь не менее одного связанного с ним экземпляра сущности-потомка;
- каждый экземпляр сущности-родителя должен иметь не более одного связанного с ним экземпляра сущности-потомка;
- каждый экземпляр сущности-родителя связан с некоторым фиксированным числом экземпляров сущности-потомка.

Связь изображается линией, проводимой между сущностью-родителем и сущностью-потомком с точкой на конце линии у сущности-потомка. Мощность связи обозначается, как показано на рис. 2.31 (мощность по умолчанию - N).

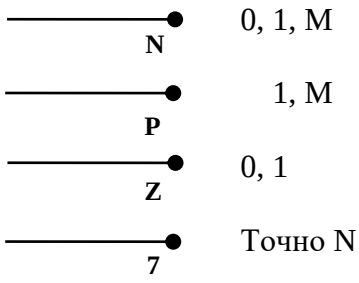


Рис. 2.31. Мощность связи

Типы связей

В IDEF1X концепция зависимых и независимых сущностей усиливается типом взаимосвязей между двумя сущностями. Различают *идентифицирующую* и *неидентифицирующую* связи.

Идентифицирующая связь между сущностью-родителем и сущностью-потомком изображается сплошной линией (рисунок 2.32). Сущность-потомок в идентифицирующей связи является зависимой от идентификатора сущностью. Сущность-родитель в идентифицирующей связи может быть как независимой, так и зависимой от идентификатора сущностью (это определяется ее связями с другими сущностями).

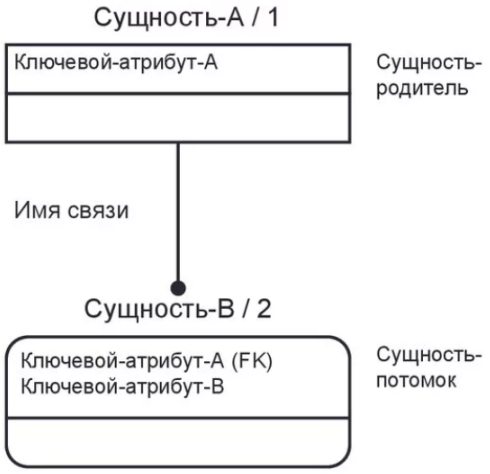


Рис. 2.32. Идентифицирующая связь

Пунктирная линия изображает неидентифицирующую связь (рисунок 2.33). Сущность-потомок в неидентифицирующей связи будет независимой от идентификатора, если она не является также сущностью-потомком в какой-либо идентифицирующей связи.

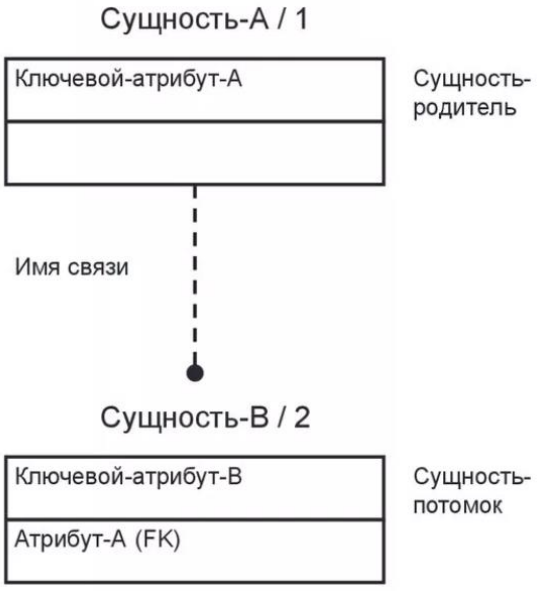


Рис. 2.33. Неидентифицирующая связь

Пример неидентифицирующей связи: (ИСПРАВИТЬ! Должен мигрировать ID отдела, а не НАЗВАНИЕ) ОТДЕЛА)

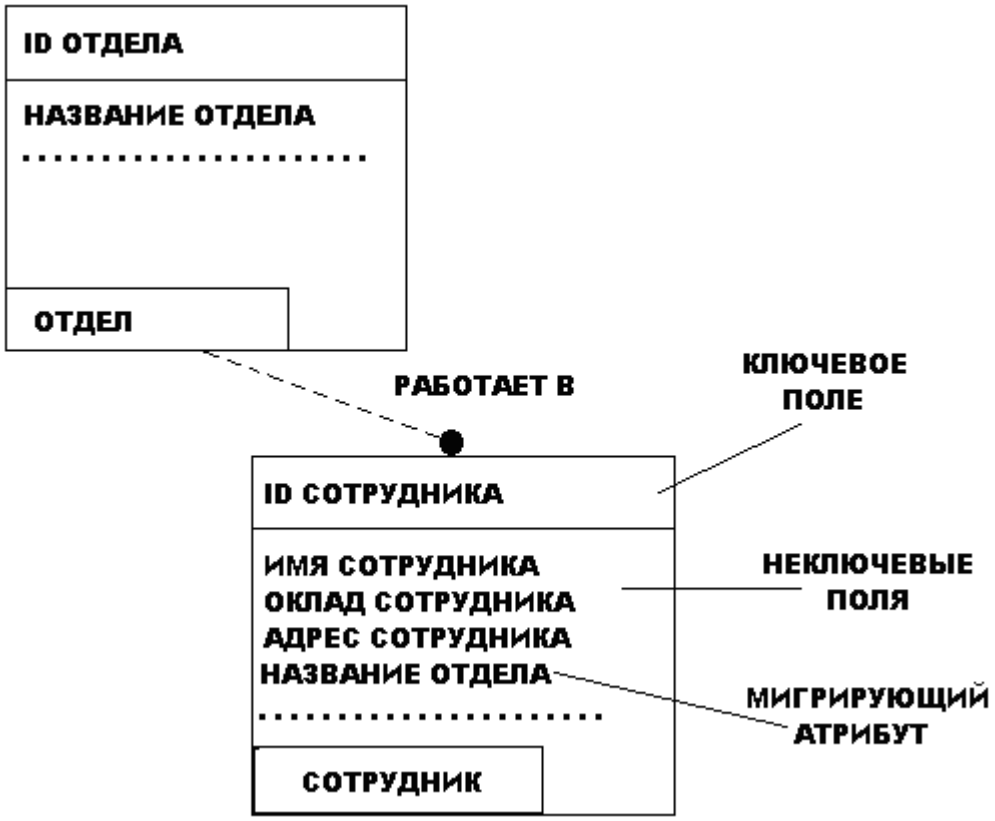


Рис. 2.34

Может ли в идентифицирующей связи FK принимать NULL-значения? Так как он входит в состав ключа, то ответ однозначный – нет. А в неидентифицирующей связи?



Рис. 2.35. Графическое представление в IDEF1X связей мощностью один-к-точно-n и один-к-диапазону-n-m

Как показывают рассмотренные выше примеры, одну и ту же информационную модель в общем случае можно разработать, используя идентифицирующие или неидентифицирую-

щие связи. Следует обратить внимание, что применение идентифицирующих связей приводит к появлению составных идентификаторов в дочерних сущностях (см. например рис. 2.35). Составные идентификаторы при работе с реальными базами данных обрабатывать сложнее, чем простые. Их использование приводит к снижению эффективности работы с базами данных. Поэтому при разработке информационных моделей предметной области предпочтение по возможности следует отдавать неидентифицирующим связям перед идентифицирующими.

Неспецифические связи

Неспецифической связью (неконкретной, неопределенной, non-specific) называется связь, при которой экземпляр каждой сущности может быть связан с некоторым количеством экземпляров другой сущности. Неспецифическая связь называется также связью многие-ко-многим. Мощность неспецифической связи в общем случае равна ноль-один или много-к-нулю- одному-или-многим. Графически неспецифическая связь изображается сплошной линией с точками на обоих концах.

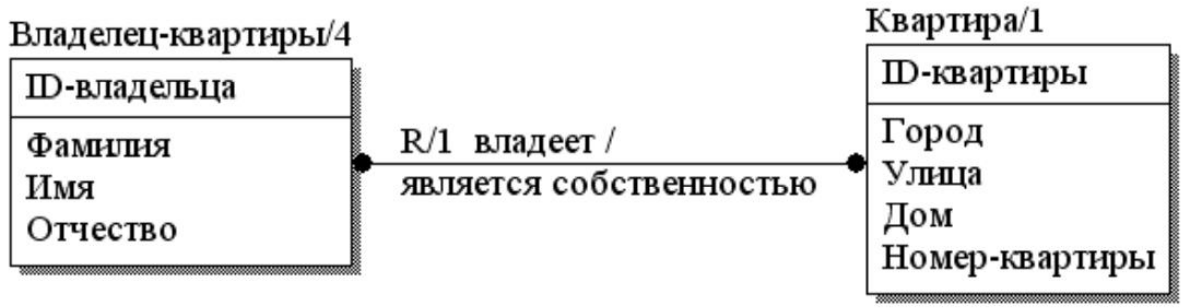


Рис. 2.36. Графическое представление в IDEF1X неспецифической связи

На рис. 2.36 приведен пример неспецифической связи между сущностями

Владелец-квартиры и Квартира. Между данными сущностями существует связь

многие-ко-многим, поскольку каждый экземпляр сущности Владелец-квартиры

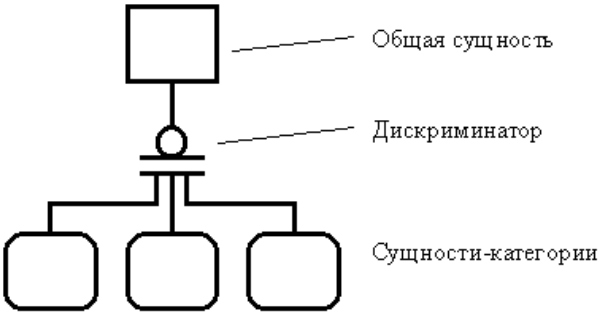
может владеть несколькими квартирами, и каждый экземпляр сущности Квартира может являться собственностью нескольких владельцев.

Неспецифические связи между сущностями могут быть полезны в начале работы над информационной моделью, когда сложно определить конкретные связи между экземплярами сущностей. На более поздних этапах разработки модели каждую неспецифическую связь следует заменять на пару специфических соединительных связей.

С этой целью создается третья сущность, представляющая собой общую дочернюю сущность, связанную соединительными связями с двумя исходными сущностями. Сущности, вводимые для устранения неспецифических связей, называются ***ассоциативными***. Ассоциативные сущности содержат идентификаторы каждого из участвующих в связи экземпляров исходных сущностей, что позволяет формализовать в данных связи многие-ко-многим, устранить избыточность информационной модели, а следовательно, и соответствующей базы данных

Связи категоризации в IDEF1X

В IDEF1X вводится понятие “отношение категоризации”, по смыслу эквивалентное рассмотренной нами иерархической связи. Отношение полной категоризации (сущности-категории составляют полное множество потомков родительской сущности) обозначается:



Также может существовать отношение неполной категоризации (сущности-категории составляют неполное множество потомков общей сущности):

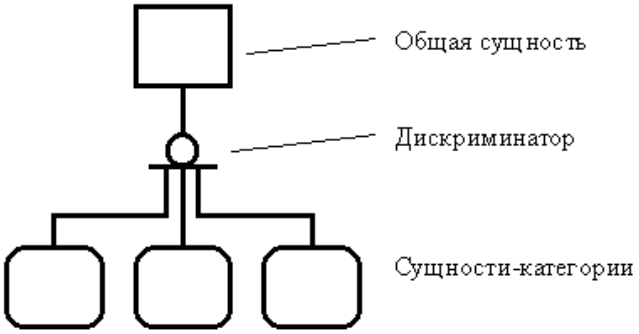




Рис. 5.43. Связь категоризации. Полная группа категорий

Обязательные и необязательные неидентифицирующие связи

Неидентифицирующая связь называется обязательной (***mandatory***), если каждый экземпляр дочерней сущности связан точно с одним экземпляром родительской сущности. Таким образом, каждый экземпляр дочерней сущности участвует в связи.

Следовательно, обязательная неидентифицирующая связь определяет безусловность связи со стороны дочерней сущности

Например, сущности Муж и Жена на рис. 5.32 связаны обязательной неидентифицирующей связью, поскольку каждая жена обязательно имеет одного мужа. Дочерняя мощность связи равна точно 1. Поэтому связь между сущностями Муж и Жена является безусловной с обеих сторон и имеет мощность один-к-одному.

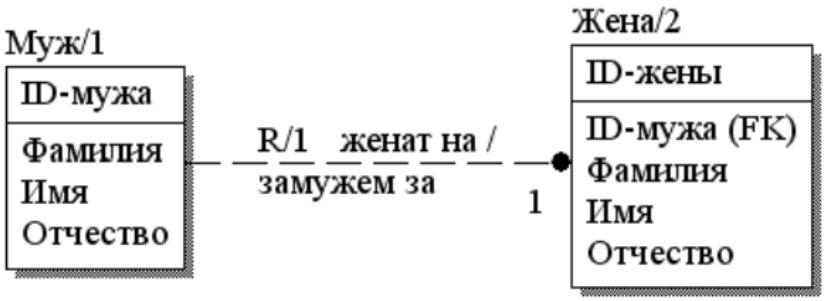


Рис. 5.32. Графическое представление в IDEF1X обязательной неидентифицирующей связи мощностью один-к-одному

Неидентифицирующая связь называется необязательной (***optional***), если каждый экземпляр дочерней сущности может быть связан с **нулем** или одним экземпляром родительской сущности. В данном случае конкретные экземпляры дочерних сущностей могут существовать без связи с конкретными экземплярами родительской сущности и существуют экземпляры дочерней сущности, не участвующие в связи.

Графически необязательная неидентифицирующая связь изображается пунктирной линией с ромбом со стороны родительской сущности.

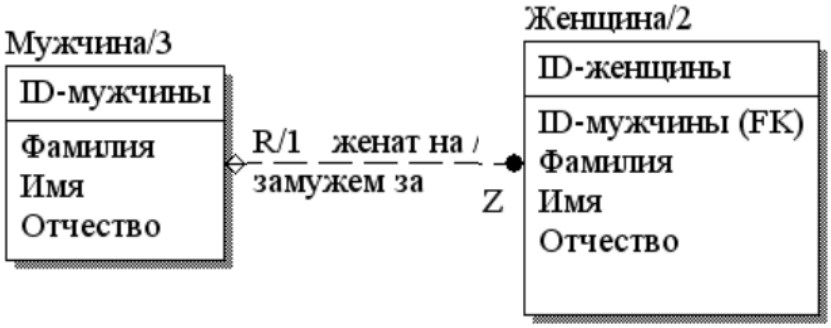


Рис. 5.33. Графическое представление в IDEF1X необязательной неидентифицирующей связи мощностью ноль-или-один-к-нулю-или-одному

Ключи

Атрибуты изображаются в виде списка имен внутри блока сущности. Атрибуты, определяющие первичный ключ, размещаются наверху списка и отделяются от других атрибутов горизонтальной чертой (рисунок 2.37).

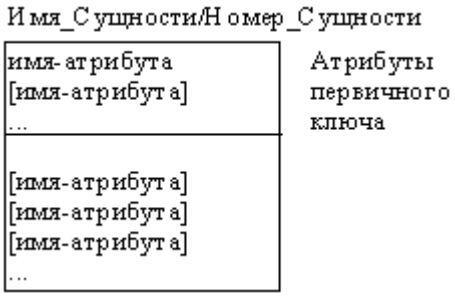


Рис. 2.37. Атрибуты и первичные ключи

При выборе первичного ключа для сущности, разработчики модели часто используют дополнительный (суррогатный) ключ, т.е. произвольный номер, который уникальным образом определяет запись в сущности. Атрибут "Номер сотрудника" является примером суррогатного ключа. **Суррогатный ключ** лучше всего подходит на роль первичного ключа потому, что является коротким и быстрее всего идентифицирует экземпляры в объекте. К тому же суррогатные ключи могут автоматически генерироваться системой так, чтобы нумерация была сплошной, т.е. без пропусков.

Сущности могут иметь также ***внешние ключи*** (*Foreign Key* – ***FK***), которые могут использоваться в качестве части или целого первичного ключа или неключевого атрибута. Внешний ключ изображается с помощью помещения внутрь блока сущности имен атрибутов, после которых следуют буквы FK в скобках (рисунок 2.38).

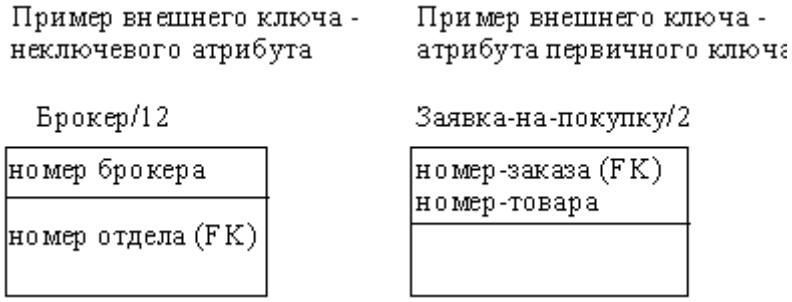


Рис. 2.38. Примеры внешних ключей

Потенциальные ключи, которые не выбраны первичными, могут быть использованы в качестве вторичных или ***альтернативных ключей*** (**AK**). С помощью альтернативных ключей часто отображают различные индексы доступа к данным в конечной реализации реляционной базы.

2.2.2. Нотация IDEF1X

Обозначения сущностей:

Элемент диаграммы	Обозначает

имя	независимая сущность
имя	зависимая сущность

Список атрибутов приводится внутри прямоугольника, обозначающего сущность. Атрибуты, составляющие ключ сущности, группируются в верхней части прямоугольника и отделяются горизонтальной чертой.

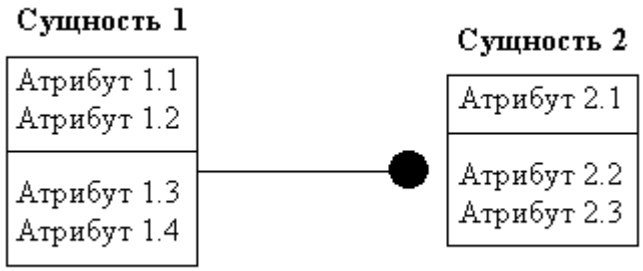
Обозначения связей:

Элемент диаграммы	Обозначает
——————	идентифицирующая связь
- - - - - - - - - - - -	неидентифицирующая связь

Обозначение кардинальности связей:

Элемент диаграммы	Обозначает
——————	1
——●—	0,1,M
——●— Z	0,1
——●— P	1,M
——●— N	точно *N* (*N* - произвольное число)

Пример:



2.2.2. Расширенные функции ERWin

Прямое и обратное проектирование

Обратное проектирование, то есть восстановление информационной модели по существующей базе данных используется при выборе оптимальной платформы для существующей настольной базы данных или базы данных центральной ЭВМ (mainframe).

После завершения процесса восстановления модели ERWin автоматически раскладывает таблицы на диаграмме. После этого можно выполнить модификацию с использованием логической схемы. По завершении изменений можно синхронизировать изменения

Синхронизация с базой данных

В процессе разработки информационной системы может возникнуть ситуация, когда структура базы данных и информационная модель не соответствуют друг другу. Предоставляется возможность привести в соответствие. Для этого предусматривается функция синхронизации с базой данных. После подключения СУБД предлагается список несоответствий между существующей структурой данных и моделью.

Программирование триггеров и процедур

ERWin реализует собственный макроязык для подготовки прототипов триггеров и процедур. Схема использования прототипов заключается в подготовке шаблона для различных типов триггеров. Например, триггер, реализующий логику каскадного удаления.

ON DELETE CASCADE

Базовые шаблоны встроены в ERWin, но пользователь может определить свои собственные шаблоны и использовать их вместо стандартных.

Макроязык шаблонов реализует большое количество макросимволов, ссылающихся на различные объекты базы данных.

%Action расширяется в UPDATE, INSERT, DELETE

%ForEachAtt(<таблица>, <разделитель>)

{<код макрокоманды>} - циклическое выполнение функции над каждым атрибутом таблицы

%ForEachEntity(){} - циклическое выполнение функции над всеми таблицами

%If %else - операторы условного выполнения

Рассмотрим шаблон триггера для организации каскадного удаления

```
%Action
   delete %Child
      from %Child, deleted
      where
         /*%%join FKPK(%Child, "=", "and")*/
         %join FKPK(%Child, deleted, "=", "and")
DELETE
delete salary_track
   from salary_track, deleted
   where
      salary_track, empl_id=deleted empl_id
```

Все макрофункции, которые могут использовать триггеры, могут использоваться также в процедурах

Правила и начальные значения

В ERWin поддерживается 2 типа правил:

- проверка допустимых значений
- проверка начальных значений

И то и другое может быть указано как со стороны клиента, так и со стороны сервера

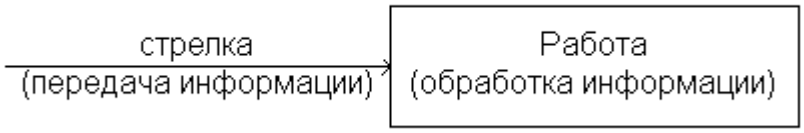
Домены

Часто используемые комбинации свойств можно именовать (дать имя). Такая комбинация свойств называется доменом, может наследоваться.

Например, можно определить домен DATA для отображения всех колонок с датами для отображения в одном стиле. Домен DATA родителя ребенка наследует все атрибуты домена и может вносить свои.

Связывание данных

- входные/выходные данные
- обработка данных

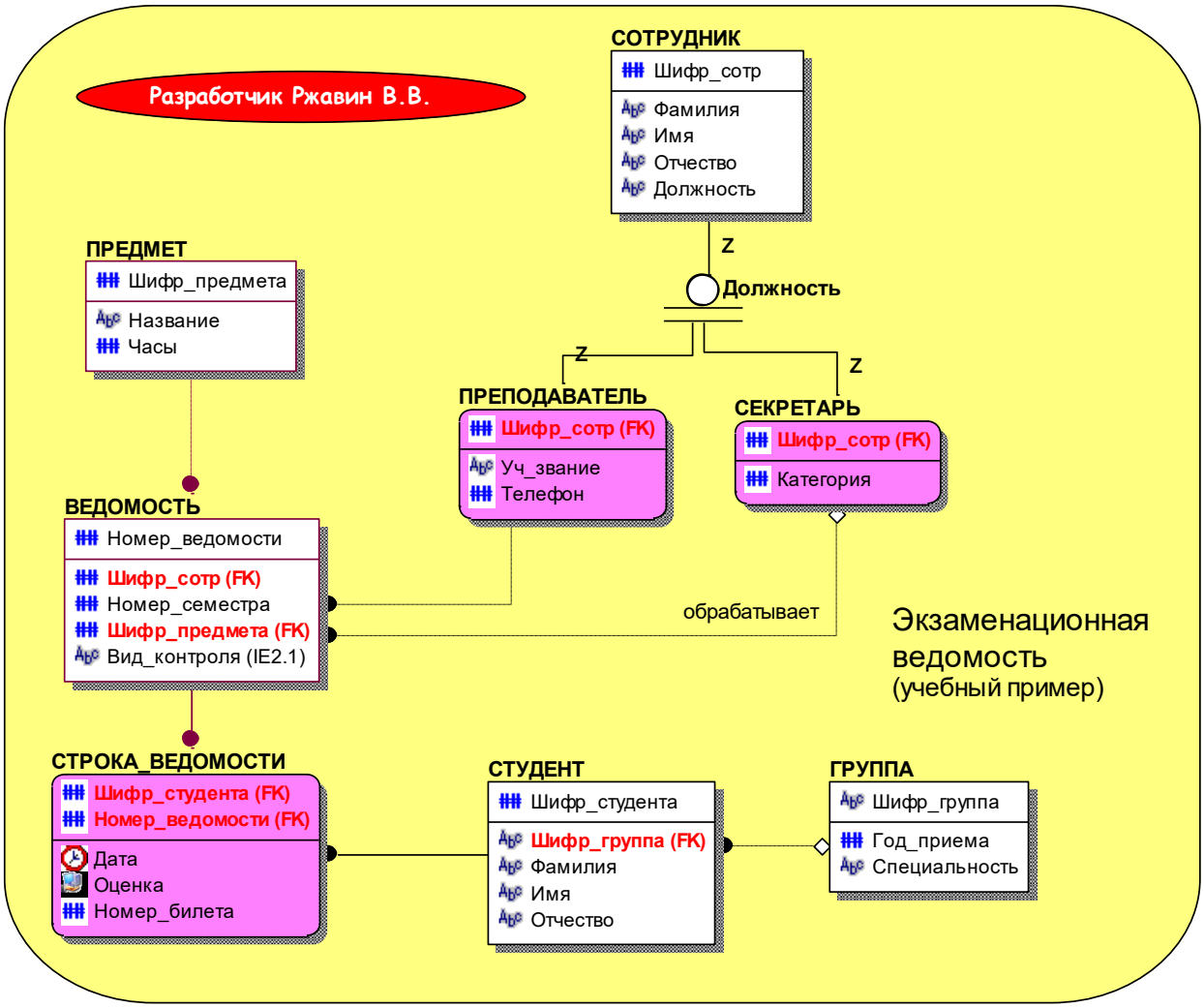


Модель экспортируется в BPWin и будет известна структура данных

Спецификации CRUD и IRUN

CRUD (Create Read Update Delete) – ассоциация для сущности

IRUN (Insert Read Update Nullify) – для атрибутов


Разработчик Ржавин В.В.
СОТРУДНИК
Шифр_сотр
Фамилия
Имя
Отчество
Должность
Z
Должность
Z
Z
ПРЕДМЕТ
Шифр_предмета
Название
Часы
ПРЕПОДАВАТЕЛЬ
Шифр_сотр (FK)
Уч_звание
Телефон
СЕКРЕТАРЬ
Шифр_сотр (FK)
Категория
ВЕДОМОСТЬ
Номер_ведомости
Шифр_сотр (FK)
Номер_семестра
Шифр_предмета (FK)
Вид_контроля (IE2.1)
обрабатывает
Экзаменационная ведомость
(учебный пример)
СТРОКА_ВЕДОМОСТИ
Шифр_студента (FK)
Номер_ведомости (FK)
Дата
Оценка
Номер_билета
СТУДЕНТ
Шифр_студента
Шифр_группа (FK)
Фамилия
Имя
Отчество
ГРУППА
Шифр_группа
Год_приема
Специальность

Пример.

2.3. Модели управления

Если *диаграммы потоков данных* отражают процессы распределения и трансформации данных, то *диаграммы потоков управления*, аналогичные DFD, выполняют ту же функцию по отношению к потокам управления. Модель управления содержит информацию о том, какие решения принимает система в ответ на изменение внешних или внутренних воздействий или режимов функционирования. Принятие решения, иначе называемого ответной реакцией системы, заключается в том, что происходит изменение состояния системы, влекущее за собой включение или выключение некоторых процессов, определенных на диаграмме потоков данных или диаграмме потоков управления. Помимо воздействий модель управления описывает, какую информацию о состоянии внешних сущностей необходимо получить системе и какие сигналы о собственном состоянии передать окружению.

Для описания управляющих воздействий в системе используются следующие средства :

- диаграмма потоков управления (Control Flow Diagram - CFD);
- спецификация управляющего процесса, определяющая управление в системе.

Ниже на рисунке показано взаимодействие модели управления с моделью обработки данных в виде классической реализации петли обратной связи.

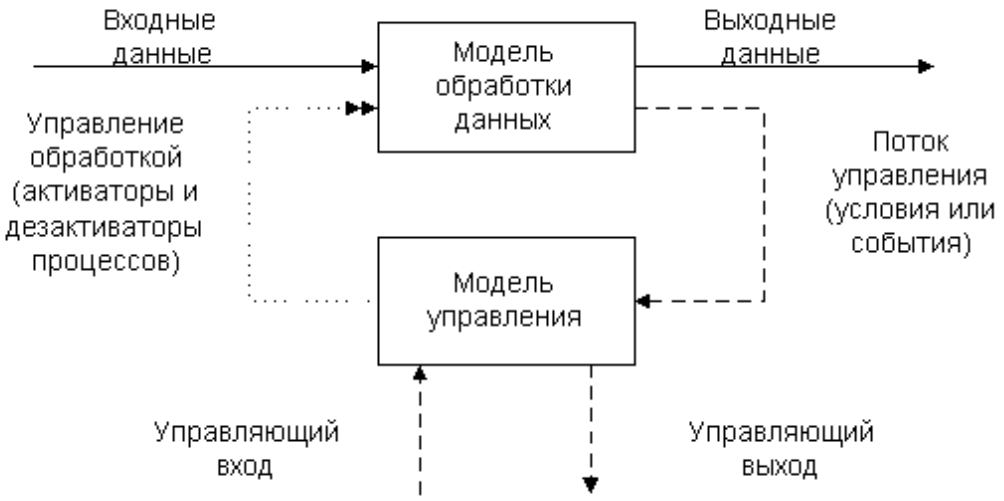


Рис. 2.8 Взаимодействие моделей обработки данных и управления

Итак, входные данные в системе перерабатываются моделью обработки данных в выходные данные, однако некоторые процессы вырабатывают еще и потоки управления (условия как некоторые логические предикаты ”обычных” потоков данных и события как изменения значений этих предикатов). В качестве управляющих входов может рассматриваться, например, переключение режимов системы, задаваемое извне.

Примечание. Основное отличие примитивного потока управления от примитивного потока данных состоит в том, что первые имеют всегда принципиально дискретные значения, а вторые в основном подразумевают непрерывную область физических значений. Хотя некоторые потоки данных в силу естественных технических ограничений уже на этапе структурного анализа могут рассматриваться как имеющие дискретные значения, сами эти значения все же обладают упорядоченностью**,** которая вовсе

не требуется при рассмотрении потоков управления (таких, как значения флагов или названия режимов).

В **модель обработки** входит набор диаграмм потоков данных (DFD) и набор спецификаций процессов.

Модель управления представляет собой некоторый аналог описания функционирования конечного автомата (старая трактовка) или описания модели состояний (более новая). Модель управления образует набор диаграмм управляющих потоков (CFD) и набор управляющих спецификаций.

По аналогии с описанием обрабатывающего процесса в виде его спецификации PSpec, описание управляющего процесса обозначается как CSpec. Описание может включать в себя такие компоненты, как:

- таблицы событий,
- таблицы решений,
- таблицы переходов в состояния,
- диаграммы переходов в состояния (STD – State Transition Diagrams).

Диаграммы переходов состояний, словарь данных (или требований) и таблицы активизации процессов в совокупности образуют модель управления системы.

Таким образом, общая ***модель требований*** к системе описывает следующие аспекты системы :

- потоки данных в системе (DFD);
- спецификации процессов обработки данных (PSPEC);
- потоки управления в системе (CFD);
- спецификации управляющих процессов в системе (STD и таблицы активизации процессов).

Взаимосвязь вышеуказанных понятий приведена на рис. 2.12.

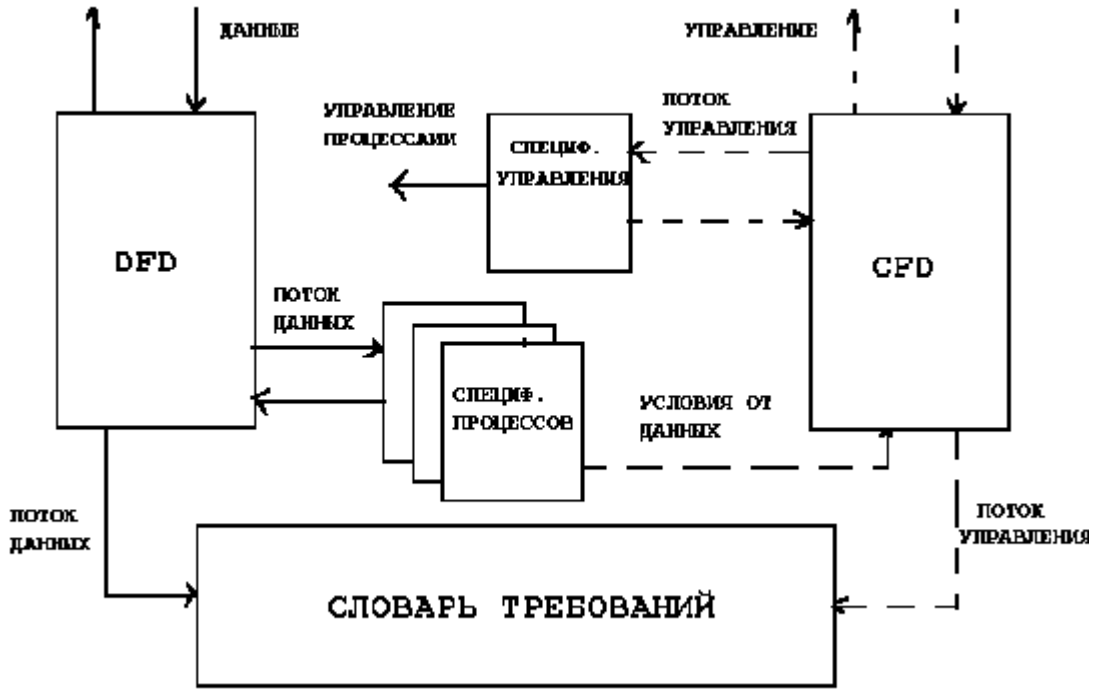


Рис.2.12 Структура модели требований

2.3.1. Диаграмма потоков управления (CFD)

Процессы, изображаемые на CFD являются в точности теми же самыми, как и на DFD, и поэтому имеют такие же номера и имена. Однако, в отличие от своих “близнецов” на DFD, эти процессы вырабатывают потоки управления в системе.

В DFD процессы взаимодействуют посредством передачи данных (сплошные стрелки), а в CFD – посредством передачи управления (пунктирные стрелки).

Стрелки показывают направление и способ передачи данных или управления (синхронный или асинхронный). Очень часто при проектировании встраиваемых систем смешивают DFD и CFD в рамках одного рисунка.

Модель управления содержит информацию о том, какие решения принимает система в ответ на изменение внешних или внутренних воздействий или режимов функционирования. Принятие решения, иначе называемого ответной реакцией системы, заключается в том, что происходит изменение состояния системы, влекущее за собой включение или выключение некоторых процессов, определенных на DFD и CFD. Помимо воздействий модель управления описывает, какую информацию о состоянии внешних сущностей необходимо получить системе и какие сигналы о собственном состоянии передать окружению.

2.3.2. Спецификации управления

Спецификации управления предназначены для моделирования и документирования аспектов систем, зависящих от времени или реакции на событие. Они позволяют осуществлять декомпозицию управляющих процессов и описывают отношения между входны-

ми и выходными управляющими потоками на управляющем процессе-предке. Для этой цели обычно используются диаграммы переходов состояний (STD).

Диаграмма переходов состояний является графической формой представления конечного автомата – математической абстракции, используемой для моделирования детерминированного поведения технических объектов или объектов реального мира.
На этапе анализа требований и определения спецификации диаграмма переходов состояний демонстрирует поведение разрабатываемой программной системы при получении управляющих воздействий. Под *управляющими воздействиями* или *сигналами* в данном случае понимают управляющую информацию, получаемую системой извне, например, управляющими воздействиями считают команды пользователя и сигналы датчиков, подключенных к компьютерной системе. Получив такое управляющее воздействие, разрабатываемая система должна выполнить определенные действия, а затем, или остаться в том же состоянии, или перейти в другое состояние, зафиксировав некоторые изменения в системе.

Для построения диаграммы переходов состояний необходимо в соответствие с тео-рией конечных автоматов определить основные состояния, управляющие воздействия (или условия перехода), выполняемые действия и возможные переходы разрабатывае-мого ПО.

Для интерактивного ПО с развитым пользовательским интерфейсом основные управляющие воздействия – команды пользователя, для ПО реального времени – сиг-налы от датчиков и/или оператора производственного процесса. Общим для этих типов ПО является наличие состояния ожидания, когда система приостанавливает работу до получения очередного управляющего воздействия (рис. 3.4). Для интерактивного ПО наиболее характерно получение команд различных типов, а для ПО реального времени – однотипных сигналов, либо от многих датчиков, либо требующих продолжительной обработки.
В отличие от интерактивных систем для систем реального времени обычно уста-новлено более жесткое ограничение на время обработки полученного сигнала. Такое ограничение часто требует выполнения дополнительных исследований поведения сис-темы во времени, например, с использованием сетей Петри или марковских процессов.

К ПО, при разработке которого требуется уточнение особенностей поведения по-средством построения диаграммы переходов состояний, относится и ПО, ориентиро-ванное на работу в сети. При этом обычно отдельно строят модели поведения сервера и клиента, представляя сообщения, передаваемые между ними в виде управляющих воздействий.

С помощью STD можно моделировать последующее функционирование системы на основе ее предыдущего и текущего функционирования. Моделируемая система в любой заданный момент времени находится точно в одном из конечного множества состояний. С течением времени она может изменить свое состояние, при этом переходы между состояниями должны быть точно определены.

STD состоит из следующих объектов:

СОСТОЯНИЕ - может рассматриваться как условие устойчивости для системы. Находясь в определенном состоянии, мы имеем достаточно информации о прошлой истории системы, чтобы определить очередное состояние в зависимости от текущих входных событий. Имя состояния должно отражать реальную ситуацию, в которой находится система, например, НАГРЕВАНИЕ, ОХЛАЖДЕНИЕ и т.п.

НАЧАЛЬНОЕ СОСТОЯНИЕ - узел STD, являющийся стартовой точкой для начального системного перехода. STD имеет только одно начальное состояние, соответствующее состоянию системы после ее инсталляции, но перед началом реальной обработки, а также любое (конечное) число завершающих состояний.

ПЕРЕХОД определяет перемещение моделируемой системы из одного состояния в другое. При этом имя перехода идентифицирует событие, являющееся причиной перехода и управляющее им. Это событие обычно состоит из управляющего потока (сигнала), возникающего как во внешнем мире, так: и внутри моделируемой системы При выполнении некоторого условия (например, СЧЕТЧИК=999 или КНОПКА НАЖАТА). Следует отметить, что не все события вызывают переходы из отдельных состояний. С другой стороны, одно и то же событие не всегда вызывает переход в то же самое состояние.

УСЛОВИЕ представляет собой событие (или события),вызывающее переход и идентифицируемое именем перехода. Если в условии участвует входной управляющий поток управляющего процесса-предка, то имя потока должно быть заключено в кавычки, например "ПАРОЛЬ"=666, где ПАРОЛЬ - входной поток.

Кроме условия с переходом может связываться действие или ряд действий, выполняющихся, когда переход имеет место. То есть действие – это операция, которая может иметь место при выполнении перехода. Если действие необходимо для выбора выходного управляющего потока то имя этого потока должно заключаться в кавычки например:

"ВВЕДЕННАЯ КАРТА" = TRUE ,
где ВВЕДЕННАЯ КАРТА - выходной поток.

Кроме того, для спецификации A-, T-, E/D-потоков (типы управляющих потоков введены в главе 2) имя запускаемого или переключаемого процесса также должно заключаться в кавычки, например:

A: "ПОЛУЧИТЬ ПАРОЛЬ" - активировать процесс ПОЛУЧИТЬ ПАРОЛЬ.

Фактически условие есть некоторое внешнее или внутреннее событие, которое система способна обнаружить и на которое она должна отреагировать определенным образом, изменяя свое состояние. При изменении состояния система обычно выполняет одно или более действий (производит ВЫВОД, выдает сообщение на терминал, выполняет вычисления). Таким образом, действие представляет собой отклик, посылаемый во внешнее окружение, или вычисление, результаты которого запоминаются в системе (обычно в хранилищах данных на DFD), для того, чтобы обеспечить реакцию на некоторые из планируемых в будущем событий.

На STD состояния представляются узлами, а переходы - дугами (рис. 6.1). Условия (по-другому называемые стимулирующими событиями) идентифицируются именем перехода и возбуждают выполнение перехода. Действия или отклики на события привязываются к переходам и записываются под соответствующим условием. Начальное состояние на диаграмме должно иметь входной переход, изображаемый потоком из подразумеваемого стартового узла (иногда этот стартовый узел изображается небольшим квадратом и привязывается к входному состоянию).

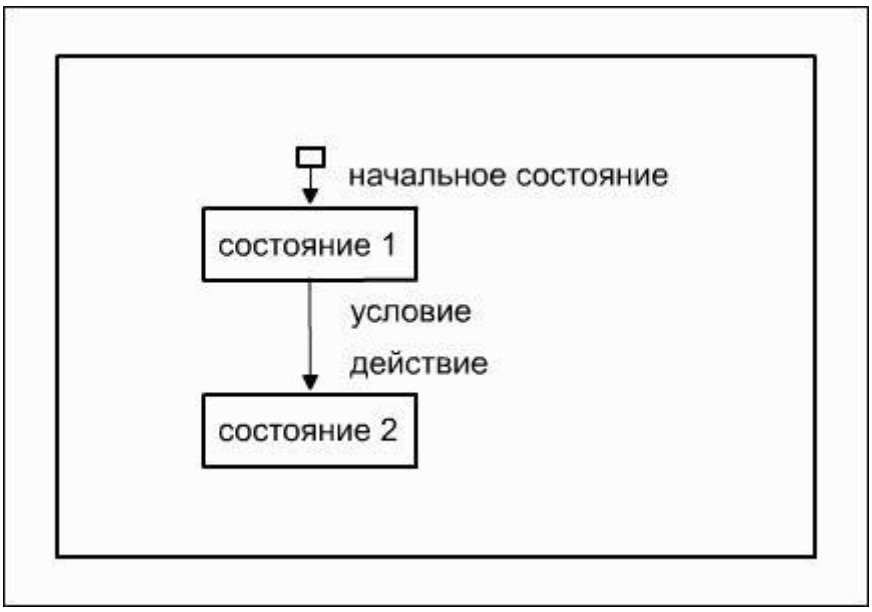


Рис. 6.1: Символы STD

Диаграмма переходов состояний для примера банковской задачи приведена на рис. 6.2. Она содержит два состояния - ОЖИДАНИЕ и ОБРАБОТКА. Переход из состояния ОЖИДАНИЕ в состояние ОБРАБОТКА осуществляется при условии ввода кредитной карты (ВВЕДЕННАЯ КРЕДИТНАЯ КАРТА). При этом выполняется действие по запуску процесса 1.1 на рис. 2.8 (ПОЛУЧИТЬ ПАРОЛЬ). Отметим, что для запуска используется А-поток, обеспечивающий непрерывность процесса 1.1, т.е. возможность повторного ввода пароля. Переход из состояния ОБРАБОТКА в состояние ОЖИДАНИЕ осуществляется двумя различными способами. При условии трехкратного ввода неверного пароля (см. спецификацию процесса 1.1) кредитная карта удаляется из системы, при этом она переходит в режим ожидания очередного клиента. При условии КОРРЕКТНЫЙ ПАРОЛЬ выполняются действия по обеспечению требуемого сервиса (последовательное включение процессов 1.2 и 1.3) и удалению кредитной карты, и затем переход в режим ожидания очередного клиента.

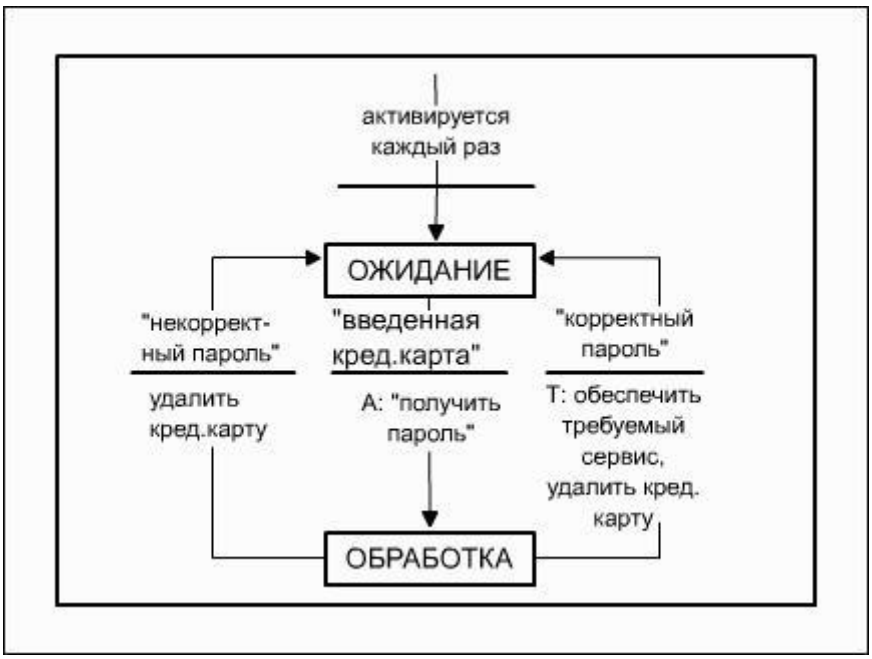


Рис. 6.2: STD для банковской задачи

При построении STD рекомендуется следовать правилам:

- строить STD на как можно более высоком уровне детализации DFD;
- строить как можно более простые STD;
- по возможности детализировать STD;
- использовать те же принципы именований состояний, событий и действий, что и при именовании процессов и потоков.

Применяются два способа построения STD. Первый способ заключается в идентификации всех возможных состояний и дальнейшем исследовании всех не бессмысленных связей (переходов) между ними. По второму способу сначала строится начальное состояние, затем следующие за ним и т.д. Результат (оба способа) - предварительная STD, для которой затем осуществляется контроль состоятельности, заключающийся в ответе на следующие вопросы:

- все ли состояния определены и имеют уникальное имя?
- все ли состояния достижимы?
- все ли состояния имеют выход?
- (для каждого состояния) реагирует ли система соответствующим образом на все возможные условия (особенно на ненормальные)?
- все ли входные (выходные) потоки управляющего процесса отражены в условиях (действиях) на STD?

В ситуации, когда число состояний и/или переходов велико, для проектирования спецификаций управления могут использоваться таблицы и матрицы переходов состояний. Обе эти нотации позволяют зафиксировать ту же самую информацию, что и диаграммы переходов состояний, но в другом формате. В качестве примера таблицы переходов состояний приведена таблица 6.1, соответствующая рассмотренной диаграмме переходов

состояний (рис. 6.2). Первая колонка таблицы содержит список всех состояний проектируемой системы, во второй колонке для каждого состояния приведены все условия, вызывающие переходы в другие состояния, а в третьей колонке – совершаемые при этих переходах действия. Четвертая колонка содержит соответствующие имена состояний, в которые осуществляется переход из рассматриваемого состояния при выполнении определенного условия.

Таблица 6.1

Текущее состояние	Условие	Действие	Следующее состояние
начальное состояние	активируется каждый раз		ОЖИДАНИЕ
ОЖИДАНИЕ	введенная кред.карта	получить пароль	ОБРАБОТКА
ОБРАБОТКА	некорректный пароль	удалить кред.карту	ОЖИДАНИЕ
ОБРАБОТКА	корректный пароль	обеспечить треб.сервис удалить кред.карту	ОЖИДАНИЕ

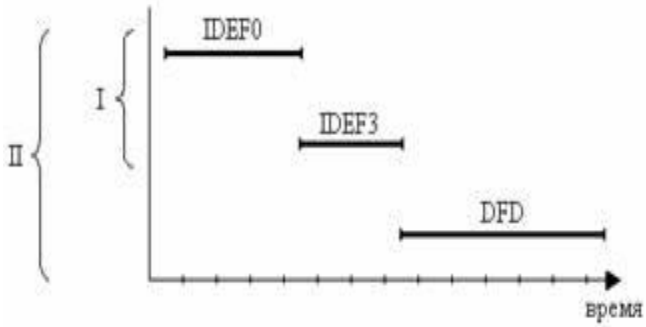
Матрица переходов состояний содержит по вертикали перечень состояний системы, а по горизонтали список условий. Каждый ее элемент содержит список действий, а также имя состояния, в которое осуществляется переход.

Используется и другой вариант данной нотации: по вертикали указываются состояния, из которых осуществляется переход, а по горизонтали - состояния, в которые осуществляется переход. При этом каждый элемент матрицы содержит соответствующие условия и действия, обеспечивающие переход из "вертикального" состояния в "горизонтальное".

Общий порядок построения модели требований

Построение CASE-моделей целесообразно выполнять в следующем порядке**:**

❶ По результатам обследования предметной области строится **модель функциональной декомпозиции** (**FDD**) объекта. На этом уровне **определяются все функции,** которые выполняет объект, **и процессы**, протекающие в объекте (например, подразделениях предприятия) для выполнения функций, определяются **связи** между функциями, между процессами. **Функция** - преобразование входных потоков в выходные, осуществляемое в соответствии с некоторыми внутренними правилами. Выполнение функции обеспечивает процесс. **Процесс** - совокупность взаимосвязанных **действий (работ)**, преобразующих некоторые входные данные в выходные. Каждый процесс характеризуется определенными задачами и методами их решения, исходными данными, полученными от других процессов, и результатами. На этапе разработки технического задания приводится текстовое **описание назначения системы** в документе "Техническое задание", в разделе "Назначение и цели создания системы", подразделе "Назначение системы". Функциональные диаграммы модели являются предварительным этапом для построения диаграмм потоков данных (DFD) и позволяют получить **общее представление** о распределении функций и процессов.



(I - этап "Обследование объекта и обоснование необходимости создания АС"; II - этап "Разработка и утверждение технического задания на создание АС")

❷ По функциональной модели (FDD) предметной области строится совокупность (иерархия) **диаграмм потоков данных** (**DFD**). Диаграммы потоков данных, используя функции, описанные на уровне функциональной модели (FDD), **позволяют детализировать описание предметной области** за счет введения накопителей, потоков данных и внешних сущностей. **Накопитель (хранилище) данных** - приспособление для хранения информации, обладающее возможностью записи и извлечения данных. Способы доступа и хранения данных в накопителях в ходе анализа не уточняются. Хранилища являются прообразами файлов или баз данных. **Поток данных** - канал передачи данных от источника к приемнику. В качестве источников и приемников данных для потоков могут выступать внешние сущности, процессы и накопители. **Внешняя сущность** - объект, являющийся поставщиком и/или получателем информации. Например, «заказчик», «банк» и т.д. Внешние сущности обозначают источники и приемники, которые не представляют для анализа интерес в данный момент и служат для ограничения моделируемой части предметной области. Отражают взаимодействие системы с внешним миром.

❸ Создается **словарь данных** (**Data Dictionary**), в котором хранится и анализируется состав потоков и накопителей данных, взаимосвязь отдельных элементов потоков и накопителей данных. Например, при моделировании документооборота вводятся сведения о структуре и реквизитном составе документов.

❹ Каждая логическая функция (процесс, действие, работа) может быть детализирована с помощью диаграмм потоков данных нижнего уровня, возможно с переходом на нотацию IDEF 3. Когда дальнейшая детализация перестает быть полезной, переходят к выражению **логики функции** (процесса, действия, работы) при помощи спецификации процесса - **миниспецификации** . **Миниспецификация** — это алгоритм описания задачи, выполняемой процессом, т.е. алгоритм преобразования входных потоков данных в выходные. Для каждого процесса нижнего уровня должна существовать одна и только одна миниспецификация . Множество всех миниспецификаций является полной спецификацией системы. Миниспецификации являются базой для кодогенерации. Решение о завершении детализации процесса с помощью диаграмм потоков данных и использовании миниспецификации принимается аналитиком исходя из следующих **критериев** : процесс имеет относительно небольшое количество входных и выходных потоков данных (2-3 потока); процесс выполняет единственную логическую функцию преобразования входной информации в выходную; возможно описать логику процесса при помощи миниспецификации небольшого объема (не более 20-30 строк, т.е. до одной страницы текста).

Миниспецификация содержит:

- списки входных и выходных данных;
- номер и/или имя процесса;
- тело (описание) процесса, являющееся спецификацией алгоритма или операции, трансформирующей входные потоки данных в выходные.

Диаграммы потоков данных (DFD) моделируют функции, которые система должна выполнять, но ничего (или почти ничего) не сообщают об отношениях между данными, а также о поведении системы в зависимости от времени - для этих целей используются специальные диаграммы.

❺ Хранимые в словаре данных (в репозитории) описания каждого накопителя (хранилища) данных используются для перехода к построению модели данных в виде **диаграмм «сущность-связь»** (**ERD**). В отличие от функциональных диаграмм (FDD) и диаграмм потоков данных (DFD) диаграммы « сущность-связь » (ERD) **описывают информационное пространство**, в рамках которого реализуются процессы объекта предметной области. Выявляются и определяются элементы базы данных, в которых будут храниться данные системы. Выявляются и определяются их атрибуты и отношения. Модель данных должна быть привязана к функциональной модели: элементы модели данных и их атрибуты должны соответствовать накопителям данных.

❻ В случае наличия реального времени диаграммы потоков данных (DFD) дополняются управляющими компонентами и спецификациями управления, например в виде **диаграмм**

переходов состояний (STD). Событийная (поведенческая) модель описывает динамику функционирования, в ней фигурируют такие категории, как состояние системы, событие, переход из одного состояния в другое, условия перехода, последовательность событий. Кроме диаграмм переходов состояний могут использоваться таблицы и матрицы переходов состояний.

Перечисленные методы дают **полное описание системы** независимо от того, является ли она существующей или вновь разрабатываемой.

Так строится **модель требований**, или **логическая модель -** подробное описание того, что должна делать система, освобожденное насколько это возможно от рассмотрения путей реализации. Это дает проектировщику четкое представление о конечных результатах, которые следует достигать.

3. Проектирование

Проектирование ПО — это следующий этап ЖЦ ПО, на котором вырабатывается, **как** реализуются требования пользователя, которые порождены и зафиксированы на этапе анализа. На этапе проектирования ПО рассмотренные **модели расширяются, уточняются и дополняются диаграммами, отражающими структуру ПО.** Осуществляется построение **модели реализации**, или **физической модели** проектируемой прикладной системы (без технических подробностей).

На фазе проектирования часть пользователей принимает участие в техническом проектировании системы под руководством специалистов-разработчиков. CASE-средства используются для быстрого получения работающих прототипов приложений. Пользователи, непосредственно взаимодействуя с ними, уточняют и дополняют требования к системе, которые не были выявлены на предыдущей фазе. Более подробно рассматриваются процессы системы. Анализируется и, при необходимости, корректируется функциональная модель. Каждый процесс рассматривается детально. При необходимости для каждого элементарного процесса создается частичный прототип: экран, диалог, отчет, устраняющий неясности или неоднозначности. Определяются требования разграничения доступа к данным. На этой же фазе происходит определение набора необходимой документации.

Инвариантная часть процесса проектированиия была рассмотрена выше. Здесь мы остановимся на особенностях проектирования ПС при структурном подходе.

3.1. Задачи этапа проектирования. Модель реализации

В предыдущих главах были рассмотрены средства структурного системного анализа, применение которых позволяет построить **модель требований** (или логическую модель) системы, состоящую из множества взаимосвязанных диаграмм, текстов и словаря данных. Используемые на этом этапе диаграммные техники включают DFD, ERD, STD и спецификации процессов. Модель требований описывает, что должна делать проектируемая система без ссылок на то, как это достигается.

Проектирование - фаза ЖЦ, на которой вырабатывается реализация требований пользователя, которые порождены и зафиксированы на фазе анализа. На этом этапе осуществляется построение **модели реализации** (или физической модели), демонстрирующей, как система будет удовлетворять предъявленным к ней требованиями (без технических подробностей). Модель реализации является расширением модели требований и состоит из взаимосвязанных диаграмм (DFD, STD, ERD, структурные карты), текстов и словаря данных.

Компоненты модели реализации:

- диаграммы потоков данных (модели системных процессов);
- модель данных (определение в ER-диаграммах всех необходимых параметров для приложений);

- модель пользовательского интерфейса (иерархия диаграмм последовательности экранных форм);
- структурные схемы программ (модели межмодульных и внутримодульных взаимодействий с использованием техники структурных карт).

3.2. Функциональные модели проектирования

Функциональные модели, используемые на стадии проектирования ПО, предназначены для описания функциональной структуры проектируемой системы.

3.2.1. Модель системных процессов (С.89 Вендров)

Для DFD переход от модели бизнес-процессов организации к модели системных процессов может происходить следующим образом:

- внешние сущности на контекстной диаграмме заменяются или дополняются техническими устройствами (рабочими станциями, принтерами и т.п.)
- для каждого потока данных определяется, посредством каких технических устройств информация передается или производится;
- процессы на диаграмме нулевого уровня заменяются соответствующими процессорами – обрабатывающими устройствами. Процессорами могут быть как технические устойства (ПК, рабочие станции, серверы БД), так и служащие-исполнители;
- определяется и изображается на диаграмме тип связи между процессорами (например, локальная сеть – LAN)
- определяются задачи для каждого процессора (приложения, необходимые для работы системы), для чего стороятся соответствующие диаграммы. Определяется тип связи между задачами.
- устанавливаются ссылки между задачами и процессами диаграмм потоков данных следующих уровней.

3.2.2. Модель пользовательского интерфейса

Модель пользовательского интерфейса определяется следующим образом. На диаграммах потоков данных среди процессов нижнего уровня (т.е. тех которые не имеют детализации в виде диаграмм) выделяются интерактивные (диалоговые) и не интерактивные процессы, например, в виде списка. После этого строят диаграммы последовательности форм (**FSD** - Form Sequence Diagrams). FSD показывает, какие формы появляются в приложении и, в каком порядке, т.е. фиксируется набор и структура вызовов экранных форм. Диаграммы последовательности форм образуют иерархию, на вершине которой находится главная форма приложения, реализующего систему/подсистему. На втором уровне находятся формы, реализующие процессы нижнего уровня на диаграммах потоков данных. Показывается взаимосвязь между каждой формой и определенным процессом, взаимосвязь между каждой формой и одной или более сущностями в ER-диаграммах.

Описание экранных форм и отчетов должно содержать:

- описание назначения формы (что делает);
- данные навигации (откуда вызвана, что может вызвать сама);
- список ошибок, которые генерируются в процессе обработки формы и реакция на них;

- ограничения доступа к форме (каковы привилегии, разрешающие действия над формой и ее элементами, каковы привилегии, запрещающие эти действия).

По сути, получается прототип экранов, отчетов, диалогов. Поэтому до начала этапов проектирования и реализации необходимо определиться **со стандартом интерфейса пользователя**. Описание интерфейса пользователя включается в Техническое задание.

Правильность интерфейса **должен проверять и утверждать** только **реальный пользователь** каждого рабочего места организации-заказчика. На практике реализация интерфейса пользователя является наиболее часто изменяемой частью исходного кода. Много проблем в интерфейсах пользователей создают сами проектировщики, если они неправильно выбирают макет и не учитывают **разницы поведения системы** на макете и на реальных данных. Распространенная ошибка проектирования интерфейса - **обработка длительных процессов**, когда пользователь должен ждать ответа на запрос. Если обработка ожидания ответа не предусмотрена (хотя бы в виде элементарного сообщения «подождите, идет обработка данных»), то пользователь думает, что приложение «зависло».

Вопросы **проверки корректности ввода** информации часто решаются проектировщиками неоднозначно. Как правило, пользователь настаивает на более детальном анализе ошибки ввода данных. Если вводимые значения выбираются из справочника, то здесь проблем меньше. Если они вводятся вручную, то возникает проблема предотвращения попадания в систему некорректных данных, например элементарных опечаток.

Построение диаграмм последовательностей экранных форм выполняеися следующим образом:

- на DFD выбираются интерактивные процессы нижнего уровня
- определяется структура меню. Для этого интерактивные процессы группируются в меню (либо так же, как в модели процессов, либо другим способом – по функциональным признакам или в зависимости от принадлежнгости к определенным объектам)
- меню изображаются над формами, соответствущими интерактивным процессам, и соединяются с ними переходами в виде стрелок, направленных от меню к формам
- определяется верхняя (главная) форма приложения, связывающая все формы с меню

3.2.3. Структурные схемы программ (структурные карты)

В структурных схемах программ определяются главные модули, маршруты связи по данным и маршруты связи по управлению между модулями, основные подпрограммы внутри каждого модуля, состав и взаимосвязь элементов данных (структуры данных), спецификации форматов входных и выходных файлов.

Техника ***структурных карт*** используется на фазе проектирования для того, чтобы продемонстрировать, каким образом системные требования будут отражаться комбинацией программных структур. При этом наиболее часто применяются две техники: структурные карты Константайна (Constantine), предназначенные для описания отношений между модулями, и структурные карты Джексона (Jackson), предназначенные для описания

внутренней структуры модулей. Структурные карты позволяют развить модель требований до модели реализации. Фактически **структурное проектирование является мостом** между структурным анализом и реализацией.

В настоящее время структурные карты применяются срвнительно редко.

Проект архитектуры идентифицирует главные модули, маршруты связи по данным и управлению между модулями, основные подпрограммы внутри каждого модуля, структуры данных, спецификации форматов входных и выходных файлов. Для ключевых процессов проектные спецификации включают детали алгоритмов.

При проектировании спецификаций, как правило, соблюдается следующая последовательность шагов:

1. разделение проекта на 10-15 модулей;
2. организация иерархии модулей;
3. определение маршрутов данных между модулями;
4. определение форматов внешних файлов;
5. определение способов доступа к внешним файлам;
6. определение структур данных;
7. проектирование ключевых алгоритмов;
8. определение подпрограмм каждого модуля.

Структурные методологии предполагают трансляцию проектных спецификаций в модель реализации, которая используется при кодогенерации.

На фазе проектирования часть пользователей принимает участие в техническом проектировании системы под руководством специалистов-разработчиков. CASE-средства используются для быстрого получения работающих прототипов приложений. Пользователи, непосредственно взаимодействуя с ними, уточняют и дополняют требования к системе, которые не были выявлены на предыдущей фазе. Более подробно рассматриваются процессы системы. Анализируется и, при необходимости, корректируется функциональная модель. Каждый процесс рассматривается детально. При необходимости для каждого элементарного процесса создается частичный прототип: экран, диалог, отчет, устраняющий неясности или неоднозначности. Определяются требования разграничения доступа к данным. На этой же фазе происходит определение набора необходимой документации.

Коллективная работа по созданию программного обеспечения

http://dmtsoft.ru/un/aticle_gos_ecz_trpo_s/

Современные технологии разработки ПО рекомендуют **определять команду как три независимые административно стороны**. Каждая сторона состоит из независящих функционально ролей. Каждая роль может включать в себя несколько человек. Роли совместимы только внутри сторон.

Сторона заказчика имеет следующие ключевые роли:

- менеджер продукта
- бизнес-аналитик.

Менеджер продукта — это представитель разработчика у заказчика, он обязан представлять интересы заказчика у разработчика. Это главный источник информации, как для заказчика, так и для разработчика, его цель - полное удовлетворение потребностей заказчика в рамках выделенных ресурсов.

Бизнес-аналитик — это представитель заказчика, ответственный за создание системы. Он досконально знает бизнес-процесс **своего предприятия** и обязан предоставлять полную и достоверную информацию.

Сторона разработчика:

- менеджер программы,
- аналитик,
- программист,
- тестировщик.

Менеджер программы — это главный технический специалист проекта, он является хранителем спецификации и заинтересован в ее минимальных изменениях во время разработки.

Аналитик - формализует информацию, получаемую со стороны заказчика в виде моделей, обязан уметь выявлять закономерности и делать правильные обобщения. Анализирует и проектирует комплекс взаимосвязанных программ для реализации функций предметной области.

Программист - непосредственно кодировщик.

Тестировщик - специалист по тестированию, отвечает за соответствие системы ее спецификации.

Сторона качества:

- бизнес-аналитик
- технолог.

Бизнес-аналитик - специалист по предметной области вообще, независимо от конкретного предприятия.

Технолог - специалист по технологии, контролирующий правильность ее использования.

В реальности каждая сторона работает в противовес двум остальным и необходима для сохранения общего баланса.

Каждая роль может включать в себя несколько человек. Нужно учитывать, что при работе над большим проектом возникает **проблема взаимодействия** исполнителей между собой. Взаимодействие исполнителей друг с другом требует времени, что в определенной степени **снижает производительность труда**. Причем взаимное непонимание приводит к дополнительным затратам на последующих стадиях разработки, например, непонимание среди программистов приводит к дополнительным затратам на тестирование.

Проблема взаимодействия возникает уже при работе группы из четырех и более человек. Для управления их работой необходим руководитель.

Решением этой проблемы может стать организация **бригад**. Если рассматривать на примере программистов, то организация **бригады главного программиста**. Бригада должна включать от 3 до 7 человек. Число 10 является верхней границей численности бригады. Это обусловлено требованием максимальной управляемости коллектива.

В бригаде в зависимости от квалификации выделяют следующих специалистов. **Руководитель бригады** (например, главный программист) — это превосходно подготовленный, творчески мыслящий, наделенный организационными способностями специалист. Выполняет работу верхнего уровня, в том числе распределение работы внутри бригады. **Заместитель руководителя** бригады (например, старший программист) - поддерживает тесную связь с руководителем бригады и дорабатывает более мелкие вопросы. При обсуждении проекта заместитель должен выступать в роли оппонента руководителю бригады. **Непосредственно исполнители** (например, младшие программисты) - два или три "менее опытных" (но не "менее способных") специалистов. Выполняют работу нижнего уровня, определенную руководителем бригады.

В большом проекте в состав каждой или нескольких бригад могут входить следующие специалисты.

- **Администратор**. Отвечает за распределение времени, подбор кадров, размещение исполнителей, финансирование, поддержку связи с высшим руководством.
- **Библиотекарь.** Ведет системную библиотеку - например, библиотеку модулей, используемых в оперативном режиме. Все изменения в программные модули вносит только библиотекарь. Также ведет системную документацию - системный журнал.
- **Технический исполнитель** (документатор). Занимается написанием документации по проекту, что освобождает более квалифицированных специалистов от этой рутинной работы.
- **Секретарь**. Выполняет обычную работу секретаря.

Опыт разработки крупных ПО показывает, что для повышения эффективности работ необходимо **разбить проект на отдельные слабо связанные по данным и функциям подсистемы**. Реализация подсистем должна выполняться отдельными группами специалистов. Поэтому после первоначального определения функций, которые должно выполнять разра-

батываемое ПО, **проект распределяется между различными командами разработчиков** (рис).

При этом необходимо обеспечить координацию ведения общего проекта и исключить дублирование результатов работ каждой проектной группы, которое может возникнуть в силу наличия общих данных и функций. В случае использования CASE -средств это означает деление функциональной модели системы (диаграммы потоков данных для структурного подхода).

Результатом стадии должны быть: общая информационная модель системы; глобальные функциональные модели системы в целом и подсистем, реализуемых отдельными командами разработчиков; точно определенные интерфейсы между автономно разрабатываемыми подсистемами (те данные, которые передаются между ними); построенные прототипы экранных форм, отчетов, диалогов (должен быть определен стандарт интерфейса пользователя).

После окончания работ каждой отдельной команды разработчиков **производится постепенная интеграция** данной части системы с остальными, формируется полный программный код, выполняется тестирование совместной работы данной части приложения, а затем тестирование системы в целом.

Инструментарий технологии разработки программного обеспечения. CASE-технология создания программного обеспечения.

Процесс разработки ПО может быть **традиционным** (неавтоматизированным, ручным) и **автоматизированным.**

Неавтоматизированная разработка ПО используется преимущественно при разработке небольшого по трудоемкости и структурной сложности ПО, имеющего преимущественно прикладной характер. При разработке не требуется участие большого числа разработчиков. При нарушении этих ограничений заметно снижается производительность труда разработчиков, падает качество разработки и документации, увеличиваются трудозатраты и стоимость ПО в целом.

Автоматизированная разработка ПО возникла с необходимостью уменьшить затраты на проектные работы, сократить сроки их выполнения, создать "типовые заготовки" алгоритмов и программ, многократно тиражируемых для различных разработок, с необходимостью координации работ большого коллектива разработчиков, стандартизации алгоритмов и программ.

Инструментарий технологии разработки представляет собой **программные продукты поддержки** (обеспечения) **технологии разработки ПО**.

1) **Средства для создания приложений**:

а) **локальные средства**, обеспечивающие выполнение отдельных работ по созданию программ: языки и системы программирования (компилятор, отладчик, документатор исходного кода, система поддержки и управления проектом, т.д.); инструментальная среда пользователя (предоставляет библиотеки функций, процедур, макрокоманды, конструкторы экранных форм и отчетов, конструкторы меню, генераторы приложений и др.);

б) **интегрированные среды разработчиков** программ, обеспечивающие выполнение комплекса взаимосвязанных работ по созданию программ (являются дальнейшим развитием локальных средств разработки).

2) **Средства CASE-технологии**, автоматизирующие весь технологический процесс анализа, проектирования, разработки и сопровождения сложных программных систем, главным образом ИС. **CASE - Computer Aided Software / System Engineering -** разработка ПО с помощью компьютера, автоматизированная разработка ПО. Это направление сформировалось в конце 80-х гг.

Средства CASE-технологии делятся на две группы:

1. **встроенные** в систему реализации - все решения по проектированию и реализации привязаны к выбранной системе управления базами данных;
2. **независимые** от **системы** реализации - все решения по проектированию ориентированы на унификацию начальных этапов жизненного цикла и средств их документирования, обеспечивают большую гибкость в выборе средств реализации.

Существует ряд признаков и свойств, наличие которых позволяет классифицировать некоторый продукт как CASE-средство. Одним из ключевых признаков является поддержка методологий структурного либо объектно-ориентированного анализа и проектирования. Примерами средств поддержки CASE-технологий являются программные продукты BPwin , ER win , OOwin .

Автоматизированный подход требует технического и программного "перевооружения" труда самих разработчиков (мощных компьютеров, дорогостоящего программного инструментария, а также повышения квалификации разработчиков и т.п.).

CASE-технология создания ПО

CASE-технология представляет собой совокупность методов разработки ПО и набор инструментальных средств, позволяющих в наглядной форме моделировать предметную область, анализировать эту модель на всех стадиях разработки и сопровождения ПО и разрабатывать приложения в соответствии с информационными потребностями пользователей. CASE-технология позволяет не только создавать "правильные" продукты, но и обеспечить "правильный" процесс их создания. При использовании CASE-технологий изменяются все

этапы ЖЦ ПО , при этом наибольшие изменения касаются этапов анализа и проектирования, происходит перераспределение затрат на создание ПО (рис).

Некоторые CASE-технологии ориентированы только на системных проектировщиков: предоставляют специальные графические средства для изображения различного вида моделей, при этом для описания моделей используются графы, диаграммы, таблицы и схемы (таким образом, обеспечивается строгое и наглядное описание проектируемой системы, которое начинается с ее общего обзора и затем детализируется, приобретая иерархическую структуру с все большим числом уровней); выполняют автоматизированное проектирование спецификаций программ (задание основных характеристик для разработки программ); ведут словарь данных.

Другой класс CASE-технологий поддерживает только разработку программ, включая: **автоматическую генерацию кодов** программ на основании их спецификаций (выполняется двумя способами - создание каркаса программ и создание полного продукта. Каркас служит для последующего ручного варианта редактирования исходных текстов, обеспечивая возможность вмешательства программиста; полный продукт не редактируется вручную); **проверку корректности** описания моделей; **документирование программ** согласно принятым стандартам и актуальному состоянию проекта; **тестирование и отладку программ** .

В рамках CASE-технологий проект сопровождается целиком, а не только его программные коды. Проектные материалы служат заданием программистам, а само программирование сводится к кодированию, если не предусмотрена автоматическая кодогенерация.

Основная **цель CASE-технологий - отделить** проектирование ПО от его кодирования и последующих этапов разработки ПО. Большинство CASE-технологий использует метод "прототипов" для быстрого создания программ на ранних этапах разработки. Основное достоинство CASE-технологии - поддержка коллективной работы над проектом за счет возможности работы в локальной сети разработчиков, экспорта/импорта любых фрагментов проекта, организационного управления проектом. CASE-технологии успешно применяются для построения практически всех типов систем ПО, однако, устойчивое положение они занимают в следующих областях: обеспечение разработки делового и коммерческого ПО . В этой прикладной области CASE-технологии применяются не только для разработки ПО, но и для создания моделей систем, помогающих коммерческим структурам решать задачи стратегического планирования, управления финансами, определения политики фирм, обучения персонала и др. Это направление получило свое собственное название - бизнес-анализ; разработка системного и управляющего ПО. Активное применение CASE-технологий связано с большой сложностью данной проблематики и со стремлением повысить эффективность работ.

Принципы построения, структура и технология использования систем автоматизированного проектирования и разработки программного обеспечения.

Состав, структура и функциональные особенности CASE-средств.

Интегрированный CASE-пакет содержит группу совместимых CASE -средств, обычно одной фирмы. CASE-пакет состоит из четырех основных компонент:

1. **Средства централизованного хранения всей информации о проектируемом ПО** в течении всего ЖЦ (репозитарий). Являются основой CASE-пакета. Соответствующая БД должна иметь возможность поддерживать большую систему описаний и характеристик и предусматривать надежные меры по защите от ошибок и потерь информации. Репозитарий должен обеспечивать:

- инкрементный режим при вводе описаний объектов;
- распространение действия нового или скорректированного описания объекта на информационное пространство всего проекта;
- синхронизацию поступления информации от различных пользователей; хранение версий проекта и его отдельных компонентов;
- сборку любой запрошенной версии; контроль информации на корректность, полноту и состоятельность.

2. **Средства ввода**. Предназначены для ввода данных в репозитарий , а также для организации взаимодействия с CASE-пакетом. Эти средства должны поддерживать различные методологии и использоваться на всем ЖЦ разными категориями разработчиков: аналитиками, проектировщиками, инженерами, администраторами и т.д.

3. **Средства анализа, проектирования и разработки**. Предназначены для того, чтобы обеспечить планирование и анализ различных описаний, а также их преобразования в процессе разработки.

4. **Средства вывода.** Служат для документирования, управления проектом и кодовой генерации.

Все перечисленные компоненты в совокупности должны:

1. поддерживать графические модели;
2. контролировать ошибки;
3. организовывать и поддерживать репозитарий;
4. поддерживать процесс проектирования и разработки.

Поддержка графических моделей

Графическая ориентация заключается в том, что программы представляются схематическими проектами и формами, которые много проще в использовании, чем многостраничные

описания. Для представления программ применяются структурные диаграммы различных типов, дополнительное достоинство которых заключается в их использовании в качестве наглядной "двумерной" документации по проекту. Для CASE существенны 4 типа диаграмм : диаграммы функционального проектирования (для этих целей наиболее часто употребляются DFD - диаграммы потоков данных); диаграммы моделирования данных (как правило, ERD - диаграммы "сущность-связь"); диаграммы моделирования поведения (как правило, STD - диаграммы переходов состояний); структурные диаграммы (карты), применяющиеся на этапе проектирования и описывающие отношения между модулями и внутримодульную структуру.

Создание и модификация подобных диаграмм осуществляется с помощью специальных графических редакторов (диаграммеров), являющихся сервисными средствами на этапах анализа требований и проектирования спецификаций. Современные диаграммеры обеспечивают: создание иерархически связанных диаграмм, в которых комбинируются графические и текстовые объекты; создание и редактирование объектов в любом месте диаграммы; создание, перемещение и выравнивание групп объектов, изменение их размеров, масштабирование; сохранение связей между объектами при их перемещении и изменении размеров; автоматический контроль ошибок и др.

Полученные диаграммы дают ясное понимание и решение проблемы, позволяют проанализировать функционирование создаваемого ПО, (фиксируют связи между разработчиками, пользователями и руководителями), обеспечивают стандартизацию представления структуры программы и данных.

Контроль ошибок

CASE обеспечивает автоматическую верификацию и контроль проекта на полноту и состоятельность на ранних этапах ЖЦ, что влияет на успех разработки в целом. В CASE диаграммеры и верификаторы способны осуществлять следующие типы контроля:

1. *Контроль синтаксиса диаграмм и типов их элементов.* Обычно такой контроль осуществляется при вводе и редактировании элементов диаграмм. Примеры контролируемых ситуаций: по синтаксису : любой функциональный элемент диаграммы должен иметь, по крайней мере, один входной и один выходной поток; два элемента данных не могут быть непосредственно связаны (например, два хранилища на DFD); по типам : функциональный элемент должен всегда использоваться для представления процедурной компоненты; поток данных всегда должен быть представлен компонентой данных.

2. *Контроль полноты и состоятельности диаграмм*: все элементы диаграмм должны быть идентифицированы и отражены в репозитарии . Например, для DFD контролируются неименованные или несвязанные потоки данных, процессы и хранилища данных; источники и стоки данных (внешние сущности) вне контекстной диаграммы; хранилища данных на контекстной диаграмме и т. п. При анализе словаря данных необходимо выявлять циклические определения, эквивалентные определения, неопределенные объекты.

3. *Контроль декомпозиции функций* включает оценку качества на основе различных метрик ПО и частичный семантический контроль. Семантика - это правила, определяющие смысл предложений.

4. *Сквозной контроль диаграмм* одного или различных типов на предмет их состоятельности по уровням - вертикальное и горизонтальное балансирование диаграмм . При вертикальном балансировании диаграмм одного типа выявляются несбалансированные потоки данных между детализируемой и детализирующей диаграммами. Горизонтальное балансирование определяет некорректности между DFD, ERD, STD, словарями данных и миниспецификациями процессов. Так при балансировании DFD - ERD контролируется соответствие каждого хранилища данных на DFD сущности или отношению на ERD. Контроль DFD - STD осуществляется по следующим правилам: каждый управляющий процесс на DFD детализируется спецификацией управления STD, и наоборот, каждой STD должен соответствовать управляющий процесс; каждое условие/действие в STD должно соответствовать входному/выходному управляющему потоку на DFD, и наоборот, каждому управляющему потоку в зависимости от его направленности должно соответствовать условие/действие на STD. При балансировании *DFD - словарь данных - миниспецификация* должны проверяться следующие правила:

- каждый поток и хранилище данных должны быть определены в словаре данных (контроль неопределенных значений), и наоборот,
- каждое определение в словаре должно быть отражено на диаграмме, в миниспецификации или другом определении (контроль неиспользуемых значений);
- каждый процесс на DFD должен детализироваться с помощью DFD или миниспецификации (но не тем и другим одновременно), и наоборот,
- каждая миниспецификация должна соответствовать единственному процессу; ссылки к данным в миниспецификациях должны соответствовать объектам на диаграммах и в словаре данных; по возможности должна контролироваться семантика миниспецификации.

Организация и поддержка репозитария

Основные функции средств организации и поддержки репозитария - хранение, доступ, обновление, анализ и визуализация всей информации по проекту ПО . Репозитарий может хранить свыше 100 типов объектов, примерами которых являются структурные диаграммы, определения экранов и меню, проекты отчетов, описания данных, логика обработки, модели данных, модели организации, модели обработки, исходные коды, элементы данных и т.п. Каждый информационный объект в репозитарии описывается перечислением его свойств: идентификатор, имена-синонимы, тип, текстовое описание, компоненты, файл-хранилище, область значений (рис).

Кроме этого, хранятся все отношения с другими объектами (например, все объекты, в которых данный объект используется; все перекрестные ссылки), правила формирования и редактирования объекта, а также контрольная информация о времени порождения объекта, времени его последнего обновления, кем и в каком проекте он был порожден, номере версии, возможности обновления и т. п.

ОПИСАНИЕ ОБЪЕКТА
ОТНОШЕНИЯ С ДРУГИМИ ОБЪЕКТАМИ
КОНТРОЛЬНАЯ ИНФОРМАЦИЯ
ПРАВИЛА ОБРАБОТКИ

Репозитарий является базой для стандартизации документов по проекту и контроля состоятельности проектных спецификаций. Все отчеты строятся автоматически по репозитарию . Основные типы отчетов :

1. Отчеты по содержимому включают сводки потоков данных и их компонент, списки входных и выходных потоков для каждого функционального блока диаграмм, списки измененных за определенный период объектов, истории всех изменений объектов, описания модулей, планы тестирования модулей и подпрограмм, списки всех данных и их атрибутов, а также отношений между их компонентами и правил их обработки.
2. Отчеты по перекрестным ссылкам включают списки всех вызывающих и вызываемых модулей; списки объектов репозитария , к которым имеет доступ конкретный разработчик; сводки диаграмм, использующих конкретные данные; маршруты движения данных от входа к выходу.
3. Отчеты по результатам анализа включают сводки балансирования диаграмм по уровням, списки неопределенных информационных объектов, списки неполных диаграмм, сводки результатов анализа структуры проекта, списки несогласованных в диаграммах и репозитарии объектов, списки неиспользуемых объектов, списки удаленных объектов.
4. Отчеты по декомпозиции объектов включают таблицы иерархии всех объектов модели.

Поддержка процесса проектирования и разработки

При поддержке процесса проектирования и разработки основную роль играют следующие возможности CASE-пакетов:

- поддержка всех этапов ЖЦ,
- поддержка прототипирования,
- поддержка структурных методологий,
- автоматическая кодогенерация .

При поддержке ЖЦ наибольшее внимание уделяется его критическим этапам - анализу требований и проектированию спецификаций. Последние являются основой всего проекта, поэтому их полнота и корректность влияют на успех разработки в целом. Важную роль при автоматизации ранних этапов ЖЦ играют возможности поддержки прототипирования. Соответствующие средства используются для определения системных требований и ответа на вопросы об ожидаемом поведении системы. Такие средства как генераторы меню, экранов и отчетов позволяют быстро построить прототипы пользовательских интерфейсов и получить модель функционирования системы с позиций конечного пользователя.

Поддержка структурных методологий осуществляется за счет средств их автоматизации, обеспечивающих: графическую поддержку построения структурных диаграмм различных типов; производство спецификаций для детализации функциональных блоков в диаграммах и структур данных на нижних уровнях; подготовку документации.

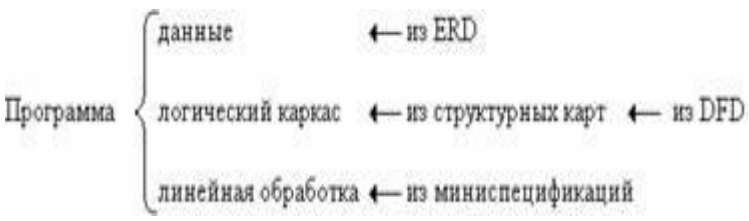
Кодогенерация осуществляется на основе репозитария и позволяет автоматически построить до 80-90% объектных кодов или текстов программ на языках высокого уровня. При этом различными CASE-пакетами поддерживаются практически все известные языки программирования.

Средства кодогенерации по отношению к полноте целевого продукта разделяются на средства генерации каркаса ПО и средства генерации полного продукта.

В первом случае автоматически строится откомментированная логика (потоки управления) ПО, а также коды для БД, файлов, экранов, отчетов и т.п., остальные фрагменты ПО кодируются вручную.

Во втором случае из проектных спецификаций генерируется полная документированная программа, включая выполняемый код, пользовательскую и программную документацию, наборы тестов и т.д. Все эти компоненты полной программы связываются в единый объект, хранящийся в репозитарии для облегчения доступа и сопровождения.

Идея автоматической кодогенерации на основе модели заключается в следующем (рис).



Любая программа схематически может быть представлена в виде тройки: обрабатываемые данные, логический каркас обработки, линейные участки обработки.

Схема базы данных может быть легко сгенерирована на основании модели "сущность-связь", и современные средства информационного моделирования (например, ERwin , Designer /2000 и др.) обеспечивают такую генерацию.

Логика обработки генерируется на основе диаграмм потоков данных: известны алгоритмы автоматического преобразования иерархии DFD в структурные карты и есть CASE - средства это выполняющие, а с задачей получения из структурных карт соответствующих кодов легко справляется теория компиляции.

Наконец, линейным участкам соответствуют миниспецификации модели. И именно здесь лежит ключ к высокому проценту автоматически сгенерированного кода, существенно зависящему от метода задания миниспецификаций.

Структурированный естественный язык для этого не подходит, а на основе таблиц решений или миниспецификаций на визуальном языке проектирования, на языке программирования можно получить код автоматически.